

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Quyết Thắng

**XÂY DỰNG HỆ THỐNG WEBHOOK TRACKING
HÀNG HÓA TRONG LĨNH VỰC LOGISTIC VỚI KHẢ
NĂNG CHỊU TẢI CAO, ĐÁP ỨNG LƯỢNG
THROUGHPUT LỚN**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin

HÀ NỘI - 2023

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Quyết Thắng

**XÂY DỰNG HỆ THỐNG WEBHOOK THEO DÕI
HÀNG HÓA TRONG LĨNH VỰC LOGISTIC VỚI KHẢ
NĂNG CHỊU TẢI CAO, ĐÁP ỨNG LƯỢNG
THROUGHPUT LỚN**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: PGS.TS. Nguyễn Trí Thành

Cán bộ đồng hướng dẫn: Phan Hồng Linh

HÀ NỘI - 2023

LỜI CẢM ƠN

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành và lòng biết ơn sâu sắc tới PGS.TS. Nguyễn Trí Thành, người đã hướng dẫn tận tình và giúp đỡ tôi rất nhiều trong quá trình thực hiện khóa luận tốt nghiệp.

Tôi cũng xin chân thành gửi lời cảm ơn tới anh Phan Hồng Lĩnh đã giúp đỡ tôi rất nhiều trong suốt quá trình thực hiện khóa luận này.

Tôi cũng xin bày tỏ lòng biết ơn chân thành đối với toàn bộ các thầy giáo, cô giáo của Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội nói chung, cũng như các thầy cô trong Khoa Công nghệ thông tin của trường nói riêng. Những người đã dạy dỗ, hướng dẫn tôi trong suốt bốn năm trên giảng đường. Những kiến thức, kinh nghiệm của các thầy cô truyền tải sẽ là hành trang vững chắc để giúp tôi thành công trong cuộc sống này.

Cuối cùng, tôi xin được gửi lời cảm ơn sâu sắc đến gia đình, những người thân đã nuôi tôi khôn lớn, trưởng thành hôm nay. Cảm ơn cha, mẹ đã luôn động viên và là điểm tựa tinh thần giúp tôi vượt qua những khó khăn, thử thách.

LỜI CAM ĐOAN

Tôi là Nguyễn Quyết Thắng, sinh viên lớp K64-CB, ngành Công nghệ thông tin. Tôi xin cam đoan khóa luận “Xây dựng hệ thống Webhook tracking hàng hóa trong lĩnh vực logistic với khả năng chịu tải cao, đáp ứng lượng throughput lớn” là công trình nghiên cứu, xây dựng của bản thân cùng với sự hướng dẫn của PGS.TS. Nguyễn Trí Thành và anh Phan Hồng Lĩnh. Các nội dung nghiên cứu, kết quả trong khóa luận là trung thực.

Các thông tin sử dụng trong khóa luận là có cơ sở và không có nội dung nào sao chép từ các tài liệu mà không ghi rõ trong trích dẫn tham khảo. Tôi xin chịu trách nhiệm về lời cam đoan này.

Hà Nội, ngày 06 tháng 02 năm 2023

Sinh viên

Nguyễn Quyết Thắng

TÓM TẮT

Tóm tắt: Trong lĩnh vực vận chuyển hàng hóa hiện nay, việc theo dõi hàng hóa là một khâu rất quan trọng trong quản lý chuỗi cung ứng bởi vì nó sẽ cung cấp thông tin chi tiết về vị trí, trạng thái và các thuộc tính khác của đơn hàng cho các đối tác, khách hàng đặt hàng. Theo dõi hàng hóa giúp các nhà cung cấp dịch vụ vận chuyển quản lý đơn hàng tốt hơn, giảm rủi ro mất hàng hóa trong quá trình vận chuyển. Đối với người tiêu dùng, theo dõi đơn hàng giúp họ cập nhật trạng thái, vị trí của đơn hàng một cách chính xác và nhanh chóng nhất. Tuy nhiên, việc theo dõi hàng hóa trong một hệ thống vận chuyển lớn là một vấn đề khó. Một số giải pháp đã được đề xuất nhưng chưa đáp ứng được hết các nhu cầu từ phía doanh nghiệp. Khóa luận sẽ trình bày chi tiết về cách xây dựng một hệ thống theo dõi hàng hóa trong lĩnh vực logistic có khả năng chịu tải cao, đáp ứng được thông lượng lớn. Đồng thời, khóa luận sẽ đưa ra các giải pháp thay thế nhằm tối ưu hơn, áp dụng các công nghệ và giải thuật để xử lý từng bài toán được đặt ra.

Từ khóa: *webhook, theo dõi hàng hóa, nền tảng xử lý dữ liệu phân tán, kỹ thuật theo dõi sự thay đổi dữ liệu, xử lý dữ liệu bất đồng bộ*

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Mục lục iii

Danh sách hình vẽ vi

Danh sách bảng viii

Danh mục các từ viết tắt ix

Chương 1: Giới thiệu 1

1.1 Đặt vấn đề 1

1.2 Mục tiêu và phạm vi của khóa luận 4

1.3 Cấu trúc của khóa luận 5

Chương 2: Cơ sở lý thuyết 7

2.1 Webhook 7

2.1.1 Định nghĩa về webhook 7

2.1.2 Trường hợp sử dụng webhook 7

2.2 Message Broker 8

2.2.1 Mô hình điều hướng message 8

2.2.2 Giới thiệu về Kafka 12

2.2.3 Kiến trúc của Kafka 13

2.2.4 Mô hình replication trong Kafka 20

2.2.5 Kafka Connect 22

2.3 Change Data Capture 23

2.3.1 Khái niệm về CDC 24

2.3.2 ETL và CDC	24
2.3.3 Các phương pháp CDC	25
2.4 Concurrency	25
2.4.1 CPU bound và I/O bound	26
2.4.2 Concurrency	27
2.4.3 Sử dụng concurrency để tối ưu I/O bound	27
2.5 Docker	28
2.6 ELK stack	30
2.6.1 Logging	30
2.6.2 Định nghĩa	31
2.6.3 Luồng hoạt động của ELK stack	31
2.7 Prometheus, Grafana và Exporter	32
2.7.1 Định nghĩa	32
2.7.2 Luồng hoạt động của Prometheus, Grafana và Exporter	33
Chương 3: Giải pháp xây dựng hệ thống theo dõi đơn hàng tự động với thông lượng lớn	35
3.1 Quy trình tổng quan của một đơn hàng	35
3.2 Xây dựng hệ thống nắm bắt sự thay đổi dữ liệu từ cơ sở dữ liệu	38
3.2.1 Phân tích bài toán	38
3.2.2 Giải pháp thực hiện	40
3.2.3 Xây dựng cơ sở dữ liệu	43
3.2.4 Log-based CDC với Debezium	47
3.3 Xây dựng hệ thống xử lý webhook	53
3.3.1 Phân tích bài toán	53
3.3.2 Giải pháp thực hiện	54
3.4 Giải pháp ưu tiên message	61
3.4.1 Phân tích bài toán	61
3.4.2 Giải pháp thực hiện	62
Chương 4: Cài đặt và kết quả thực nghiệm	70
4.1 Cài đặt hệ thống	70
4.1.1 Triển khai môi trường và cài đặt hệ thống bằng Docker	70
4.2 Xác định thông lượng của hệ thống	71
4.2.1 Phân tích và thống kê response log với ELK stack	72

4.2.2 Thống kê message trong Apache Kafka với Grafana, Prometheus và Exporter	73
Kết luận	78
Tài liệu tham khảo	81

Danh sách hình vẽ

1.1 Sự khác biệt giữa API polling và webhook	2
1.2 Kịch bản mô tả hoạt động khi một đơn hàng thay đổi trạng thái . . .	5
2.1 Mô hình point-to-point messaging	9
2.2 Mô hình publish/subscribe messaging	10
2.3 Kiến trúc cơ bản của Kafka	13
2.4 Các partition được chia đều trong cụm broker	15
2.5 Các offset trong partition của một topic	16
2.6 Nhiều consumer group cùng sử dụng một topic	18
2.7 Leader và follower replica trong cụm broker	21
2.8 Mô hình sử dụng Kafka Connect	22
2.9 Mô tả về một chương trình CPU Bound	26
2.10 Mô tả về một chương trình I/O Bound	27
2.11 Sự khác biệt giữa Docker và Virtual Machine	29
2.12 Luồng hoạt động của ELK stack	32
2.13 Luồng hoạt động của Prometheus, Grafana và Exporter	34
3.1 Mô hình BPMN quy trình đơn hàng của Giao hàng tiết kiệm	36
3.2 Kiểm tra thông tin đơn hàng trên Shopee	38
3.3 Lược đồ cơ sở dữ liệu của hệ thống	46
3.4 Mô hình hệ thống nắm bắt sự thay đổi dữ liệu	48
3.5 Dữ liệu dạng JSON được Debezium tạo ra khi có sự kiện thay đổi . .	49
3.6 Luồng hoạt động tracking tài xế (người giao hàng)	53
3.7 So sánh thử nghiệm 3 module concurrency trong bài toán xử lý nhiều request	58
3.8 Mô hình hệ thống xử lý webhook	59
3.9 Biểu đồ tuần tự mô tả luồng hoạt động xử lý message và gửi lại message khi hệ thống đối tác gặp lỗi	60

3.10	Mô hình tổng quan hệ thống	61
3.11	Phân chia các topic theo độ ưu tiên khác nhau	63
3.12	Biểu đồ hoạt động của giải pháp ưu tiên message	64
3.13	Mô hình tối ưu giải pháp ưu tiên message	66
3.14	Tối ưu xử lý tính toán nhiều lần trên Redis	67
3.15	Biểu đồ hoạt động khi tối ưu giải pháp ưu tiên message	67
3.16	Mô hình tổng quan của hệ thống	69
4.1	Kiến trúc Kafka được triển khai trong hệ thống	71
4.2	Thông số phần cứng của VPS sử dụng để xác định thông lượng . . .	72
4.3	Sử dụng Kibana để trực quan hóa dữ liệu	73
4.4	Trực quan hóa dữ liệu message bằng Grafana	74
4.5	Số lượng response thành công và số message trễ trên một giây tương ứng với số request/s	75
4.6	Chỉ số CPU sử dụng tương ứng với số request/s	76
4.7	Chỉ số RAM sử dụng tương ứng với số request/s	76

Danh sách bảng

2.1 So sánh sự khác nhau giữa Message base và Data pipeline	11
3.1 Các trạng thái chuyển của đơn hàng trên hệ thống Giao hàng tiết kiệm	37
3.2 So sánh sự khác nhau giữa các phương pháp CDC	40
3.3 Thiết kế cơ sở dữ liệu bảng packages	43
3.4 Thiết kế cơ sở dữ liệu bảng shops	44
3.5 Thiết kế cơ sở dữ liệu bảng stations	44
3.6 Thiết kế cơ sở dữ liệu bảng customers	45
3.7 Thiết kế cơ sở dữ liệu bảng addresses	45
3.8 Thiết kế cơ sở dữ liệu bảng cods	46
3.9 Mô tả các trường dữ liệu của message tạo bởi Debezium	52

Danh mục các từ viết tắt

STT	Từ viết tắt	Cụm từ đầy đủ	Cụm từ tiếng Việt
1	CDC	Change Data Capture	Kỹ thuật bắt sự thay đổi dữ liệu
2	ETL	Extract, Transform, Load	Trích xuất, Biến đổi, Tải
4	ELK	Elasticsearch, Logstash, Kibana	Elasticsearch, Logstash, Kibana
5	API	Application Programming Interface	Giao diện lập trình ứng dụng
6	HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
7	JSON	JavaScript Object Notation	Ký hiệu đối tượng JavaScript
8	CPU	Central Processing Unit	Bộ xử lý trung tâm
9	RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
10	URL	Uniform Resource Locator	Định vị thống nhất tài nguyên
11	SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
12	I/O	Input/Output	Vào/Ra
13	ACID	Atomicity, Consistency, Isolation, Durability	Tính nguyên tử, tính nhất quán, tính đảm bảo, tính bền vững

Chương 1: Giới thiệu

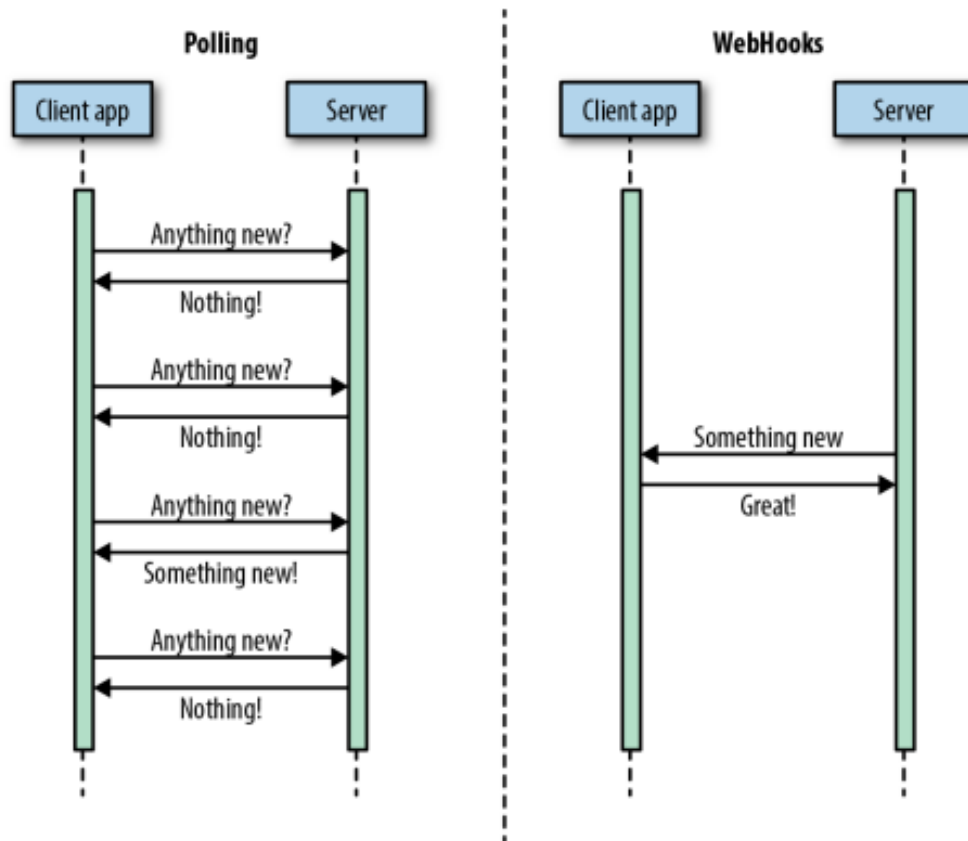
1.1 Đặt vấn đề

Hiện nay, việc mua sắm trực tuyến tại Việt Nam và các nước trên thế giới đang ngày càng trở nên phổ biến và nhận được sự quan tâm lớn của người dân, nhất là sau khi dịch bệnh Covid-19 bùng phát vì khi thực hiện giãn cách xã hội người tiêu dùng có thói quen chuyển từ mua sắm truyền thống sang mua sắm trực tuyến nhiều hơn. Cho dù dịch bệnh hiện nay đã giảm nhưng vẫn còn phần lớn khách hàng giữ thói quen mua sắm online vì tính tiện lợi của nó. Theo đó, dịch vụ vận chuyển hàng hóa cũng tăng trưởng mạnh nhờ số lượng người mua sắm trực tuyến khổng lồ. Việc theo dõi hàng hóa, đơn hàng vì thế mà cũng trở nên quan trọng. Đối với người nhận hàng, việc theo dõi hàng hóa giúp họ theo dõi, cập nhật chính xác vị trí đơn hàng của mình ở đâu hay có gặp sự cố gì trong quá trình vận chuyển không. Còn đối với người gửi hàng như các cửa hàng đăng ký dịch vụ vận chuyển, theo dõi đơn hàng giúp họ xác định rõ ràng quá trình vận chuyển đến người nhận. Đồng thời khi có sự cố xảy ra thì nhà cung cấp sẽ xử lý vấn đề một cách nhanh chóng nhờ việc lần theo dấu vết trạng thái, vị trí của đơn hàng.

Các nhà cung cấp dịch vụ vận chuyển như Giao hàng tiết kiệm, J&T Express, GHN,...sẽ là bên chịu trách nhiệm cung cấp các cách thức để khách hàng theo dõi hàng hóa dễ dàng hơn. Một trong những cách triển khai đó là cung cấp cho khách hàng một public API và họ sẽ thực hiện request đến API này liên tục để xem thông tin về đơn hàng của họ đã được cập nhật hay chưa. Cách triển khai này gọi là API polling, được nhiều nhà cung cấp dịch vụ vận chuyển sử dụng. Tuy nhiên, nếu đơn hàng chưa có sự thay đổi gì thì việc request lên hệ thống liên tục như vậy sẽ gây quá tải hệ thống cũng như lãng phí tài nguyên, nhất là đối với các công ty cung cấp dịch vụ vận chuyển lớn như Giao hàng tiết kiệm có nhiều cửa hàng và khách hàng đăng ký dịch vụ theo dõi hàng hóa.

Vậy nên, việc sử dụng API polling chỉ phù hợp cho các bên không cần cập nhật theo thời gian thực (real-time). Từ những vấn đề trên, cần tìm một giải pháp để theo dõi hàng hóa tự động và tối ưu hơn. Thay vì sử dụng API polling, chúng

ta sẽ sử dụng **webhook**. Webhook hoạt động bằng cách gửi một request đến cho một endpoint URL đã được khách hàng đưa ra trước khi có một sự kiện thay đổi. Trong ngữ cảnh vận chuyển hàng hóa, sự kiện cụ thể ở đây là khi trạng thái đơn hàng thay đổi, kho của đơn hàng thay đổi,...



Hình 1.1: Sự khác biệt giữa API polling và webhook [1]

Webhook sẽ giải quyết các vấn đề như: giảm thiểu request đến hệ thống vì bên khách hàng sẽ không cần phải request liên tục để theo dõi đơn mà hệ thống sẽ tự động gửi cho khách hàng khi có thay đổi về đơn hàng, cập nhật trạng thái của đơn hàng theo thời gian thực. Ngoài ra, việc thông báo cho bên thứ ba khi trạng thái đơn hàng thay đổi bằng cách sử dụng webhook cũng có thể là xu hướng tương lai, với mong muốn việc theo dõi đơn của khách hàng là chính xác và nhanh nhất, giảm thiểu rủi ro, không tốn quá nhiều tài nguyên.

Tuy nhiên, để hệ thống nhận biết được dữ liệu đã thay đổi để đưa dữ liệu cần cập nhật này đến đối tác thì cũng là một vấn đề mà khóa luận cần giải đáp. Một trong những phương pháp có thể được áp dụng bằng cách theo dõi dữ liệu thay

đổi ở tầng ứng dụng bằng cách hook (gọi) một hàm có khả năng theo dõi dữ liệu vào các hàm làm thay đổi thông tin đơn hàng. Ví dụ trong service Xử lý đơn hàng (Package Service) có hàm thay đổi trạng thái đơn hàng sau khi nhân viên giao hàng báo giao hàng thành công. Bằng cách hook 1 hàm nắm bắt dữ liệu nằm trong hàm này có thể giúp bắt được sự thay đổi của trạng thái đơn hàng rồi gửi thông tin trạng thái được cập nhật này cho đối tác bằng webhook. Có thể thấy phương pháp này có thể dễ dàng triển khai, không tốn nhiều thời gian để cài đặt. Nhưng phương pháp nào cũng có sự đánh đổi, nhược điểm của phương pháp này là:

- **Dễ gây lỗi:** Việc lặp lại nhiều lần có thể dẫn đến việc xảy ra lỗi nếu có sự thay đổi về chức năng hoặc thiếu cập nhật đồng bộ trong các hàm.
- **Khó bảo trì:** Khi cần thay đổi chức năng, ta phải sửa đổi nhiều hàm liên quan đến chức năng đó. Điều này làm cho việc bảo trì hệ thống trở nên khó khăn và tốn thời gian.
- **Tốn tài nguyên:** Nếu mỗi khi thay đổi trạng thái đơn hàng lại gọi đến các hàm tương tự nhau, điều này sẽ tốn nhiều tài nguyên hệ thống và có thể làm chậm tốc độ xử lý.
- **Khó mở rộng:** Nếu hệ thống cần thêm các chức năng mới liên quan đến cập nhật trạng thái đơn hàng, việc lặp lại các hàm có thể làm cho việc mở rộng hệ thống trở nên khó khăn và tốn nhiều thời gian.

Sau khi đã nhận biết được dữ liệu được thay đổi và dữ liệu này được đưa qua các module xử lý dữ liệu và cuối cùng webhook thông báo cho phía khách hàng. Một bài toán khác đặt ra là khi dữ liệu đưa vào các hệ thống xử lý dữ liệu, các hệ thống này nhận rất nhiều dữ liệu mỗi giây, tất cả các dữ liệu cần đảm bảo không bị mất và cần được xử lý bởi một hệ thống có thông lượng cao, không gây chậm trễ cho các hệ thống có liên quan. Các hệ thống có thể trao đổi dữ liệu với nhau một cách ổn định đảm bảo hai yếu tố:

- **Đảm bảo việc gửi thành công dữ liệu.** Ví dụ hệ thống bên đối tác có thể bị ngừng hoạt động, trong khoảng thời gian downtime đó, các trạng thái thông tin đơn hàng được gửi sang sẽ bị mất. Cần phải đảm bảo các dữ liệu này được gửi lại sau khi hệ thống hoạt động trở lại.

- Đảm bảo thứ tự các dữ liệu. Ví dụ một đơn hàng có trạng thái **Đang giao hàng** sẽ không thể được gửi trước trạng thái **Đã giao hàng**.

Hệ thống luôn phải ở trạng thái sẵn sàng cao và sẵn sàng nhận một yêu cầu mới thay vì bị chặn bởi quá trình xử lý các yêu cầu đã nhận trước đó. Hoặc khi khối lượng yêu cầu cần xử lý tăng lên và hệ thống cần được mở rộng, việc đảm bảo cho hệ thống không bị chậm trễ và dữ liệu đảm bảo được gửi đến bên thứ ba trong thời điểm cao tải thật sự gây khó khăn.

1.2 Mục tiêu và phạm vi của khóa luận

Từ những vấn đề được nêu trên, mục tiêu của khóa luận hướng tới là xây dựng một hệ thống hoàn toàn tự động giúp theo dõi trạng thái của đơn hàng được cập nhật theo thời gian thực cho đối tác và khách hàng của các công ty vận chuyển hàng hoá và đưa ra các giải pháp, thuật toán để tăng lượng thông lượng vì hiện tại có rất nhiều đơn hàng trong một ngày.. Trong phạm vi đề án, đối tượng hướng đến là những khách hàng, những đối tác đăng ký dịch vụ theo dõi trạng thái đơn hàng của các công ty, tập đoàn chuyên về lĩnh vực vận chuyển. Hệ thống theo dõi hàng hóa được chia làm 2 hệ thống nhỏ hơn đó là: **Hệ thống bắt sự thay đổi dữ liệu từ cơ sở dữ liệu** và **Hệ thống xử lý webhook**.

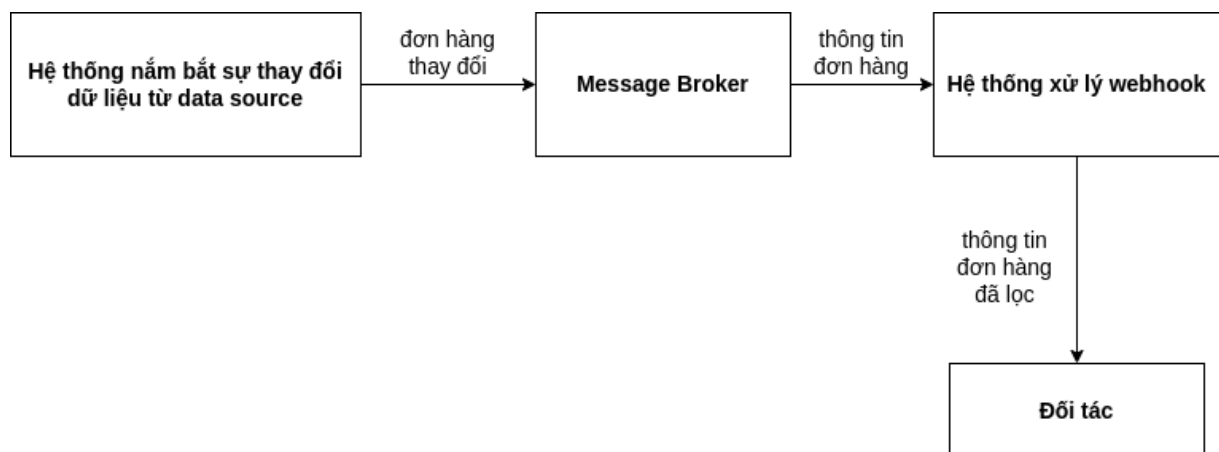
Đối với hệ thống nắm bắt sự thay đổi dữ liệu từ cơ sở dữ liệu, khóa luận cũng đưa ra một giải pháp cho việc theo dõi sự thay đổi dữ liệu ở tầng database đó là sử dụng **Change Data Capture (CDC)**. So với các cách xử lý đã được nêu ở mục 1.1, CDC có ưu điểm hơn là liên tục theo dõi và phát hiện thay đổi bên phía database, từ đó giúp cho hệ thống có thể xử lý và phân tích dữ liệu ngay lập tức. Bằng cách chỉ nắm bắt những thay đổi được thực hiện đối với dữ liệu, thay vì truyền toàn bộ bộ dữ liệu, CDC giảm thiểu lượng dữ liệu cần xử lý, giảm yêu cầu băng thông mạng và cải thiện thời gian xử lý. Tài nguyên dùng để xử lý dữ liệu cũng được giảm bớt mà không cần tạo mới thêm bảng hay sử dụng các câu query làm chậm database.

Đối với hệ thống xử lý webhook, những dữ liệu đã được thay đổi ngay lập tức được đưa đến hệ thống này. Tại đây, dữ liệu sẽ được lọc ra các trường cần thiết để gửi sang cho đối tác. Mỗi đối tác sẽ cần những thông tin khác nhau để sử dụng trong hệ thống của họ. Sau khi lọc ra được dữ liệu cần thiết, nhằm tránh việc trong thời

gian đợi response trả về (thời gian chờ I/O) khiến thời gian sử dụng CPU ít đi hay nói cách khác là CPU rảnh rỗi không làm gì, khóa luận cũng áp dụng xử lý đồng thời (*concurrency*) để giải quyết nhiều tác vụ cùng lúc khi có nhiều request trong thời điểm cao tải. Trong khi đợi response trả về, CPU tiếp tục xử lý những dữ liệu tiếp theo và gửi cho đối tác.

Ngoài ra, hệ thống cần một module trung gian để kết nối hai hệ thống con với nhau nhằm đáp ứng được bài toán chịu tải cao, đảm bảo tính nhất quán dữ liệu, dễ dàng mở rộng khi có thêm nhiều dữ liệu. **Message Broker** là giải pháp được đưa ra để giải quyết vấn đề này. Message Broker sẽ giúp tách rời các service với nhau, dữ liệu sẽ được đưa vào hàng đợi đảm bảo khi một trong các service gặp lỗi vẫn có thể tiếp tục xử lý dữ liệu ngay khi nó hoạt động trở lại. Ngoài ra, việc sử dụng Message Broker có khả năng mở rộng dễ dàng, có thể triển khai được nhiều worker chạy song song với nhau, nhằm giúp tăng số lượng request được xử lý hơn. Lúc này, hệ thống sẽ được đặt trên một nền tảng phân tán dữ liệu giúp xử lý nhanh hơn trong giờ cao điểm. Thông lượng mong muốn là 250 request/giây tương đương với xấp xỉ 3600000 request/ngày. Tức là số lượng thông tin đơn hàng được cập nhật gửi đến cho đối tác phải đạt tối thiểu 250 đơn hàng trên một giây.

Hệ thống hoạt động với kịch bản như sau:



Hình 1.2: Kịch bản mô tả hoạt động khi một đơn hàng thay đổi trạng thái

1.3 Cấu trúc của khóa luận

Khóa luận này được trình bày theo cấu trúc như sau.

- **Chương 1 - Giới thiệu** tập trung giới thiệu tổng quan về tầm quan trọng của việc theo dõi hàng hóa trong lĩnh vực vận chuyển hàng hóa cũng như các vấn đề, bài toán mà các nhà cung cấp dịch vụ vận chuyển đang gặp phải trong việc triển khai hệ thống quản lý, theo dõi đơn hàng.
- **Chương 2 - Cơ sở lý thuyết** sẽ trình bày về cơ sở lý thuyết và công nghệ mà khóa luận sử dụng bao gồm: Webhook, hệ thống trung chuyển message (Message Broker), kỹ thuật theo dõi sự thay đổi phát sinh của dữ liệu (Change Data Capture), xử lý đồng thời (Concurrency). Từ đó nhằm đưa ra các đánh giá, hướng đi của khóa luận trong việc phân tích, đưa ra các giải pháp phù hợp cho hệ thống.
- **Chương 3 - Phương pháp giải quyết vấn đề** của khóa luận đi sâu về các phương pháp được áp dụng để giải quyết vấn đề khả năng mở rộng, chịu tải cao của hệ thống dựa vào các công nghệ, kiến trúc đã được đề cập ở chương 2. Đồng thời cũng đưa ra các giải thuật để tối ưu lượng throughput.
- **Chương 4 - Triển khai, kết quả đạt được và so sánh** sẽ triển khai, cài đặt và các quá trình thực nghiệm. Phần thực nghiệm sẽ đi sâu vào việc áp dụng các công cụ để giám sát, phân tích và quản lý hệ thống.
- **Kết luận** Phần kết luận của khóa luận sẽ tổng kết lại kết quả đã đạt được từ đó đề xuất hướng nghiên cứu tiếp theo nhằm cải thiện hệ thống trong tương lai.

Chương 2: Cơ sở lý thuyết

Trong chương này, khóa luận sẽ trình bày về cơ sở lý thuyết của các công nghệ, thuật toán được sử dụng trong giải pháp nêu ở chương 1 bao gồm Webhook, Message Broker, Change Data Capture và concurrency để có thể thực hiện đồng thời nhiều request hơn trong thời điểm cao tải.

2.1 Webhook

2.1.1 Định nghĩa về webhook

Webhook thường được gọi là *reverse APIs* hay *push APIs*. Mục đích chính của webhook là server sẽ cung cấp dữ liệu cho client theo thời gian thực khi có sự kiện phát sinh bên server. Trong thời đại công nghệ hiện nay, có rất nhiều sự kiện được thực hiện bên phía máy chủ mỗi giây như cập nhật cơ sở dữ liệu, tính toán và trả về kết quả, ...Vậy nên một công cụ giúp đưa các dữ liệu này đến client ngay tại thời điểm sự kiện đó xảy ra là điều hoàn toàn cần thiết. Đối với các API thông thường, client cần phải tạo request API đến server thường xuyên để biết có sự kiện mới hay không thì webhook lại có thể ngay lập tức thông báo cho client biết khi có sự kiện mới diễn ra.

Webhook được triển khai bằng cách client sẽ đưa cho server 1 end-point URL và chỉ định sự kiện mà client cần được biết. Một khi có sự kiện xảy ra tại server, server sẽ tự động gửi dữ liệu bằng một HTTP request (thường là POST) về sự kiện mà client đã đăng ký đến URL đó.

2.1.2 Trường hợp sử dụng webhook

Webhook thường được sử dụng để thông báo về các sự kiện theo thời gian thực một cách tiết kiệm tài nguyên nhất có thể. Cũng chính vì vậy mà webhook được sử dụng trong trường hợp này.

Việc sử dụng webhook có thể dẫn đến hai vấn đề sau:

- Do webhook thực hiện HTTP request để thông báo đến client nên sau khi thông báo xong thì sợi dây liên kết giữa client và server sẽ bị cắt đứt (do HTTP là *stateless*). Nói cách khác, nếu client gặp lỗi thì client sẽ mất dữ liệu đã được trong khoảng thời gian client không hoạt động. Việc áp dụng Message Broker sẽ giúp giải quyết vấn đề này và sẽ gửi lại message đã mất.
- Webhook có thể thực hiện rất nhiều request vì sẽ có rất nhiều sự kiện thay đổi. Khi số lượng lớn request đến đồng thời như vậy mà client không có khả năng chịu tải sẽ rất dễ gây sập hệ thống.

2.2 Message Broker

Message broker là một module trung gian giữa các service và ứng dụng, cho phép các services và ứng dụng có thể giao tiếp với nhau thông qua message, kể cả các service hay ứng dụng này không cùng một ngôn ngữ. Message là một đơn vị dữ liệu trong message broker, nó có thể là request, response, text, binary hoặc JSON. Message broker có thể lưu trữ và điều hướng message đến đến đích phù hợp, message broker cho phép bên gửi gửi message mà không cần biết rõ bên nhận, trạng thái của bên nhận hay số lượng là bao nhiêu. Điều này đảm bảo rằng hai thành phần gửi và nhận được tách rời nhau và không phải chờ đợi nhau, khi đó nếu có nhiều message được gửi đến trong một khoảng thời gian ngắn, hệ thống vẫn có thể xử lý tất cả. Để cung cấp khả năng lưu trữ tin cậy và đảm bảo dữ liệu không bị mất khi truyền tải, message broker cung cấp một message queue chứa các message theo cơ chế vào trước thì ra trước (First In First Out) và các thành phần có thể tương tác với queue này để gửi và nhận message. Để gửi message, một thành phần được gọi là producer tạo ra các message và đẩy vào queue. Message được lưu trữ trong queue cho đến khi một thành phần khác là consumer nhận message và xử lý các tác vụ với message nhận được. Message broker cung cấp 2 mô hình chính để điều hướng message.

2.2.1 Mô hình điều hướng message

Point-to-point messaging

Trong mô hình point-to-point messaging (giao tiếp dữ liệu dạng điểm-điểm), message được gửi từ producer đến một và chỉ một consumer cho dù có nhiều consumer

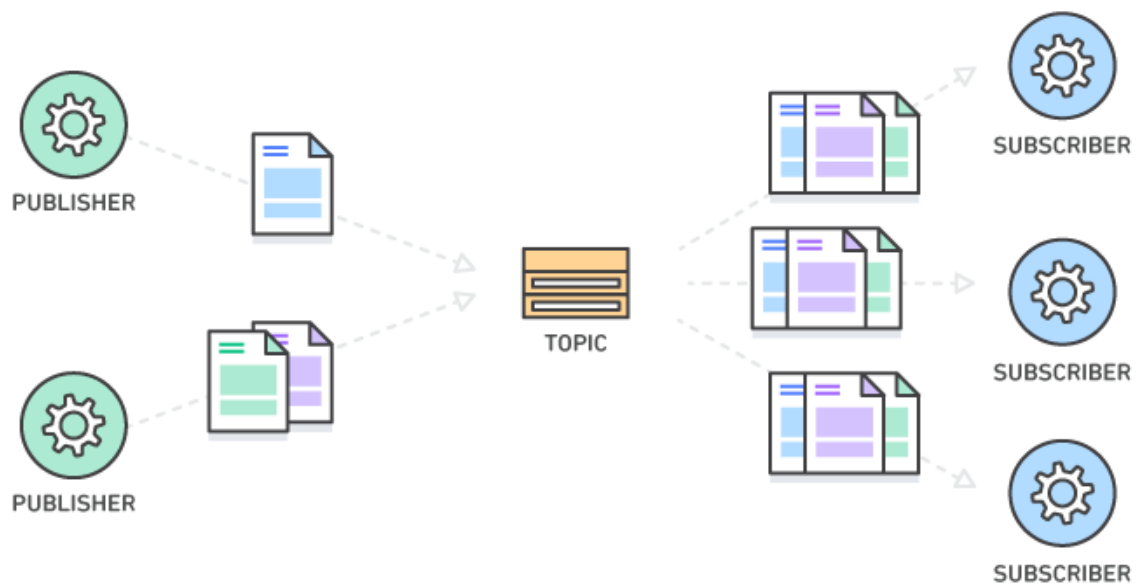
lắng nghe chung trong một queue. Mỗi message đều chỉ được gửi đến một endpoint duy nhất. Mô hình này được sử dụng khi muốn bảo đảm message được nhận bởi duy nhất một consumer và chỉ được xử lý một lần duy nhất bởi consumer. Một ví dụ về trường hợp sử dụng point-to-point messaging là xử lý giao dịch tài chính. Trong hệ thống này, bên gửi tiền và bên nhận tiền cần đảm bảo rằng mỗi giao dịch thanh toán sẽ được gửi một lần và chỉ một lần duy nhất.



Hình 2.1: Mô hình point-to-point messaging [2]

Publish/Subscribe messaging

Mô hình Publish/Subscribe messaging (giao tiếp dữ liệu dạng phát hành/đăng ký) cho phép message được gửi tới nhiều endpoint khác nhau thay vì chỉ duy nhất một như point-to-point. Lúc này message queue được gọi là message topic hay topic. Trong mô hình này, producer được gọi là publisher còn consumer được gọi là subscriber. Các publisher đẩy các message vào trong topic sau đó những subscribers mà subscribe chung vào topic đó sẽ nhận message và xử lý chúng. Điểm khác biệt giữa point-to-point messaging và publish/subscribe messaging là số lượng consumer (subscriber) subscribe vào một queue.



Hình 2.2: Mô hình publish/subscribe messaging [2]

Việc sử dụng message broker cung cấp một cách truyền tải không đồng bộ giữa bên gửi và bên nhận vì các bên gửi không cần phải không cần phải đợi phản hồi của bên nhận, việc này nhằm đảm bảo khi có nhiều message được gửi đến, hệ thống có thể chịu tải cao, hai bên gửi và nhận không cần phải chờ đợi nhau vì bị chặn bởi request trước đó. Ngoài ra, khả năng đảm bảo message không bị mất vì các message được lưu trong topic nhằm tránh mất thông tin đến client trong trường hợp client gặp sự cố cũng là một điểm nổi bật của message broker. Để đáp ứng được khả năng chịu tải cao của hệ thống của khóa luận, mô hình publish/subscribe được chọn để triển khai để có thể giúp phân phối message đến nhiều bên nhận cùng lúc và có thể nâng cấp dễ dàng khi có thêm nhiều bên nhận.

Note: Để đơn giản và đỡ gây nhầm lẫn, các thuật ngữ **publisher/subscriber/message topic** đã được đề cập trong mô hình publish/subscribe đề cập ở trên sẽ lần lượt chuyển thành **producer/consumer/topic**.

Hiện nay, có nhiều Message Broker hoạt động trên nền tảng và cách thức khác nhau, nhưng mục đích chính của Message Broker vẫn là điều hướng, trung chuyển message từ bên gửi đến bên nhận. Có thể kể đến một vài Message Broker được sử dụng phổ biến hiện nay như: Apache Kafka, RabbitMQ, Apache ActiveMQ,...

mỗi Message Broker lại có một kiến trúc và use case khác nhau và được phân chia thành hai loại chính là Message base gồm RabbitMQ, ActiveMQ,... và Data pipeline như Kafka. Bảng 2.1 sẽ nêu rõ sự khác biệt giữa hai loại Message Broker này từ đó nhằm có thể đưa ra lựa chọn phù hợp cho khóa luận:

Message Base	Data Pipeline
Sau khi consumer nhận được message thì message này sẽ bị xóa khỏi queue mà consumer subscribe	Sau khi consumer nhận được message thì message này vẫn được lưu lại trong vài ngày kể từ khi topic nhận được message
Lưu trạng thái của consumer để đảm bảo tất cả các consumer subscribe vào topic đều nhận được message	Không lưu trạng thái của consumer
Khi một message mới được đẩy vào topic, consumer đang subscribe vào topic đó chỉ lấy được duy nhất một message mới đó	Khi có message mới, consumer có thể tùy ý lấy message theo dạng batch, có thể chỉ lấy message mới hoặc cả message cũ

Bảng 2.1: So sánh sự khác nhau giữa Message base và Data pipeline

Dựa vào bảng trên, có thể thấy rõ sự khác nhau cơ bản giữa 2 loại và lựa chọn phù hợp với từng bài toán:

- Đối với các bài toán yêu cầu đảm bảo mỗi consumer đều nhận được message và diu nhất một lần nhận thì Message base sẽ thích hợp làm hệ thống truyền tải message giữa các service và ứng dụng với nhau.
- Còn đối với các bài toán yêu cầu có khả năng lưu trữ message, tốc độ truyền tải message cao và khả năng phân tán mạnh. Khi có message mới thì consumer có thể lựa chọn message muốn lấy thay vì chỉ được lấy message mới nhất và cùng một message consumer có thể lấy lại nhiều lần khi bị mất message thì Data pipeline là sự lựa chọn dành cho bài toán này.

Để phù hợp với nhu cầu của khóa luận, bài toán đặt ra là yêu cầu sự chính xác cao, hệ thống có thể xử lý lượng message lớn với tốc độ cao, mở rộng dễ dàng, đảm bảo không bị mất message bởi vì đối với các thông tin đơn hàng khi được gửi sang

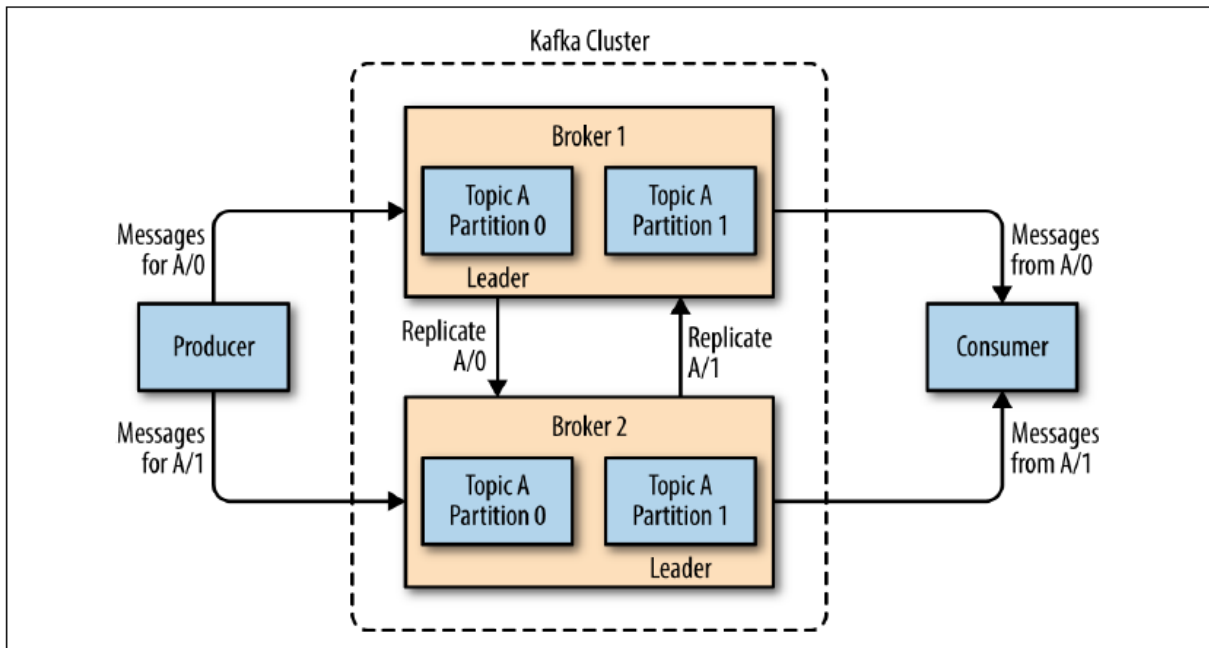
cho đối tác phải phải đảm bảo nhanh chóng, yêu cầu sự chính xác và hệ thống phải có khả năng mở rộng dễ dàng khi có thêm nhiều đối tác nên Data pipeline như **Apache Kafka** là sự lựa chọn phù hợp để giải quyết bài toán này.

2.2.2 Giới thiệu về Kafka

Apache Kafka hay Kafka là một nền tảng publish/subscribe message phân tán dùng để xử lý streaming, xây dựng data pipeline theo thời gian thực hoặc được sử dụng như là một Message Broker. Kafka ban đầu được phát triển bởi LinkedIn vào năm 2011, được viết bằng ngôn ngữ Java và Scala. Sau đó trở thành một project open-source với Apache license nên được gọi là Apache Kafka. Kafka hiện nay có thể xử lý từ 1 triệu message mỗi giây hay hàng nghìn tỷ message một ngày và là nền tảng xử lý dữ liệu streaming phổ biến nhất thế giới được các tổ chức hàng đầu áp dụng trong cơ sở hạ tầng dữ liệu của mình phục vụ cho các hệ thống quan trọng. Dưới đây là các lý do chính giải thích tại sao Kafka được sử dụng rộng rãi:

- **Đảm bảo thông lượng cao:** Có khả năng xử lý dữ liệu tốc độ cao và khối lượng lớn mà không gặp vấn đề về performance, Kafka có thể xử lý một triệu message trong một giây.
- **Khả năng mở rộng cao:** Vì Kafka là hệ thống phân tán, nên có khả năng mở rộng rất nhanh và dễ dàng.
- **Khả năng sẵn sàng cao:** Vì các message được lưu trữ trong topic và các topic này được lưu trữ trên server (broker) nên các message không thể bị mất khi hệ thống gặp sự cố. Ngoài ra Kafka còn có cơ chế tự cân bằng các message cho các broker khi các broker khác gặp sự cố.
- **Khả năng lưu trữ lâu bền:** Lưu trữ an toàn, bảo mật các luồng dữ liệu trong một cụm phân tán, đáng tin cậy và có khả năng chịu lỗi cao.

2.2.3 Kiến trúc của Kafka



Hình 2.3: Kiến trúc cơ bản của Kafka [3]

Topic

Message trong Kafka được lưu trữ vào trong các topic. Topic có thể được coi như là một table trong cơ sở dữ liệu hoặc tập tin (folder) trong hệ thống file (filesystem), trong khi các message được coi là các file trong folder đó. Một topic có các đặc điểm sau:

- *Topic là dạng queue*: Mỗi khi một message mới được đẩy vào topic, message sẽ được đẩy vào cuối topic và được đọc từ đầu topic.
- Message trong Topic là bất biến, nghĩa là message không thể được sửa đổi khi đã được ghi vào Topic.
- *Topic trong Kafka có thể có nhiều producer và consumer*: Một topic có thể không có, có một hoặc nhiều producer ghi message vào hoặc có nhiều consumer đọc message từ topic đó.

Không giống như các message broker khác như RabbitMQ, message được lưu ở topic trong Kafka không thể bị xóa khi chúng đã được consumed. Mặc định message

trong topic được giữ lại 7 ngày kể từ khi được đẩy vào bởi producer, tuy nhiên có thể config thời gian này tùy ý.

Broker

Một broker là một server trong Kafka. Topic được lưu trữ trên disk và disk được lưu trữ trên server. Broker nhận message từ producer, gán offset cho các message này và lưu trữ message vào disk. Broker cũng nhận các request từ consumer để lấy message trong topic. Phụ thuộc vào phần cứng của mỗi máy tính, một broker có thể xử lý hàng nghìn partition và hàng triệu message mỗi giây. Mỗi broker được định danh bằng ID duy nhất, mỗi broker lưu trữ một vài partition trong topic.

Cluster

Kafka broker là một server hoặc một node trong Kafka cluster, có thể hiểu cluster là tập hợp các broker tạo thành một cụm làm việc với nhau (multi-brokers). Mỗi broker trong một cluster cũng có thể là một bootstrap server¹, nghĩa là kết nối đến một broker trong một cluster, ta có thể kết nối đến bất kì broker nào trong cụm đó, điều đó giúp consumer có thể đọc được message của partition nằm trên bất kì các broker nào. Mỗi broker có thông tin về các broker, topic và partition của Kafka cluster, thông tin này được gọi là *metadata*. Việc triển khai nhiều broker theo một cụm như vậy nhằm đảm bảo 4 tính chất đã được nêu ở phần 2.2.2, giải thích cho lí do này như sau: nếu chỉ có một server (broker) được triển khai, nếu không may broker đó gặp sự cố thì producer và consumer sẽ không thể gửi và nhận message, nhưng nếu triển khai nhiều server (multi-brokers) thì khi một broker gặp sự cố vẫn sẽ còn các broker khác nhau hoạt động như là các **replica**². Điều này giúp đảm bảo tính sẵn sàng cao và có thể mở rộng dễ dàng.

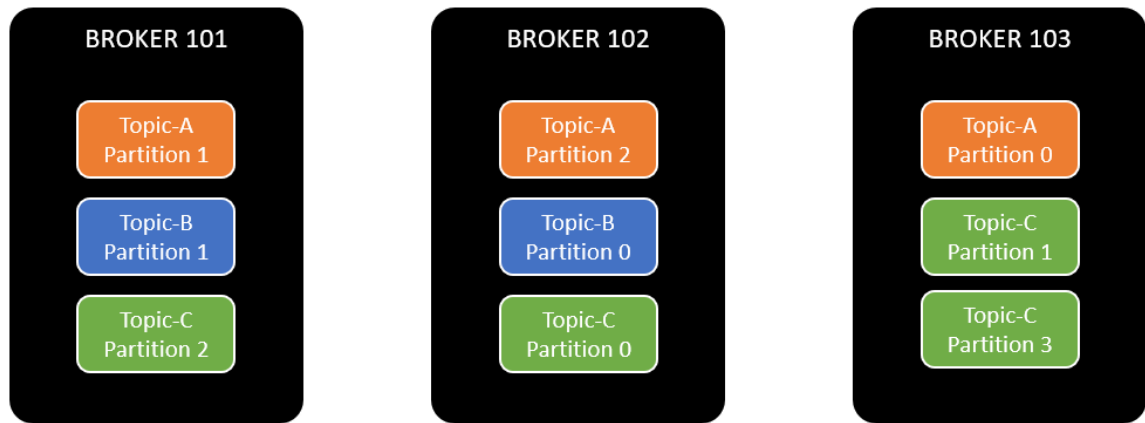
Partition và Offset

Topic được chia nhỏ thành các partition, nơi mà message thật sự được lưu trữ. Một partition được hiểu là một log nằm trong một Kafka broker riêng. Bằng cách này, việc lưu trữ, ghi và xử lý message có thể được chia trong nhiều nodes trong một cụm broker. Các message khi được append vào trong partition bắt đầu từ giá trị 0 và cứ thế tăng dần, giá trị này được gọi là *offset*. Offset chỉ có thể tăng dần cho dù message này bị xóa sau khi hết hạn thì offset cũng không quay lại. Offset

¹Một danh sách các endpoint của các broker trong cluster

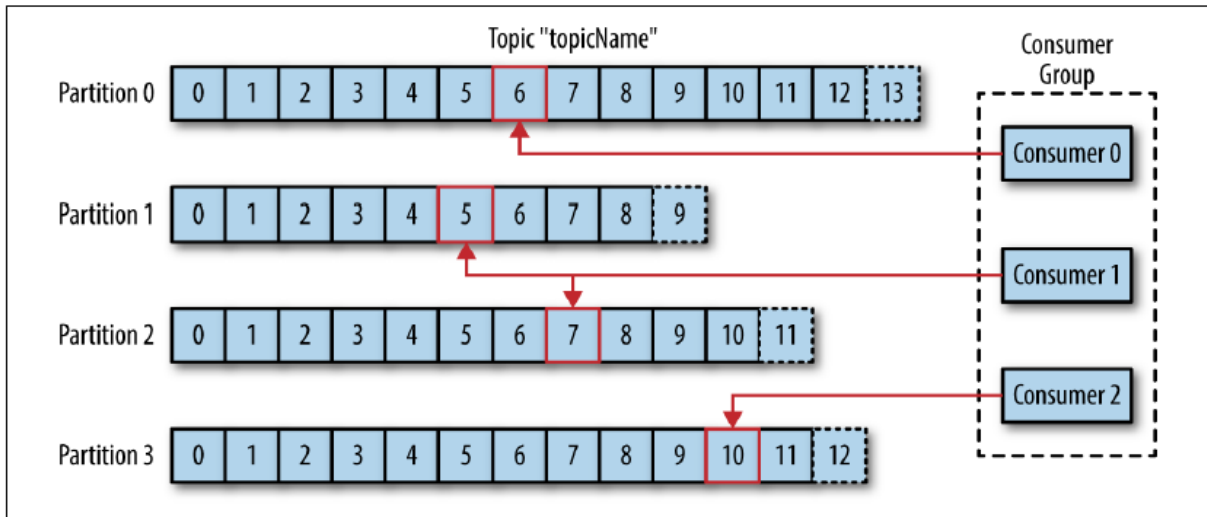
²Là các bản sao của broker

giúp đảm bảo thứ tự của các message trong cùng partition, ví dụ offset = 1 thì sẽ không thể đọc trước offset = 0. Các partition được phân tán trong các broker, một hoặc nhiều partition của một topic sẽ nằm trên một broker (như hình 2.4) để đảm bảo tính tin cậy của Kafka



Hình 2.4: Các partition được chia đều trong cụm broker

Các message có cùng **message key** ví dụ như cùng *mã đơn hàng*, *căn cước công dân* sẽ được đưa vào chung một partition. Nếu key này được định nghĩa thì Kafka sẽ băm (hash) key này sử dụng các thuật toán băm (hash algorithm) sau đó tất cả các message có cùng kết quả sẽ đi vào chung một partition nằm trong một broker. Còn nếu key này được để là null, message khi này sẽ được gửi theo thuật toán round-robin - thuật toán theo nguyên tắc vòng tròn giúp chia đều dữ liệu thành các phần bằng nhau - nhằm giúp cân bằng các message giữa các partition.



Hình 2.5: Các offset trong partition của một topic [3]

Producer

Producer là một thành phần giúp ghi message vào partition của topic. Mặc định, producer sẽ gửi message đến tất cả các partition trong một topic một cách cân bằng theo cơ chế round-robin. Tuy nhiên trong một vài trường hợp, producer cũng có thể gửi đến một partition cụ thể được chỉ định. Ngoài ra, để producer biết message đã được gửi thành công đến partition, Kafka sử dụng một cơ chế đó là ack (acknowledgment), ack sẽ có các giá trị config như sau [3]:

- **acks=0**: sẽ không sử dụng ack, gửi message nhưng không cần phản hồi là partition đã nhận được message hay chưa, cách này có thể giúp tăng throughput nhưng cũng gây ra việc mất message khi được gửi lỗi trong lần đầu.
- **acks=1**: đây là config mặc định của Kafka. Producer sẽ chờ cho tới khi nhận phản hồi từ broker khi *leader replica* của partition đó nhận được message. Nếu message không thể được ghi vào partition, producer sẽ nhận được một error từ broker và producer sẽ tiến hành gửi lại message tránh tình trạng mất mát dữ liệu. Tuy nhiên, message vẫn có thể bị mất nếu leader replica bị lỗi và replica thay thế của leader đó chưa nhận được message.
- **acks=all**: khi acks=all, producer sẽ nhận được tất cả các response thành công từ các leader replica, điều này sẽ đảm bảo message chắc chắn không bị mất cho dù broker của partition đó gặp lỗi. Tuy nhiên, đánh đổi lại cho việc đó là

độ trễ sẽ cao hơn $acks=0$ và $acks=1$, bởi vì producer sẽ phải đợi lâu hơn để các broker nhận message.

Consumer

Consumer là thành phần đọc message từ topic. Consumer sẽ subscribe vào một hoặc nhiều topic và đọc message theo thứ tự offset diễn ra theo tuần tự để đảm bảo thứ tự message. Một consumer cũng có thể đọc message từ một hoặc nhiều partition trong một topic. Đồng thời, consumer có cơ chế đọc message từ broker khác trong trường hợp chưa đọc message xong mà broker gặp sự cố.

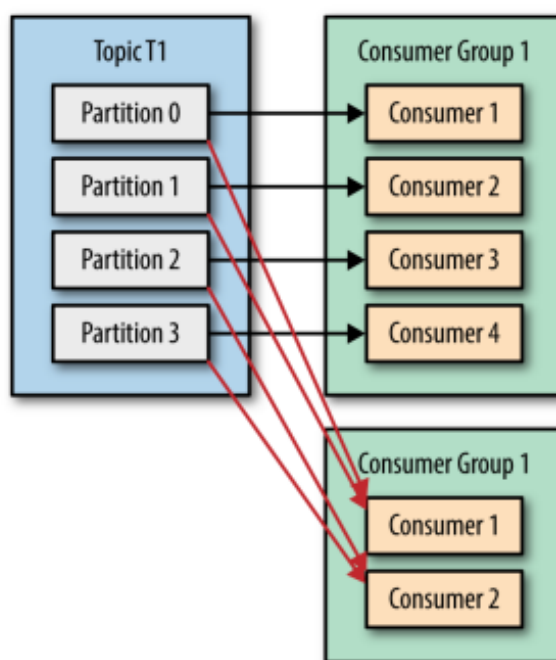
Consumer Group

Khi số lượng producer tăng lên và các producer này đồng thời gửi message đến cùng một topic, topic này chỉ có 1 partition và chỉ có một consumer thì khả năng xử lý sẽ rất chậm thì có quá nhiều message lag, dẫn đến **bottleneck**. Giải pháp được đưa ra là tăng số lượng consumer lên nhằm chia đều số lượng message giữa các consumer, lúc này số lượng message sẽ được xử lý nhiều hơn vì có nhiều worker xử lý hơn. Ví dụ 1 topic có 1 partition và 1 consumer đọc message từ partition đó có throughput là 100 message/s nhưng với 1 topic chia thành 10 partition và có 10 consumer đọc topic đó thì có thể có lượng throughput khoảng 1000 message/s. Khi đó tất cả các consumer cùng thuộc một nhóm subscribe vào topic này sẽ là **consumer group**. Mỗi consumer trong consumer group sẽ đọc message từ một hay nhiều partition để đảm bảo thứ tự message, không tồn tại trường hợp nhiều consumer trong consumer group cùng đọc một partition. Như vậy sẽ có các trường hợp sau xảy ra trong cùng một consumer group:

- *Số lượng partition > số lượng consumer*: Lúc này sẽ xảy ra trường hợp một consumer đọc vào nhiều partition bởi vì số lượng consumer ít hơn partition.
- *Số lượng partition = số lượng consumer*: Đây là trường hợp lý tưởng, mỗi một consumer xử lý message từ một partition, trường hợp này sẽ tối ưu lượng throughput xử lý được.
- *Số lượng partition < số lượng consumer*: Trong trường hợp này, khi số lượng consumer dư thừa vì các consumer không thể cùng lấy chung message từ một partition, tức là consumer dư thừa (inactive consumer) đó không nhận message nào. Tuy nhiên, các inactive consumer đó không hề vô nghĩa vì nếu trong

trường hợp một active consumer nào đó gặp lỗi và không thể hoạt động được tiếp, một inactive consumer sẽ được thay thế vị trí đó và tiếp tục lấy message từ partition, quá trình re-assign consumer này được gọi là **rebalance**¹.

Trên thực tế, consumer group được dùng để ứng dụng trong việc có nhiều ứng dụng hoặc hệ thống khác nhau cùng đọc message chung từ một topic. Để đảm bảo rằng tất cả message được đọc bởi tất cả ứng dụng khác nhau, mỗi ứng dụng đều phải có một consumer group riêng. Không giống như các message broker khác, khi tăng số lượng consumer và consumer group sẽ không làm giảm hiệu suất của hệ thống.



Hình 2.6: Nhiều consumer group cùng sử dụng một topic [3]

Tuy nhiên không phải cứ chia càng nhiều partition thì càng tốt vì việc tăng số lượng partition cũng kéo theo các hệ lụy như tăng độ trễ, lượng chênh lệch throughput giữa các partition khi có message key, số lượng files xử lý I/O nhiều hơn,...

Consumer offset

Như đã đề cập, offset là vị trí của mỗi message được lưu trữ trong partition. Khi chúng ta cập nhật vị trí này, hành động đó gọi là *commit*. Khi consumer commit

¹Quá trình re-assign các consumer trong consumer group khi có sự thay đổi số lượng consumer

giá trị offset, Kafka sử dụng một topic đặc biệt là `__consumer_offsets`, topic này được dùng để commit các giá trị offset trong các partition. Khi consumer đang chạy bình thường, sẽ không có bất kì ảnh hưởng gì. Tuy nhiên, khi consumer gặp lỗi hoặc buộc phải dừng hay một consumer mới được đưa vào consumer group thì lúc này sẽ xảy ra rebalance. Sau khi rebalance, mỗi consumer có thể được xử lý một partition khác với partition trước đó. Để biết được vị trí chính xác tiếp tục lấy message, consumer sẽ đọc giá trị offset được commit gần nhất của mỗi partition và tiếp tục từ đó. Vậy có thể hiểu consumer offset như là một bookmark hay là một checkpoint cho consumer group.

Nếu offset được commit nhỏ hơn giá trị offset cuối cùng mà consumer đã xử lý thì khi đó các message nằm giữa giá trị offset được commit và offset cuối cùng sẽ tiếp tục được xử lý lại, điều này gây ra duplicate.

Nếu giá trị offset được commit lớn hơn giá trị offset cuối cùng, tất cả các message kể từ offset cuối trở đi đến offset được commit sẽ bị bỏ qua trong consumer group.

Để tránh điều này xảy ra, việc lưu trữ offset trong topic `__consumer_offsets` là cực kì quan trọng để tránh đọc lại hoặc bị mất message. Kafka cung cấp các cách để commit offset như sau:

- **At most once:** Cách đơn giản nhất để commit là để consumer tự động làm việc này. Mặc định, sau mỗi 5s commit sẽ tự động thực hiện ngay sau khi nhận được message. Tuy nhiên, cách này có một nhược điểm như sau. Trong trường hợp consumer đang xử lý message mà consumer gặp sự cố và message đó chưa được commit thì coi như message đó đã mất.
- **At least once:** Cách này sẽ commit ngay sau khi message đã được xử lý thành công. Như vậy khi consumer gặp sự cố trong lúc xử lý message đó thì cũng không xảy ra vấn đề gì. Vì khi consumer khác xử lý sẽ tiếp tục xử lý lại message đó. Nhưng nó sẽ gây ra việc xử lý message hai lần. Nếu trong kịch bản sử dụng là giao dịch tài chính, consumer sẽ xử lý trừ tiền một lần rồi nhưng consumer này gặp sự cố và consumer khác xử lý sẽ lại tiếp tục trừ tiền một lần nữa, điều này sẽ gây ảnh hưởng nghiêm trọng. Vậy nên khi sử dụng cách commit này phải đảm bảo quá trình xử lý là **idempotent**¹ hoặc đưa ra các biện pháp kiểm tra phù hợp.

¹Những xử lý có thể lặp lại nhiều lần mà không ảnh hưởng đến hệ thống

- **Exactly once:** đây là trường hợp đặc biệt chỉ dùng riêng cho việc message chuyển từ topic này sang topic khác sử dụng transaction API.

2.2.4 Mô hình replication trong Kafka

Zookeeper

Zookeeper là một service điều phối phân tán và có chức năng hỗ trợ quản lý các servers. Bởi vì khi làm việc trong hệ thống phân tán, việc điều phối và quản lý các thành phần trong hệ thống là một quá trình phức tạp vì tính bất đồng bộ của các thành phần này. Zookeeper được sinh ra để giải quyết vấn đề này. Cụ thể trong Kafka, Zookeeper có các chức năng chính sau:

- Zookeeper có nhiệm vụ quản lý các broker trong cluster.
- Zookeeper nắm giữ các thông tin của topic, bao gồm một danh sách các topic tồn tại, số lượng partition trong topic.
- Gửi thông tin đến Kafka về các event phát sinh ví dụ như có topic mới được tạo, broker nào đó gặp lỗi,...
- Sử dụng thuật toán leader election để bầu ra partition leader mới khi một broker gặp sự cố.

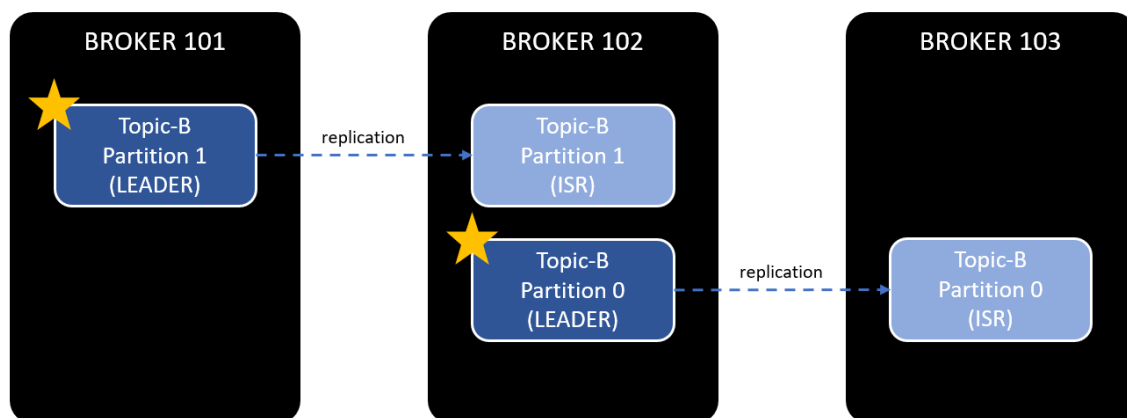
Replication

Replication là trái tim của kiến trúc Kafka. Bởi vì Kafka được biết đến là một nền tảng xử lý message phân tán, nên replication là yếu tố để đảm bảo các tính sẵn sàng, tính phân tán mạnh của Kafka.

Dữ liệu trong Kafka nằm trong partition và mỗi partition có các replica là các bản sao của partition đó nằm trên các broker khác nhau. Mỗi broker có thể lưu trữ hàng trăm đến hàng nghìn replica của các topic và partition khác nhau. Một khi một broker không may gặp sự cố, producer và consumer vẫn có thể gửi và nhận message thông qua các broker khác bởi vì các partition được sao chép thành các bản sao khác nhau. Replica được chia làm hai thành phần chính:

- **Leader replica:** Mỗi partition có một replica được chọn làm leader. Tất cả producer và consumer thực hiện quá trình đọc/ghi từ partition leader này để đảm bảo tính nhất quán và tối ưu hiệu suất.

- **Follower replica:** Tất cả replica của partition không phải leader được gọi là follower. Follower có nhiệm vụ đồng bộ message từ leader replica. Để có thể đồng bộ, các follower gửi một Fetch request đến cho leader, những Fetch request này chứa các offset của message mà follower yêu cầu, các leader sau đó sẽ gửi lại một response có chứa message cho follower.



Hình 2.7: Leader và follower replica trong cụm broker

Trong trường hợp broker gặp sự cố, leader replica của partition đó cũng bị ảnh hưởng, một trong các follower replica được bầu để trở thành leader replica mới. Quá trình quyết định này được Controller thực hiện bằng thuật toán **leader election**¹. Để có thể dễ dàng quyết định xem replica nào được bầu thì số các node trong cluster thường được triển khai là các số lẻ 3, 5, 7,... Nếu một replica mất kết nối với Zookeeper, dừng fetch các message mới hoặc không thể đồng bộ message trong một khoảng thời gian ngắn, replica đó đang trong trạng thái *out-of-sync* [2]. Một out-of-sync replica sẽ nhanh chóng đồng bộ lại với leader khi replica đó kết nối với Zookeeper lại khi sự cố là đường truyền network không ổn định nhưng có thể sẽ lâu hơn nếu broker của replica đó gặp sự cố trong một khoảng thời gian dài.

Số lượng replica của partition được cấu hình bằng **replication.factor**, ví dụ **replication.factor = N** thì partition đó sẽ được nhân bản thành N lần và được chia đều vào các broker. Số lượng replication factor càng nhiều thì sẽ giúp tăng tính sẵn sàng cao, tăng độ tin cậy và tránh mất dữ liệu hơn. Tuy nhiên, khi

¹Thuật toán xác định leader giữa một cụm node chưa có leader

`replication.factor = N` thì sẽ cần ít nhất N broker và số lượng dữ liệu được nhân bản sẽ là N điều đó có nghĩa sẽ tốn bộ nhớ hơn N lần. Trong khóa luận, số lượng replication factor được để là 3 để có thể tránh việc mất dữ liệu trong trường hợp một broker gặp sự cố mà cũng không tốn quá nhiều bộ nhớ để lưu trữ khi được nhân bản.

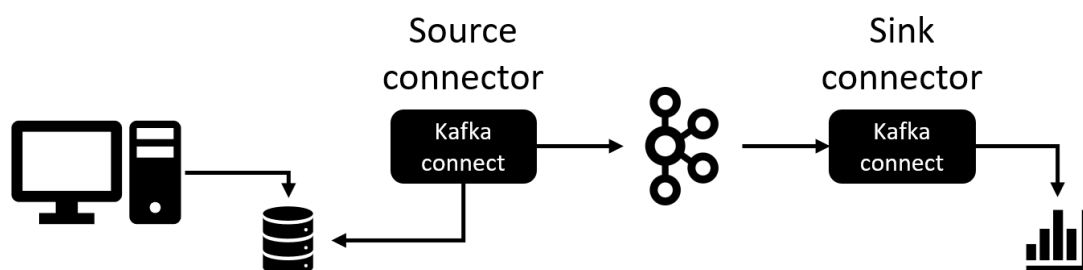
2.2.5 Kafka Connect

Kafka Connect là một thành phần của Kafka với nhiệm vụ kết nối Kafka với các hệ thống dữ liệu khác như database, file system (hệ thống file). Kafka Connect sẽ thu thập dữ liệu từ database hoặc các nguồn khác vào Kafka topic, từ đó tạo thành một luồng dữ liệu với độ trễ thấp, đáng tin cậy. Về bản chất Kafka Connect giống như một interface, tạo ra API để bên thứ ba sử dụng có thể implement theo từng cách thức xử lý của từng service. Kiến trúc của Kafka Connect bao gồm các thành phần sau:

Connector

Connector trong Kafka Connect là các thành phần dùng để truyền tải dữ liệu giữa Kafka và các hệ thống khác. Kafka Connect bao gồm hai loại connector:

- **Source connector:** là các connector có nhiệm vụ kéo dữ liệu từ hệ thống nguồn implement Kafka Connect và gửi dữ liệu đó đến Kafka topic.
- **Sink connector:** connector sử dụng các dữ liệu từ topic và đẩy đến cho hệ thống đích.



Hình 2.8: Mô hình sử dụng Kafka Connect

Worker

Connector và task là các tác vụ được thực thi như một process. Kafka Connect gọi các process này là worker, worker có hai loại: standalone (độc lập) và distributed (phân tán).

- **Standalone worker:** Đây là loại worker đơn giản, chỉ có một process thực thi tất cả các connector và task. Ưu điểm của loại worker này là cấu hình đơn giản, tiện lợi, sử dụng trong tình huống chỉ cần một process để xử lý. Tuy nhiên, bởi vì chỉ có một process, nó sẽ có hạn chế nhất định là không có khả năng mở rộng cũng không có khả năng chịu lỗi khi process duy nhất này gặp sự cố.
- **Distributed worker:** Vì có tính phân tán, worker này có khả năng mở rộng dễ dàng. Nếu một worker gặp sự cố, task và connector của worker đó sẽ được chuyển cho các worker khác để tiếp tục thực thi.

Task

Các task là tác vụ mà worker phải thực thi. Bằng cách cho phép connector chia một công việc thành nhiều task là cách để tăng performance cho Kafka connect. Ví dụ khi có một công việc là đọc dữ liệu từ nhiều bảng khác nhau trong một cơ sở dữ liệu, việc chia thành nhiều task, mỗi task đọc một table để tối ưu hóa các luồng khi chạy song song.

2.3 Change Data Capture

Để trích xuất dữ liệu, sau đó chuyển đổi dữ liệu dạng thô từ một nguồn dữ liệu (cơ sở dữ liệu nguồn) trong hệ thống đến một hệ thống dữ liệu khác để xử lý dữ liệu hoặc sang data warehouse (kho dữ liệu) cho mục đích phân tích. Theo cách truyền thống các hệ thống thường xử lý dữ liệu từng batch (đã được đề cập ở 1.2) trong một vài lần trong ngày. Tuy nhiên, cách này sẽ có độ trễ và giảm giá trị của dữ liệu vì dữ liệu có thể không kịp cập nhật. Các business hiện nay thường yêu cầu dữ liệu phải cập nhật liên tục như dữ liệu tài chính, mạng xã hội hay trong việc theo dõi hàng hóa là các trạng thái của đơn hàng.

Từ đây, nhu cầu về một phương pháp để nắm bắt sự thay đổi dữ liệu và đồng bộ dữ liệu liên tục giữa cơ sở dữ liệu nguồn cho các hệ thống khác mà không chiếm

tài nguyên của cơ sở dữ liệu, là rất cần thiết. Change Data Capture đã xuất hiện như một giải pháp để có thể liên tục phát hiện thay đổi ở cơ sở dữ liệu nguồn, từ đó giúp cho hệ thống có thể xử lý và phân tích dữ liệu ngay lập tức.

2.3.1 Khái niệm về CDC

Change Data Capture (CDC) là một kỹ thuật theo dõi và nắm bắt những phát sinh thay đổi ở phía cơ sở dữ liệu nguồn từ đó giúp đồng bộ dữ liệu theo thời gian thực (real-time) từ cơ sở dữ liệu nguồn sang các nguồn dữ liệu khác một cách nhanh chóng và dễ dàng. Nhờ có CDC, hệ thống sẽ không cần phải load một lượng lớn dữ liệu như batch processing thay vào đó có thể load dần dần nhưng vẫn đảm bảo real-time. [4]

2.3.2 ETL và CDC

ETL là viết tắt của Extract, Transform, Load, là quá trình trích xuất dữ liệu nguồn, chuyển đổi nó sang định dạng thích hợp và sau đó tải nó vào một hệ thống khác để xử lý hoặc phân tích. Quá trình ETL là một trong những công việc quan trọng nhất trong việc tích hợp dữ liệu giữa các hệ thống khác nhau. Phương pháp ETL sẽ trích xuất dữ liệu từ dữ liệu nguồn khác nhau sau đó biến đổi dữ liệu (tính toán, xử lý) và cuối cùng là tải dữ liệu vào data warehouse (kho dữ liệu) hoặc Kafka Connect của Kafka. Các giai đoạn của quá trình ETL được diễn ra như sau:

- **Extract:** Trong quá trình trích xuất, dữ liệu sẽ được trích xuất từ cơ sở dữ liệu nguồn, dữ liệu này có thể có cấu trúc (structured) hoặc không có cấu trúc (unstructured) và cung cấp một luồng dữ liệu đến các hệ thống khác.
- **Transform:** Trong quá trình này, dữ liệu thô sẽ được xử lý qua các giai đoạn lọc dữ liệu, định dạng, tính toán nhưng vẫn đảm bảo tính nhất quán của dữ liệu. Sau đó dữ liệu này sẽ được chuẩn bị để đưa vào quá trình tải.
- **Load:** Bước cuối cùng trong quy trình ETL là tải dữ liệu đã được chuyển đổi vào hệ thống đích. Dữ liệu có thể được tải cùng một lúc hoặc theo các khoảng thời gian đã lên lịch.

Change Data Capture là một cơ chế giúp cho quá trình ETL được thực hiện nhanh chóng, tiện lợi hơn.

2.3.3 Các phương pháp CDC

Có nhiều phương pháp CDC có thể thực hiện phục thuộc vào yêu cầu của hệ thống. Dưới đây là các phương pháp phổ biến để sử dụng CDC:

- **Log-based:** Khi một transaction mới được lưu vào database (cơ sở dữ liệu), nó sẽ được ghi vào một file log hay transaction log. Database có thể sử dụng transaction log để backup lại dữ liệu. đối với phương pháp CDC, dữ liệu transaction log được sử dụng để trích xuất dữ liệu sau đó truyền tải đến hệ thống đích. Ví dụ như hệ quản trị cơ sở dữ liệu MySQL sử dụng file binlog để ghi lại các transaction log.
- **Trigger-based:** Phương pháp này được triển khai bằng cách sử dụng Trigger và một bảng lưu lại lịch sử của những thay đổi trong bảng được tracking, các dữ liệu có thể được ghi lại như câu lệnh DML hoặc giá trị cũ và mới trước và sau khi thay đổi. Về bản chất, trigger là một dạng thủ tục trong SQL được thực hiện khi có một sự kiện: INSERT, UPDATE, DELETE xảy ra. Ví dụ, mỗi khi một dữ liệu trong bảng packages được thay đổi, trigger UPDATE của bảng này sẽ INSERT một bản ghi mới vào trong bảng packages_logs (là bảng lưu lại lịch sử của những thay đổi của bảng packages). Tuy nhiên, phương pháp này sẽ gây giảm hiệu suất cho cơ sở dữ liệu vì trigger thực hiện các câu lệnh DML lên bảng vì thế nó làm gia tăng lượng công việc lên cơ sở dữ liệu và làm cho hệ thống chạy chậm lại. Hơn nữa, đối với mỗi bảng lại cần thêm một bảng logs để lưu lại lịch sử thì việc này cũng làm cho cơ sở dữ liệu trở nên rất lớn.
- **Timestamps-based:** Một số bảng trong cơ sở dữ liệu có trường MODIFIED để lưu trữ thời điểm mà bản ghi đó được cập nhật nên có thể nắm bắt sự thay đổi bằng cách lọc ra những bản ghi được thay đổi gần đây nhất. Dễ dàng nhận thấy nhược điểm chính của phương pháp này là thiếu độ chính xác và không có khả năng nhận biết được những bản ghi đã bị xóa.

2.4 Concurrency

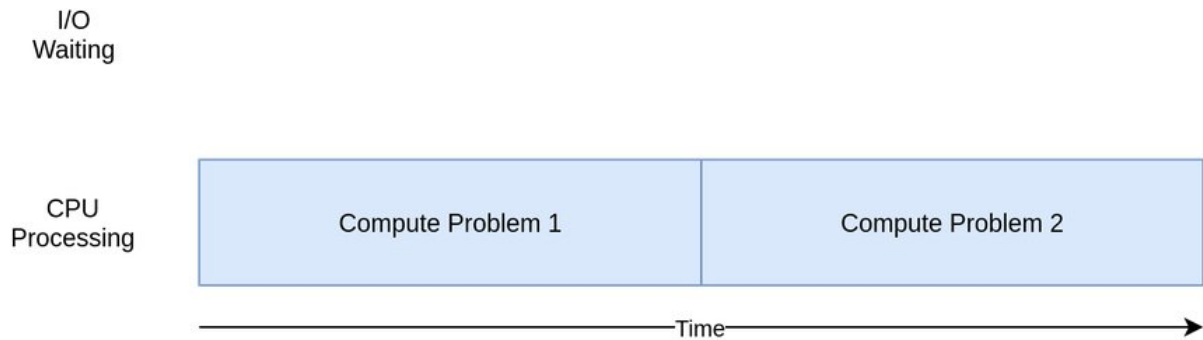
Trước khi đưa ra các cơ sở lý thuyết về Concurrency, các khái niệm về I/O Bound, CPU Bound được đề cập trước phục vụ cho việc sử dụng Concurrency để tối ưu

một chương trình I/O Bound.

2.4.1 CPU bound và I/O bound

CPU bound

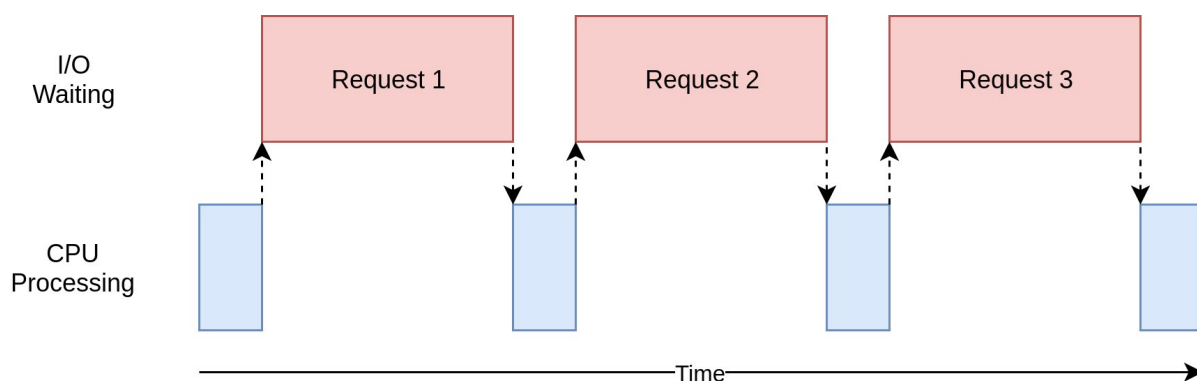
Một chương trình được gọi là CPU bound khi phần lớn thời gian xử lý một tác vụ sử dụng CPU để tính toán. Ví dụ như tác vụ liên quan đến chỉnh sửa ảnh, video là tác vụ CPU bound.



Hình 2.9: Mô tả về một chương trình CPU Bound

I/O bound

Một chương trình được gọi là I/O bound nếu thời gian xử lý phụ thuộc chủ yếu vào khoảng thời gian dành ra chờ đợi các hoạt động input/output hoàn thành. I/O bound có thể khiến chương trình chậm đi bởi vì nó dành phần lớn thời gian chờ đợi input/output (I/O) từ các luồng bên ngoài, trong thời gian đợi đó CPU sẽ rảnh rỗi nên sẽ gây lãng phí CPU. Ví dụ về I/O bound có thể kể đến như đọc/ghi file, tác vụ liên quan đến kết nối mạng,...



Hình 2.10: Mô tả về một chương trình I/O Bound

2.4.2 Concurrency

Concurrency nói về một chương trình xử lý nhiều tác vụ trên 1 bộ xử lý CPU cùng lúc trong một khoảng thời gian. Concurrency là một cách tiếp cận để giảm thời gian phản hồi của hệ thống chỉ bằng 1 core CPU. Nói cách khác, concurrency cho phép chồng chéo nhiều tác vụ để cải thiện hiệu quả của một hệ thống. Concurrency là điều cần thiết trong thời đại công nghệ hiện nay, đặc biệt là trong các hệ thống đòi hỏi hiệu suất và hiệu quả cao.

Ví dụ về concurrency như sau: Trình duyệt Google Chrome có nhiều tác vụ khác nhau ví dụ như render nội dung website, download file từ Internet. Nếu hai tác vụ này chạy đồng thời trên 1 core CPU thì CPU sẽ chuyển đổi xen kẽ giữa 2 tác vụ render nội dung website và download file từ Internet với nhau.

2.4.3 Sử dụng concurrency để tối ưu I/O bound

Concurrency có thể được sử dụng để cải thiện hiệu suất của các chương trình I/O bound, là những chương trình dành một lượng thời gian đáng kể để chờ các hoạt động đầu vào/đầu ra (I/O) hoàn tất, chẳng hạn như đọc hoặc ghi dữ liệu qua đĩa hoặc mạng. Dưới đây là một số cách có thể sử dụng concurrency để cải thiện hiệu suất của các chương trình I/O bound:

- **Asynchronous I/O:** Một cách để cải thiện hiệu suất của chương trình I/O bound là sử dụng asynchronous I/O, còn được biết đến là non-blocking I/O. Với asynchronous I/O, chương trình có thể đưa ra nhiều request I/O cùng một

lúc và tiếp tục thực thi các tác vụ khác trong khi chờ các thao tác I/O hoàn tất. Điều này có thể giảm đáng kể lượng thời gian mà một chương trình dành để chờ đợi các thao tác I/O hoàn tất.

- **Multithreading (Đa luồng):** Một cách khác để cải thiện hiệu suất của các chương trình I/O bound là sử dụng đa luồng. Với đa luồng, một chương trình có thể tạo nhiều luồng để thực thi đồng thời các tác vụ khác nhau. Ví dụ, một chương trình có thể tạo một luồng để thực hiện các thao tác I/O trong khi một luồng khác tiếp tục thực hiện các tác vụ khác. Điều này có thể cải thiện hiệu suất bằng cách cho phép chương trình tiếp tục thực thi các tác vụ khác trong khi chờ các thao tác I/O hoàn tất.
- **Parallel processing (Xử lý song song):** Nếu một chương trình I/O bound đang xử lý một lượng lớn dữ liệu, thì có thể sử dụng xử lý song song để chia dữ liệu thành các phần nhỏ hơn và xử lý chúng đồng thời trên nhiều bộ xử lý CPU. Điều này có thể cải thiện hiệu suất bằng cách cho phép nhiều bộ xử lý hoạt động đồng thời trên các phần khác nhau của dữ liệu.

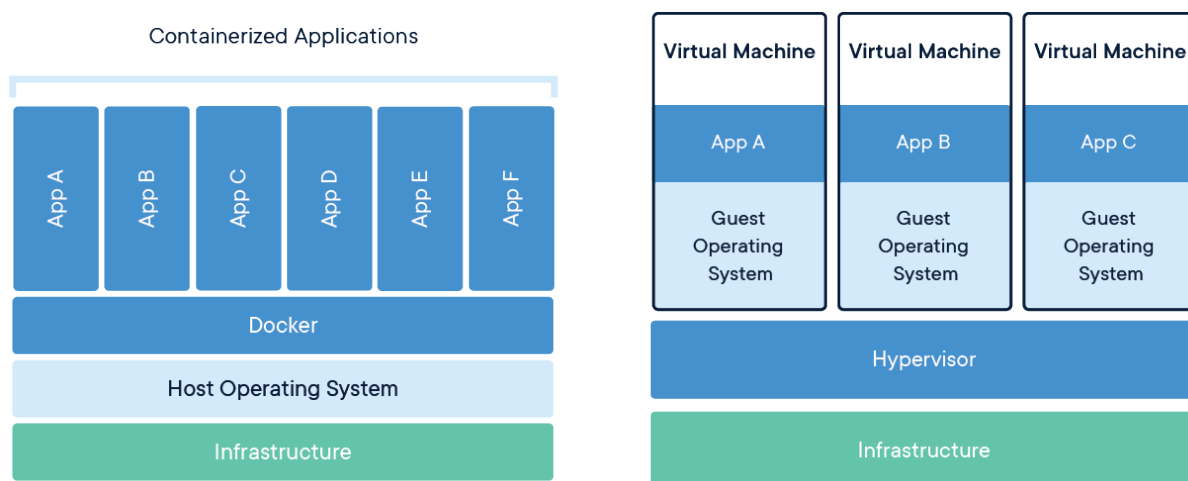
Tóm lại, concurrency có thể được sử dụng để cải thiện hiệu suất của các chương trình I/O bound bằng cách cho phép xử lý asynchronous I/O (I/O không đồng bộ), multithreading (đa luồng) và parallel processing (xử lý song song). Những kỹ thuật này có thể giảm lượng thời gian mà một chương trình dành để chờ đợi các thao tác I/O hoàn tất, cho phép chương trình tiếp tục thực hiện các tác vụ khác và cải thiện hiệu suất tổng thể.

2.5 Docker

Việc triển khai một hệ thống, việc cài đặt môi trường khá là vất vả, và có thể gây lỗi nếu như có xung đột trong hệ thống. Những ngôn ngữ lập trình khác nhau, hệ quản trị cơ sở dữ liệu khác nhau lại yêu cầu các phần mềm hỗ trợ, gói thư viện và môi trường khác nhau. Chính vì lý do đó Docker được ra đời để giải quyết vấn đề này.

Mặc dù máy ảo như VMWare hay Virtual Box cũng có thể tạo ra các môi trường ảo, độc lập với hệ điều hành gốc. Tuy nhiên, hạn chế của máy ảo đó là cài đặt phức tạp, tốn nhiều tài nguyên và chậm hơn so với việc sử dụng Docker. Khi cài

đặt máy ảo, ta phải cấp phát cứng RAM và dữ liệu cho máy ảo, dẫn đến việc hệ điều hành gốc phải cấp phát lượng lớn tài nguyên cho các máy ảo đồng thời làm giảm hiệu năng của cả máy ảo lẫn hệ điều hành gốc. Còn Docker sử dụng cấu trúc container, các container tạo ra được sử dụng chung kernel và phần cứng với hệ điều hành gốc như RAM, CPU. Đồng thời việc tạo ra một môi trường ảo trong Docker cũng rất nhanh chóng và dễ dàng.



Hình 2.11: Sự khác biệt giữa Docker và Virtual Machine

Docker được định nghĩa là một công cụ giúp có thể đóng gói và triển khai các ứng dụng, hệ thống trên một môi trường ảo hóa, được định nghĩa rõ ràng, độc lập với môi trường gốc. Các ứng dụng chạy bên trong Docker được gọi là container. Vì mỗi dự án (project) được triển khai trên môi trường khác nhau nên các project sẽ không bị xung đột phiên bản lẫn nhau. Ví dụ như project A yêu cầu Python có phiên bản 3.4 nhưng project B yêu cầu Python phiên bản 3.8, nếu không sử dụng Docker thì lúc này trên hệ điều hành gốc sẽ bị xung đột. Từ đó Docker giảm thiểu sự phụ thuộc lẫn nhau, có thể tùy thích cài đặt thư viện, ứng dụng khác nhau mà không bị ảnh hưởng đến các project khác cũng như hệ điều hành gốc. Ngoài ra, Docker còn có thể chạy trên nhiều nền tảng khác nhau, cho dù hệ điều hành gốc dùng Windows, Linux hay MacOS thì khi build Docker vẫn có thể chạy ổn định.

Các khái niệm cơ bản trong Docker:

- **Image:** Image là nơi lưu trữ các cài đặt môi trường như hệ điều hành, thư viện, ứng dụng cần triển khai. Nếu nhìn nhận Docker theo hướng lập trình đối

tượng thì image được hiểu như là một class còn Container là các instance hay object của class đó. Vậy nên có thể tạo nhiều container với môi trường giống hệt nhau từ một image.

- **Container:** Container là một môi trường độc lập được tạo từ một Docker Image. Mỗi container có thể chạy một ứng dụng riêng biệt và được cô lập hoàn toàn với các container khác trên cùng một hệ điều hành gốc.
- **Port:** Vì môi trường trong Docker độc lập hoàn toàn so với môi trường gốc. Nên để có thể sử dụng được ứng dụng chạy trong Docker từ hệ điều hành gốc thì ta map port từ Docker ra port bên ngoài để bên ngoài truy cập được.
- **Volume:** Trong Docker, volume là một khái niệm quan trọng để quản lý dữ liệu trong các container. Volume là một cơ chế để lưu trữ và chia sẻ dữ liệu giữa các container hoặc giữa container và host.
- **Network:** Docker network cho phép các container trên một host có thể liên lạc được với nhau, hoặc các container trên nhiều host có thể liên lạc được với nhau. Các Docker network giúp tạo ra một môi trường mạng riêng cho các container, vì vậy các container có thể truy cập vào các nguồn tài nguyên bên ngoài một cách an toàn và bảo mật.
- **Docker-compose:** Docker compose là một công cụ giúp định nghĩa và chạy nhiều container cho ứng dụng Docker. Với Docker compose, ta có thể config các services để phục vụ cho ứng dụng. Và tiện hơn khi chỉ với một câu lệnh, chúng ta có thể tạo và khởi chạy tất cả các container service mà được định nghĩa trong file `docker-compose.yml`.

2.6 ELK stack

2.6.1 Logging

Logging là một công cụ đơn giản và mạnh mẽ, ghi lại toàn bộ những hoạt động của hệ thống. Logging là một phần không thể thiếu trong bất kì hệ thống nào, nó giúp lưu lại dấu vết, hoạt động của ứng dụng, giúp phân tích từ đó nhằm đưa ra những hướng giải quyết phù hợp.

2.6.2 Định nghĩa

ELK stack là một bộ công cụ mã nguồn mở Elasticsearch, Logstash và Kibana được sử dụng để thu thập, lưu trữ, xử lý và truy vấn các log từ các ứng dụng và hệ thống khác nhau phục vụ cho công việc logging. [5]

- **Elasticsearch:** Cơ sở dữ liệu dùng để lưu trữ, tìm kiếm, phân tích và query log.
- **Logstash:** là một công cụ sử dụng để thu thập, xử lý log. Nhiệm vụ của Logstash là tiếp nhận log từ nhiều nguồn, sau đó xử lý log và ghi dữ liệu vào Elasticsearch.
- **Kibana:** là một công cụ cho phép trực quan hóa dữ liệu được lưu trong Elasticsearch.

2.6.3 Luồng hoạt động của ELK stack

- **Thu thập log:** Logstash là công cụ được sử dụng để thu thập và xử lý các log từ các nguồn khác nhau (server gửi UDP request chứa log tới logstash, Beat¹ đọc file log và gửi lên logstash hay logstash có thể đọc dữ liệu từ topic trong Kafka)
- **Xử lý log:** Logstash tiếp nhận các log, lọc và chuẩn hóa chúng, sau đó chuyển đổi chúng thành định dạng có thể truy vấn được, cuối cùng ghi xuống cơ sở dữ liệu Elasticsearch.
- **Lưu trữ log:** Elasticsearch là một cơ sở dữ liệu tìm kiếm và phân tán được sử dụng để lưu trữ các log đã được xử lý. Elasticsearch sử dụng một cơ chế phân tán và tối ưu hóa tìm kiếm để tìm kiếm log một cách nhanh chóng và hiệu quả.
- **Truy vấn log:** Kibana là một giao diện người dùng web để tìm kiếm, truy vấn và hiển thị các log đã được lưu trữ trong Elasticsearch. Kibana cho phép người dùng tạo các bảng điều khiển, biểu đồ và thông báo để hiển thị thông tin log một cách trực quan.

¹<https://www.elastic.co/beats/>



Hình 2.12: Luồng hoạt động của ELK stack

Với ELK Stack, các log được thu thập từ các nguồn khác nhau, được xử lý và lưu trữ một cách phân tán, và được truy vấn bằng cách sử dụng một giao diện người dùng trực quan. Điều này giúp cho việc quản lý các log trở nên dễ dàng và hiệu quả hơn, giúp người quản trị hệ thống có thể phát hiện các sự cố và tìm kiếm nguyên nhân một cách nhanh chóng và chính xác. Ngoài ra, Elasticsearch còn cung cấp một khả năng search và filter mạnh mẽ, hỗ trợ Full-Text Search nên việc query rất dễ dàng.

2.7 Prometheus, Grafana và Exporter

2.7.1 Định nghĩa

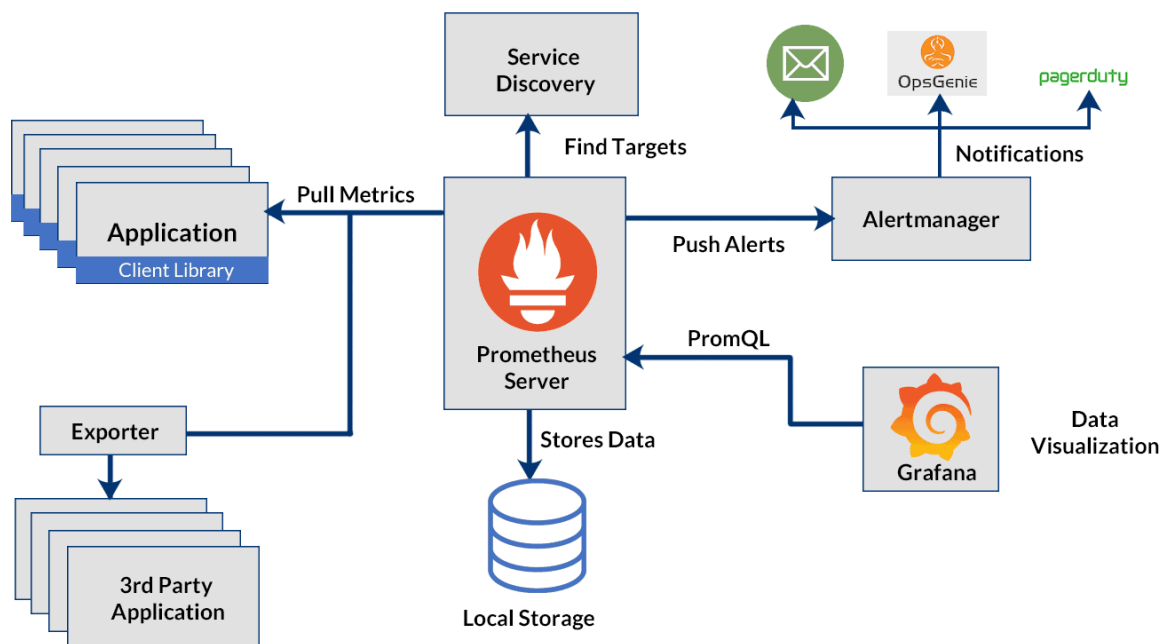
Prometheus, Grafana và Exporter là ba công cụ thường được sử dụng trong việc monitor và quản lý hệ thống.

- **Prometheus** là một hệ thống giám sát mã nguồn mở, được sử dụng để thu thập và lưu trữ các metric từ các nguồn khác nhau, chẳng hạn như máy chủ, ứng dụng và container. Có khả năng truy vấn mạnh mẽ với PromQL và lưu trữ các metric trong cơ sở dữ liệu time-series. Ngoài ra, Prometheus còn cung cấp một hệ thống cảnh báo tích hợp, cho phép người dùng nhận được cảnh báo khi các metric vượt quá ngưỡng nhất định.
- **Grafana** là một công cụ trực quan hoá, thường được sử dụng để hiển thị các metric được thu thập bởi Prometheus. Nó cung cấp một loạt các tùy chọn trực quan hóa, bao gồm đồ thị, bảng và heatmap, và cho phép người dùng tạo các trang điều khiển tùy chỉnh.

- **Exporter** là một công cụ giúp thu thập metric từ các nguồn dữ liệu khác nhau và chuyển đổi chúng sang định dạng được hỗ trợ bởi Prometheus. Exporter cung cấp các phương pháp khác nhau để thu thập metric, bao gồm các metric của hệ điều hành, ứng dụng, cơ sở dữ liệu, và nhiều hơn nữa. Các exporter có thể được triển khai như các container Docker hoặc các ứng dụng độc lập.

2.7.2 Luồng hoạt động của Prometheus, Grafana và Exporter

- **Thu thập metric:** Exporter thu thập các metric từ các nguồn dữ liệu khác nhau, chẳng hạn như ứng dụng, cơ sở dữ liệu, Message Broker hoặc hệ thống máy chủ.
- **Chuyển đổi metric:** Exporter chuyển đổi các metric thu thập được sang định dạng được hỗ trợ bởi Prometheus.
- **Lưu trữ metric:** Prometheus thu thập các metric từ các exporter và lưu trữ chúng trong cơ sở dữ liệu time-series. Prometheus cung cấp PromQL, một ngôn ngữ truy vấn mạnh mẽ để truy vấn và phân tích các metric thu thập được.
- **Truy vấn metric:** Grafana kết nối đến Prometheus và các nguồn dữ liệu khác để truy xuất các metric. Grafana sử dụng các metric để hiển thị các biểu đồ, bảng và các trang điều khiển khác để giúp người dùng hiểu và quản lý các hệ thống.



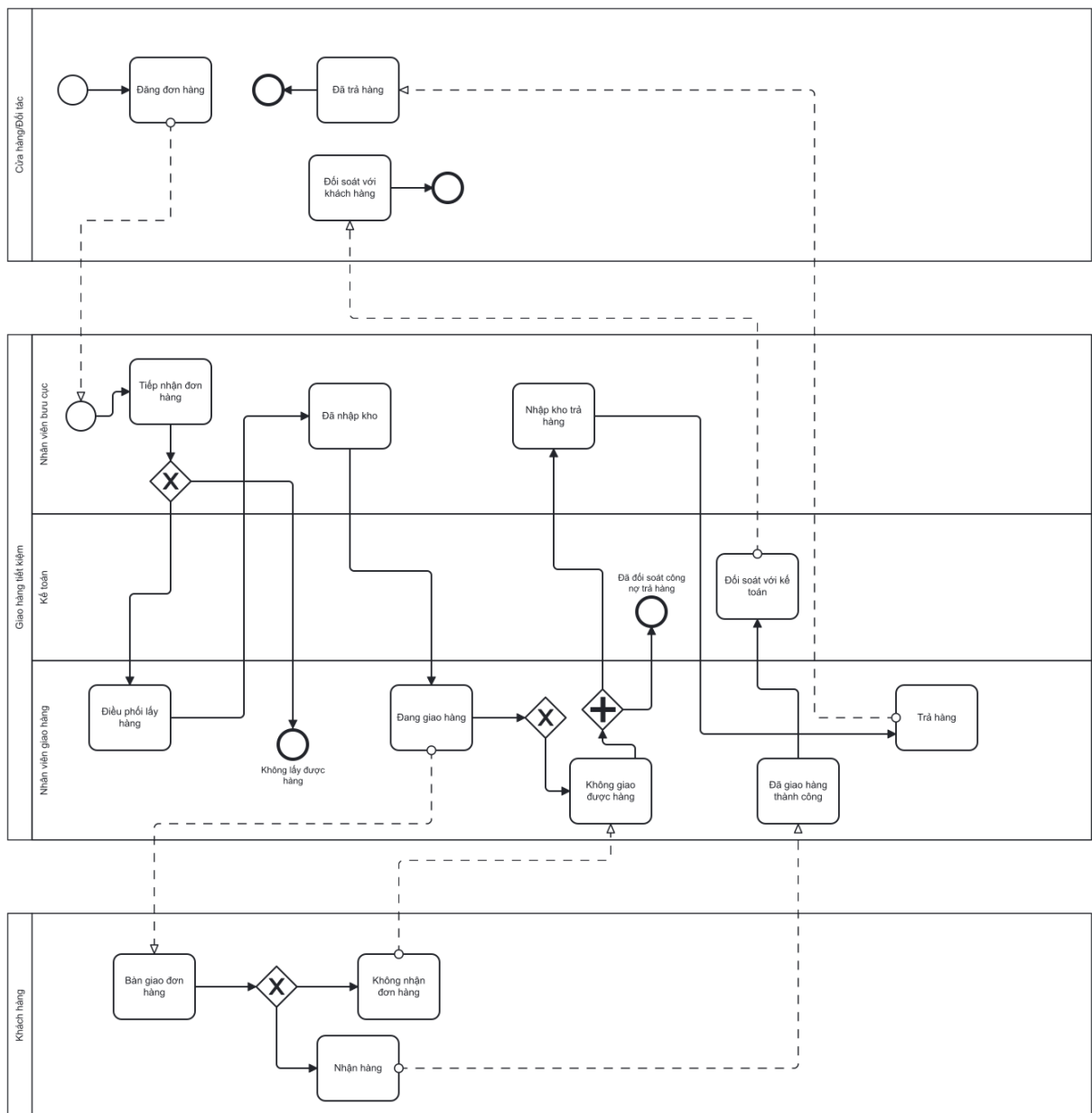
Hình 2.13: Luồng hoạt động của Prometheus, Grafana và Exporter [6]

Chương 3: Giải pháp xây dựng hệ thống theo dõi đơn hàng tự động với thông lượng lớn

Chương 3 sẽ đưa ra giải pháp xây dựng một hệ thống theo dõi đơn hàng tự động với thông lượng lớn bằng cách lần lượt xây dựng hai hệ thống chính là hệ thống theo dõi sự thay đổi dữ liệu ở cơ sở dữ liệu và hệ thống xử lý webhook. Đồng thời, khóa luận cũng sẽ cung cấp giải pháp ưu tiên đơn hàng để tăng thông lượng.

3.1 Quy trình tổng quan của một đơn hàng

Trong quá trình vận chuyển hàng hoá, việc quản lý và xử lý đơn hàng là một phần quan trọng để đảm bảo sự hài lòng của khách hàng. Công ty Giao hàng tiết kiệm hiểu được điều này và đã thiết lập một quy trình quản lý đơn hàng chặt chẽ và toàn diện. Từ khi khách hàng đặt hàng trên trang web của công ty, đến khi sản phẩm được giao đến tay khách hàng, mọi bước trong quy trình đều được Giao hàng tiết kiệm chú trọng và quản lý cẩn thận. Về cơ bản, quy trình xử lý một đơn hàng của Giao hàng tiết kiệm (GHTK) được thực hiện theo mô hình sau:

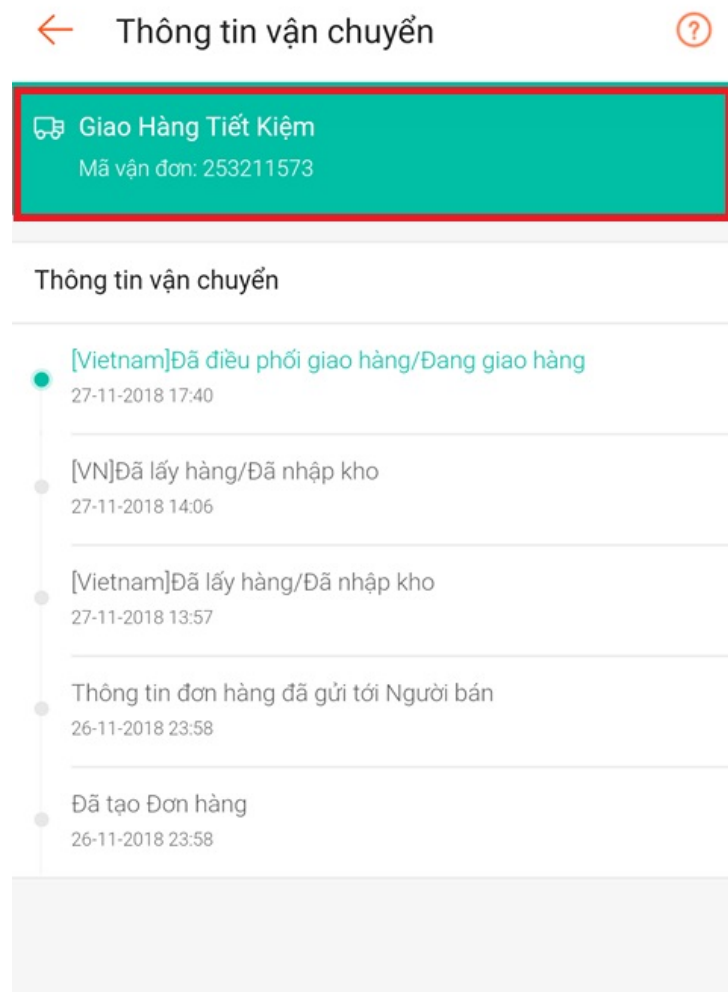


Hình 3.1: Mô hình BPMN quy trình đơn hàng của Giao hàng tiết kiệm

Trạng thái	Mô tả
-1	Hủy đơn hàng
1	Chưa tiếp nhận
2	Đã tiếp nhận
3	Đã lấy hàng/Đã nhập kho
4	Đã điều phối giao hàng/Đang giao hàng
5	Đã giao hàng/Chưa đối soát
6	Đã đối soát
7	Không lấy được hàng
8	Hoãn lấy hàng
9	Không giao được hàng
10	Hoãn giao hàng
11	Đã đối soát công nợ trả hàng
12	Đã điều phối lấy hàng/Đang lấy hàng
13	Đơn hàng bồi hoàn
20	Đang trả hàng (COD cầm hàng đi trả)
21	Đã trả hàng (COD đã trả xong hàng)

Bảng 3.1: Các trạng thái chuyển của đơn hàng trên hệ thống Giao hàng tiết kiệm

Sau khi khách hàng đăng đơn thành công, mỗi khi đơn hàng đi qua các công đoạn khác nhau sẽ thay đổi trạng thái. Trạng thái đơn hàng được thay đổi và các thông tin liên quan đến đơn hàng sẽ được gửi đến các đối tác thông qua webhook. Đối tác có thể sử dụng các thông tin đơn hàng thay đổi để hiển thị cho khách hàng của họ biết về thông tin đơn hàng của mình đang đến đâu. Ví dụ dưới đây là thông tin vận chuyển trên Shopee mà Giao hàng tiết kiệm cung cấp trạng thái, thông tin đơn hàng thông qua webhook để Shopee có thể hiển thị cho khách hàng của họ.



Hình 3.2: Kiểm tra thông tin đơn hàng trên Shopee

3.2 Xây dựng hệ thống nắm bắt sự thay đổi dữ liệu từ cơ sở dữ liệu

3.2.1 Phân tích bài toán

Như đã đề cập ở mục 1.1, bài toán nắm bắt sự thay đổi dữ liệu là bài toán quan trọng để có thể giúp hệ thống nhận biết được thông tin của đơn hàng vừa được thay đổi trong cơ sở dữ liệu từ đó có thể gửi đến đối tác. Bài toán nắm bắt sự thay đổi dữ liệu là bài toán nhỏ hơn trong bài toán tích hợp dữ liệu giữa các hệ thống (system integration). Tích hợp dữ liệu hệ thống là quá trình kết hợp các hệ thống và ứng dụng khác nhau để chia sẻ thông tin và dữ liệu giữa chúng. Mục đích của

tích hợp dữ liệu hệ thống là tạo ra một hệ thống toàn diện, cung cấp cho các hệ thống dữ liệu đầy đủ và chính xác từ các nguồn dữ liệu.

Các hệ thống và ứng dụng khác nhau trong một tổ chức có thể được phát triển bởi các công nghệ khác nhau, được lưu trữ trên các máy chủ và cơ sở dữ liệu khác nhau, sử dụng các chuẩn khác nhau. Tích hợp dữ liệu hệ thống giúp kết nối các hệ thống này lại với nhau, cho phép chúng tương tác và trao đổi thông tin, tránh việc lặp lại dữ liệu và đảm bảo tính toàn vẹn và chính xác của dữ liệu.

Một trong những phương pháp có thể được áp dụng bằng cách theo dõi dữ liệu thay đổi ở tầng ứng dụng bằng cách hook (gọi) một hàm có khả năng theo dõi dữ liệu vào các hàm làm thay đổi thông tin đơn hàng. Ví dụ trong service Xử lý đơn hàng (Package Service) có hàm thay đổi trạng thái đơn hàng sau khi nhân viên giao hàng báo giao hàng thành công. Bằng cách hook 1 hàm nắm bắt dữ liệu nằm trong hàm này có thể giúp bắt được sự thay đổi của trạng thái đơn hàng rồi gửi thông tin trạng thái được cập nhật này cho đối tác bằng webhook. Có thể thấy phương pháp này có thể dễ dàng triển khai, không tốn nhiều thời gian để cài đặt. Nhưng phương pháp nào cũng có sự đánh đổi, nhược điểm của phương pháp này là:

- **Dễ gây lỗi:** Việc lặp lại nhiều lần có thể dẫn đến việc xảy ra lỗi nếu có sự thay đổi về chức năng hoặc thiếu cập nhật đồng bộ trong các hàm.
- **Khó bảo trì:** Khi cần thay đổi chức năng, ta phải sửa đổi nhiều hàm liên quan đến chức năng đó. Điều này làm cho việc bảo trì hệ thống trở nên khó khăn và tốn thời gian.
- **Tốn tài nguyên:** Nếu mỗi khi thay đổi trạng thái đơn hàng lại gọi đến các hàm tương tự nhau, điều này sẽ tốn nhiều tài nguyên hệ thống và có thể làm chậm tốc độ xử lý.
- **Khó mở rộng:** Nếu hệ thống cần thêm các chức năng mới liên quan đến cập nhật trạng thái đơn hàng, việc lặp lại các hàm có thể làm cho việc mở rộng hệ thống trở nên khó khăn và tốn nhiều thời gian.

Vì vậy, tốt nhất là nên sử dụng một phương pháp chung để xử lý cập nhật trạng thái đơn hàng, giảm thiểu sự lặp lại và tăng tính bảo trì và mở rộng của hệ thống.

Thay vì theo dõi dữ liệu ở tầng ứng dụng ta nên theo dõi dữ liệu ở tầng cơ sở dữ liệu vì mọi sự thay đổi dữ liệu cuối cùng đều nằm ở cơ sở dữ liệu.

3.2.2 Giải pháp thực hiện

Change Data Capture (CDC) được tích hợp vào để xây dựng hệ thống nắm bắt sự thay đổi dữ liệu từ cơ sở dữ liệu. CDC có thể được triển khai qua các phương pháp Log-based, Trigger-based và Timestamps-based như đã đề cập ở mục 2.3.3. So sánh các phương pháp trên, ta thu được bảng sau:

Phương pháp CDC	Hỗ trợ cập nhật dữ liệu theo thời gian thực	Ảnh hưởng tới cơ sở dữ liệu
Log-based	Có	Ít ảnh hưởng đến cơ sở dữ liệu
Trigger-based	Có	Ảnh hưởng lớn đến hiệu suất của cơ sở dữ liệu vì sử dụng trigger thường xuyên và cần tạo thêm bảng
Timestamps-based	Không	Ảnh hưởng đến hiệu suất của cơ sở dữ liệu vì phải update lại MODIFIED sau mỗi lần cập nhật

Bảng 3.2: So sánh sự khác nhau giữa các phương pháp CDC [7]

Dựa vào bảng so sánh sự khác nhau giữa các phương pháp CDC và các cơ sở lý thuyết đã được đề cập ở phần 2.3, ta có thể đưa ra nhận xét như sau:

- **Phương pháp trigger-based CDC** có ưu điểm là có thể theo dõi và bắt được các thay đổi dữ liệu ngay lập tức khi chúng xảy ra, không cần phải chờ đến thời điểm đồng bộ hóa. Tuy nhiên, phương pháp này cũng có nhược điểm là sử dụng trigger có thể ảnh hưởng đến hiệu suất của cơ sở dữ liệu và tốn tài nguyên hệ thống. Do đó, khi triển khai phương pháp này, cần phải đảm bảo rằng hệ thống có đủ tài nguyên để xử lý các trigger và bảng tạm thời được sử dụng.
- **Phương pháp timestamp-based CDC** có ưu điểm là dễ dàng triển khai,

không yêu cầu nhiều tài nguyên và không ảnh hưởng đến hiệu suất của cơ sở dữ liệu. Tuy nhiên, phương pháp này cũng có một số hạn chế như không thể xác định được sự thay đổi của một số trường dữ liệu cụ thể, đặc biệt là các trường kiểu blob hoặc text, vì chúng thường không có thời gian cập nhật.

- **Phương pháp log-based CDC** thường được sử dụng trong các hệ thống có nhiều giao dịch thay đổi dữ liệu và yêu cầu xử lý dữ liệu theo thời gian thực. Nó cho phép các hệ thống khác nhau truy cập dữ liệu đã thay đổi một cách nhanh chóng và dễ dàng, giảm thiểu thời gian và công sức cần thiết để tích hợp dữ liệu giữa các hệ thống. Tuy nhiên, phương pháp log-based CDC cũng có một số nhược điểm, bao gồm độ phức tạp khi triển khai và cấu hình, sử dụng tài nguyên của cơ sở dữ liệu để lưu trữ và quản lý log transaction, và đòi hỏi các công cụ và phương pháp chuyên môn để trích xuất và xử lý dữ liệu từ log transaction.

Dựa vào yêu cầu bài toán là cần một hệ thống nắm bắt sự thay đổi dữ liệu theo thời gian thực, không ảnh hưởng đến hiệu suất của cơ sở dữ liệu, có thể nhận thấy Log-based là phương pháp phù hợp để triển khai trong khóa luận. Phương pháp này đảm bảo tính **ACID**¹ khi làm việc với transaction, ít ảnh hưởng đến hiệu năng của cơ sở dữ liệu và quan trọng nhất là hỗ trợ cập nhật dữ liệu theo thời gian thực qua đó sẽ cập nhật dữ liệu mới nhất cho phía đối tác. Tuy nhiên, việc triển khai Log-based sẽ yêu cầu cơ sở dữ liệu phải có *transaction log*. Transaction log là một file trong database mà lưu trữ thông tin các thao tác cập nhật dữ liệu transaction được thực hiện lên database, hoạt động ngầm bên dưới database. Thông thường dữ liệu trong database sẽ bao gồm hai thành phần:

- data file, dùng để chứa các thông tin để hình thành nên cơ sở dữ liệu (dữ liệu trong database, thông tin về cấu trúc database, cấu trúc bảng, indexes,...)
- log file (transaction log) sẽ ghi lại tất cả các transaction hoặc thay đổi do các câu lệnh DML, DDL được thực hiện đối với dữ liệu trong cơ sở dữ liệu.

Lí do phải tách thành hai thành phần dữ liệu như vậy bởi vì do hiệu suất xử lý. Vì data file là một cấu trúc dữ liệu phức tạp và để thực hiện các thao tác (lưu trữ vào ổ cứng, quản lý dữ liệu) trong một cấu trúc dữ liệu phức tạp thì sẽ rất mất thời

¹4 tính chất Atomicity, Consistency, Isolation, và Durability

gian. Vậy nên phải chia dữ liệu đầu vào thành hai phần, bản thân giá trị dữ liệu cần lưu trữ sẽ đưa vào buffer của RAM thay vì lưu thẳng vào ổ cứng, mỗi thao tác với dữ liệu sẽ ngay lập tức được đưa vào 1 file khác transaction log. Vì transaction log lưu trữ dữ liệu theo cơ chế tuần tự và theo một cấu trúc đơn giản nên xử lý sẽ rất nhanh.

Transaction log không tồn tại ở tất cả các hệ quản trị cơ sở dữ liệu, ví dụ đối với MySQL file transaction log được gọi là *binary log* hay PostgreSQL thì transaction log được gọi là *write-ahead log (WAL)*. Khóa luận sử dụng MySQL làm hệ quản trị cơ sở dữ liệu vì MySQL là open-source, có hiệu suất cao, tương thích mạnh với các ngôn ngữ và quan trọng là có hỗ trợ transaction log. Trong MySQL, binlog được sử dụng để phục hồi dữ liệu và sao chép dữ liệu.

Bằng cách phân tích dữ liệu từ binary log (binlog), kỹ thuật CDC trích xuất ra các thông tin cần thiết để theo dõi các thay đổi dữ liệu. Các bước cơ bản được triển khai CDC với binlog trong MySQL như sau:

- *Bước 1*, Kích hoạt binary logging: Chế độ binary logging phải được kích hoạt để dữ liệu transaction được ghi vào file binlog. Kích hoạt bằng cách thêm dòng `log_bin = /path/to/binlog` vào file cấu hình `my.cnf` của MySQL.
- *Bước 2*, Cấu hình binary logging: Có một số tùy chọn cấu hình cho binary logging trong MySQL bao gồm: định dạng của log file, kích thước tối đa của log file và thời gian lưu trữ log file. Các tùy chọn này có thể được cấu hình bằng các biến `binlog_format`, `max_binlog_size`, `expire_log_days` tương ứng.
- *Bước 3*, Trích xuất các thay đổi từ binlog: Để trích xuất các thay đổi từ binlog, một chương trình hoặc một đoạn mã (script) được sử dụng để phân tích file binlog và trích xuất các thông tin cần thiết. Hiện nay, có một số công cụ có sẵn có thể sử dụng cho việc này, bao gồm công cụ `mysqlbinlog` của MySQL, có thể được sử dụng để trích xuất nội dung file binlog thành định dạng mà con người có thể đọc hiểu.
- *Bước 4*, Áp dụng các thay đổi cho hệ thống đích: Khi các dữ liệu thay đổi đã được trích xuất từ binlog, các dữ liệu này phải được tải sang hệ thống đích để đồng bộ với cơ sở dữ liệu. Điều này có thể được thực hiện bằng nhiều kỹ thuật khác nhau chẳng hạn đưa các dữ liệu vào Message Broker nhằm tăng khả năng xử lý dữ liệu.

3.2.3 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống được thiết kế gồm các bảng sau để mô tả cơ bản về cách tổ chức dữ liệu của một hệ thống logistic:

Bảng packages: Lưu thông tin về đơn hàng, đơn hàng là thành phần quan trọng nhất trong hệ thống, dữ liệu của đơn hàng sẽ được triển khai CDC để theo dõi.

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
id	int	primary key	Mã của đơn hàng, mã này được sinh tự động bằng uuid
code	varchar	unique	Mã định danh đơn hàng
shop_id	int	foreign key	Mã của cửa hàng
current_station_id	int	foreign key	Mã của kho hiện tại
customer_id	int	foreign key	Mã của khách hàng
cod_id	int	foreign key	Mã của nhân viên giao hàng
status	int	foreign key	Trạng thái của đơn hàng
picked_at	datetime		Thời điểm nhân viên lấy hàng từ khách hàng
delivered_at	datetime		Thời điểm nhân viên giao hàng giao hàng thành công
done_at	datetime		Thời điểm nhân viên giao hàng hoàn lại tiền cho kế toán
audited_at	datetime		Thời điểm đối soát vận chuyển
created	datetime		Thời điểm đơn hàng được tạo
modified	datetime		Thời điểm đơn hàng được cập nhật

Bảng 3.3: Thiết kế cơ sở dữ liệu bảng packages

Bảng shops: Lưu thông tin về cửa hàng, một cửa hàng có nhiều đơn hàng, cửa hàng này có thể đăng ký dịch vụ webhook để nhận thông tin về trạng thái đơn hàng.

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
id	int	primary key	Mã của cửa hàng
name	varchar		Tên của cửa hàng
email	varchar		Email của cửa hàng
webhook_url	int		Endpoint URL để webhook gửi thông tin đến
created	datetime		Thời điểm cửa hàng được tạo
modified	datetime		Thời điểm cửa hàng được cập nhật

Bảng 3.4: Thiết kế cơ sở dữ liệu bảng shops

Bảng stations: Lưu thông tin về kho lưu trữ đơn hàng.

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
id	int	primary key	Mã của kho
name	varchar		Tên của cửa hàng
address_id	int	foreign key	Địa chỉ của kho
created	datetime		Thời điểm kho lưu trữ đơn được tạo
modified	datetime		Thời điểm kho lưu trữ đơn được cập nhật

Bảng 3.5: Thiết kế cơ sở dữ liệu bảng stations

Bảng customers: Lưu thông tin về khách hàng đặt hàng.

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
id	int	primary key	Mã của khách hàng
name	varchar		Tên của khách hàng
email	varchar		Email của khách hàng
address_id	int	foreign key	Địa chỉ của khách hàng
created	datetime		Thời điểm thông tin khách hàng được tạo
modified	datetime		Thời điểm thông tin khách hàng được cập nhật

Bảng 3.6: Thiết kế cơ sở dữ liệu bảng customers

Bảng addresses: Lưu thông tin về địa chỉ.

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
id	int	primary key	Mã của địa chỉ
name	varchar		Tên của địa chỉ
created	datetime		Thời điểm thông tin địa chỉ được tạo
modified	datetime		Thời điểm thông tin địa chỉ được cập nhật

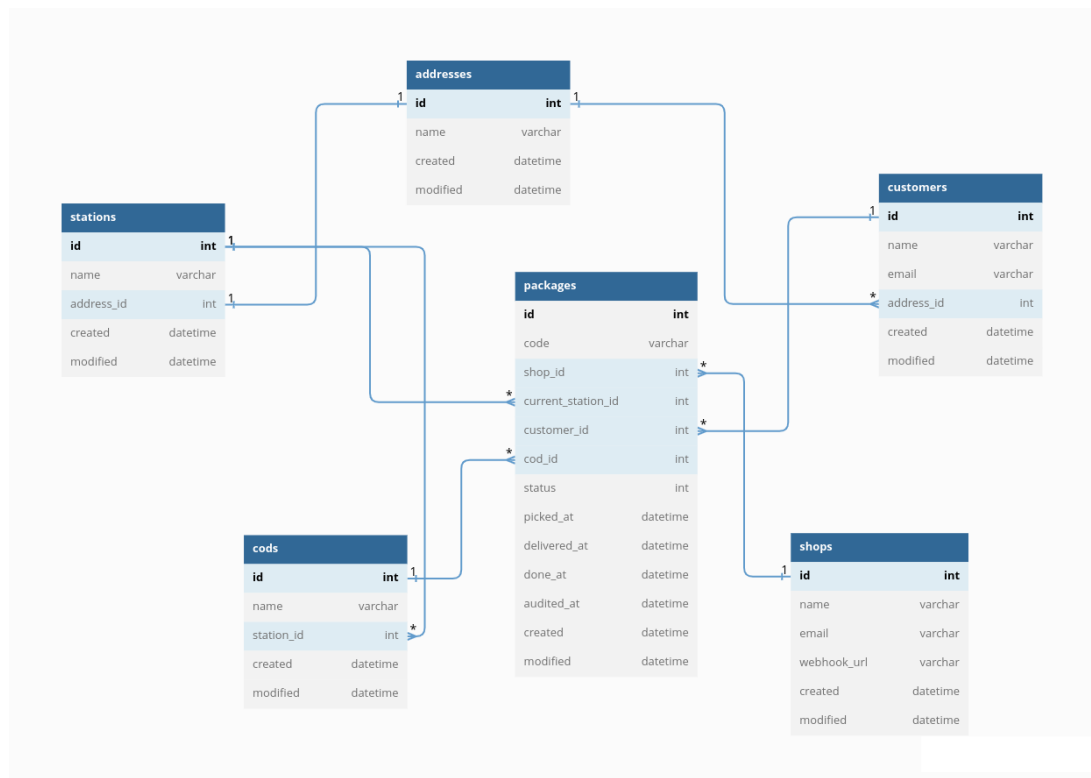
Bảng 3.7: Thiết kế cơ sở dữ liệu bảng addresses

Bảng cods: Lưu thông tin về nhân viên lấy/giao hàng.

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
id	int	primary key	Mã của nhân viên lấy/giao hàng
name	varchar		Tên của nhân viên lấy/giao hàng
station_id	int	foreign key	Địa chỉ kho của nhân viên lấy/-giao hàng
created	datetime		Thời điểm nhân viên lấy/giao hàng được tạo
modified	datetime		Thời điểm nhân viên lấy/giao hàng được cập nhật

Bảng 3.8: Thiết kế cơ sở dữ liệu bảng cods

Dưới đây là thiết kế lược đồ cơ sở dữ liệu của hệ thống:



Hình 3.3: Lược đồ cơ sở dữ liệu của hệ thống

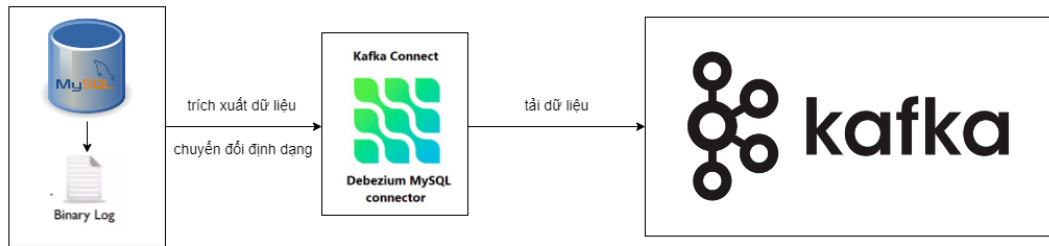
3.2.4 Log-based CDC với Debezium

Debezium là một công cụ dùng để CDC theo phương pháp Log-based, cốt lõi của Debezium là sử dụng Kafka để tạo ra các message tương ứng với các transaction thay đổi dữ liệu. Debezium sử dụng các Kafka Connector (đề cập ở mục 2.2.5) để kết nối đến các cơ sở dữ liệu tương thích như MySQL, PostgreSQL, MongoDB, SQL Server, Oracle,...từ đó có thể bắt các event thay đổi dữ liệu từ transaction log và truyền tải message đến Kafka topic. Sau đó các Kafka consumer sẽ đọc dữ liệu từ topic này và xử lý.

Bằng cách tận dụng nền tảng streaming message của Kafka, Debezium giúp các ứng dụng có thể bắt được các sự thay đổi xảy ra trong cơ sở dữ liệu một cách chính xác và đầy đủ. Ngay cả khi ứng dụng mất kết nối hoặc dừng đột ngột khi xảy ra lỗi, các event xảy ra trong thời gian ứng dụng dừng hoạt động được lưu trong các topic của Kafka. Do đó, sau khi ứng dụng khởi động lại, nó sẽ tiếp tục thực hiện từ vị trí mà nó đã dừng lại khi gặp lỗi.

Kiến trúc phổ biến nhất của hệ thống sử dụng Debezium để CDC đó chính là tích hợp Debezium cùng với Kafka Connect. Kafka Connect như đã giới thiệu ở chương mục 2.2.5 thì đây là một service riêng biệt chạy song song với Kafka cluster, mục đích là để truyền tải dữ liệu từ các hệ thống nguồn đến Kafka và các hệ thống đích sẽ lấy dữ liệu từ Kafka. Với kiến trúc như hình 3.4, các thành phần trong hệ thống nắm bắt sự thay đổi dữ liệu được mô tả như sau:

- Hệ thống sử dụng cơ sở dữ liệu là MySQL.
- Debezium MySQL connector tương ứng với hệ quản trị cơ sở dữ liệu MySQL. Debezium MySQL connector sẽ đọc dữ liệu từ binlog khi có các sự kiện thay đổi dữ liệu đối với các bản ghi bằng các câu lệnh `INSERT`, `UPDATE`, `DELETE`. Connector này sẽ được Kafka Connect quản lý và truyền dữ liệu đến Kafka topic.
- Apache Kafka là một Message Broker nhận dữ liệu từ Kafka Connect.



Hình 3.4: Mô hình hệ thống nắm bắt sự thay đổi dữ liệu

Một ví dụ dưới đây chỉ ra message được Debezium trích xuất, xử lý và tải dữ liệu vào Kafka topic sau khi thay đổi trạng thái đơn hàng trong bảng *packages*:

```

{
  "schema": {
    "type": "struct",
    "fields": [
      {
        "type": "struct",
        "fields": [
          {
            "type": "int32",
            "optional": false,
            "field": "id"
          }
        ],
        "optional": true,
        "name": "connector.logistic.packages.Value",
        "field": "before"
      },
      {
        "optional": true,
        "name": "connector.logistic.packages.Value",
        "field": "after"
      }
    ],
    "optional": false,
    "name": "connector.logistic.packages.Envelope",
    "version": 1
  },
  "payload": {
    "before": {
      "id": 3,
      "shop_id": 3,
      "current_station_id": 4,
      "customer_id": 4,
      "status": 2,
      "pkg_order": "P503"
    },
    "after": {
      "id": 3,
      "shop_id": 3,
      "current_station_id": 4,
      "customer_id": 4,
      "status": 3,
      "pkg_order": "P503"
    },
    "source": {
      "version": "2.0.1.Final",
      "connector": "mysql",
      "name": "connector",
      "ts_ms": 1679710907000,
      "snapshot": "false",
      "db": "logistic",
      "sequence": null,
      "table": "packages",
      "server_id": 1,
      "gtid": null,
      "file": "binlog.000007",
      "pos": 1794,
      "row": 0,
      "thread": 11,
      "query": null
    },
    "op": "u",
    "ts_ms": 1679710907924,
    "transaction": null
  }
}

```

Hình 3.5: Dữ liệu dạng JSON được Debezium tạo ra khi có sự kiện thay đổi

Các trường dữ liệu này được mô tả như sau:

Trường dữ liệu	Mô tả
schema	Mô tả lược đồ các trường trong bảng packages. Lược đồ của sự kiện thay đổi giống nhau trong mọi sự kiện thay đổi mà connector tạo ra cho một bảng cụ thể (Do có rất nhiều trường dữ liệu trong bảng packages nên các trường này đã được thu gọn)
payload	Dữ liệu của bản ghi được tác động bằng các câu lệnh INSERT, UPDATE, DELETE, đây là trường quan trọng nhất trong message. Trong trường hợp này dữ liệu này là dữ liệu của đơn hàng
payload.before	Trường chỉ định dữ liệu của bản ghi trước khi có sự kiện xảy ra. Khi trường op có giá trị là c (có nghĩa là sự kiện sử dụng câu lệnh INSERT) thì trường payload.before sẽ có giá trị là null vì sự kiện tạo mới bản ghi sẽ không có giá trị dữ liệu trước đó. Còn khi sử dụng câu lệnh DELETE, UPDATE thì trường payload.before sẽ nhận giá trị của bản ghi trước khi được thay đổi hoặc xóa.
payload.after	Trường chỉ định dữ liệu của bản ghi sau khi có sự kiện xảy ra. Ngược lại với trường payload.before, trường này sẽ có giá trị bản ghi sau khi thay đổi hoặc tạo mới, và sẽ có giá trị null sau khi bị xóa.

payload.source	Trường mô tả metadata của cơ sở dữ liệu. Trường này chứa thông tin có thể sử dụng để so sánh sự kiện này với các sự kiện khác, liên quan đến nguồn gốc của các sự kiện, thứ tự xảy ra các sự kiện. Các metadata này bao gồm: • version: Phiên bản Debezium được sử dụng • connector: Tên của connector • file: Tên của file binlog • pos: Vị trí trong file binlog • row: Vị trí của bản ghi • snapshot: Kiểm tra xem sự kiện này có sử dụng lược đồ snapshot hay không • db, table: Tên của cơ sở dữ liệu và bảng chứa bản ghi • thread: ID của thread MySQL đã tạo ra sự kiện • server_id: ID của máy chủ MySQL • ts_ms: Timestamps¹ khi thay đổi được thực hiện trong cơ sở dữ liệu
payload.op	Chuỗi mô tả loại sự kiện, bao gồm các giá trị: c (create - tạo mới), u (update - cập nhật), d (delete - xóa), r (read - đọc). Trong trường hợp này câu lệnh UPDATE được thực thi để cập nhật trạng thái đơn hàng nên giá trị của op sẽ là u

¹Dấu thời gian mô tả thông tin về thời gian xác định khi một sự kiện nào đó xảy ra

payload.ts_ms	Đưa ra thời điểm mà connector xử lý sự kiện. Thời gian của timestamps dựa trên đồng hồ hệ thống trong JVM đang chạy tác vụ Kafka Connect. Dữ liệu ts_ms này khác với dữ liệu ts_ms trong payload.source.ts_ms là dữ liệu ts_ms này lưu trữ thời điểm khi connector xử lý sự kiện còn payload.source.ts_ms lưu thời điểm mà sự kiện được thực hiện trong cơ sở dữ liệu. Bằng cách xác định hiệu của hai giá trị này, ta có thể xác định độ trễ thời gian giữa Debezium và cơ sở dữ liệu.
---------------	---

Bảng 3.9: Mô tả các trường dữ liệu của message tạo bởi Debezium

Bằng các dữ liệu này, Kafka connect sẽ gửi đến topic tương ứng, sau đó hệ thống xử lý webhook sẽ subscribe vào topic đó và chuyển đổi dữ liệu thành các định dạng tương ứng với từng khách hàng đăng ký webhook.

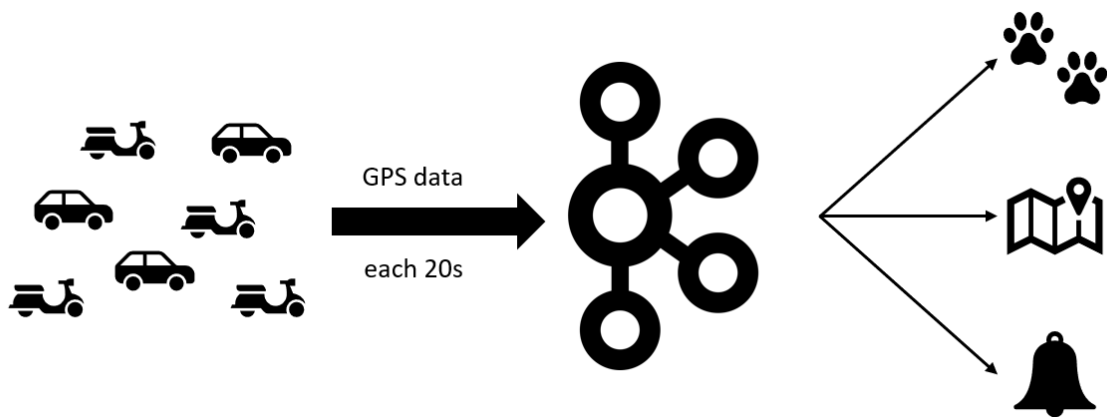
Lược đồ cơ sở dữ liệu có thể thay đổi, điều đó có nghĩa là connector phải có khả năng xác định lược đồ cơ sở dữ liệu tại thời điểm mà mỗi thao tác INSERT, UPDATE, DELETE được sử dụng. Ngoài ra, connector không thể chỉ sử dụng một lược đồ duy nhất vì connector có thể đang xử lý các dữ liệu đã cũ trước khi lược đồ của bảng bị thay đổi.

Để đảm bảo xử lý chính xác các thay đổi xảy ra sau khi thay đổi lược đồ, trong binlog không chỉ có các thay đổi đối với dữ liệu mà còn cả các câu lệnh DDL được áp dụng cho cơ sở dữ liệu. Khi connector đọc binlog và xử lý đến các câu lệnh DDL này, connector sẽ phân tích cú pháp và cập nhật định dạng lược đồ cơ sở dữ liệu hiện tại để xử lý dữ liệu cho đúng mỗi khi các câu lệnh INSERT, UPDATE, DELETE được thực thi. Khi connector khởi động lại sau khi gặp sự cố hoặc bị dừng đột ngột, connector bắt đầu đọc binlog từ một vị trí đã được lưu. Connector sẽ xây dựng lại cấu trúc bảng đã tồn tại tại thời điểm này bằng cách đọc lịch sử lược đồ cơ sở dữ liệu trong các topic và phân tích cú pháp tất cả các câu lệnh DDL cho đến vị trí của binlog nơi connector đang xử lý.

Ngoài theo dõi dữ liệu của đơn hàng, việc theo dõi tài xế cũng có thể khả thi. Ứng dụng trên điện thoại của tài xế sử dụng GPS để cập nhật vị trí hiện tại của tài

xế mỗi 20 giây. Thông tin vị trí và id của tài xế được đóng gói vào một message và gửi tới Kafka broker. Kafka broker nhận message về vị trí tài xế và lưu trữ vào topic `driver_gps` tương ứng. Sau khi message được đẩy lên Kafka, sẽ có rất nhiều consumer phía sau đọc message từ topic `driver_gps` để xử lý:

- Consumer cho việc tracking location: Đây là một consumer sử dụng dữ liệu vị trí của tài xế được gửi từ producer thông qua message broker để hiển thị vị trí hiện tại của tài xế trên bản đồ.
- Consumer cho notification: Đây là một consumer sử dụng dữ liệu từ producer để thông báo cho khách hàng về tình trạng vận chuyển hàng hóa, bao gồm việc tài xế đã xuất phát chưa, đã đến nơi chưa.
- Consumer để tracking tài xế: Đây là một consumer sử dụng dữ liệu từ producer để theo dõi và đánh giá hoạt động của tài xế, bao gồm việc tài xế đang làm việc hay nghỉ ngơi, đã làm quá giờ chưa và đưa ra thông báo cho quản lý khi cần thiết.



Hình 3.6: Luồng hoạt động tracking tài xế (người giao hàng)

3.3 Xây dựng hệ thống xử lý webhook

3.3.1 Phân tích bài toán

Khi một message trạng thái đơn hàng thay đổi được hệ thống xử lý webhook nhận từ Kafka sau đó thông tin về trạng thái đơn hàng này sẽ được gửi bằng một POST

HTTP request đến endpoint URL của khách hàng thông qua webhook. Sau khi nhận được response đã nhận thông tin thay đổi đơn hàng thành công, hệ thống xử lý webhook sẽ lấy message tiếp theo để xử lý. Cách xử lý này là cách xử lý tuần tự, hệ thống sẽ phải đợi response trả về mới được xử lý tiếp đến message tiếp theo. Điều này có thể gây chậm trễ trong việc cập nhật trạng thái đơn hàng cho các bên đối tác, có thể gây ra hậu quả nghiêm trọng.

3.3.2 Giải pháp thực hiện

Để giải quyết vấn đề này, khóa luận đưa ra giải pháp là sử dụng xử lý đồng thời (concurrency), cơ sở lý thuyết của concurrency đã được đề cập ở mục 2.4. Concurrency là khả năng xử lý nhiều tác vụ một cách đồng thời trong một khoảng thời gian. Kỹ thuật lập trình giúp có thể triển khai concurrency là kỹ thuật lập trình bất đồng bộ (asynchronous programming). Lập trình bất đồng bộ là một phương pháp lập trình cho phép các tác vụ được thực thi mà không phải chờ đợi cho tác vụ trước đó hoàn thành. Kỹ thuật này cho phép tối ưu hóa sử dụng tài nguyên máy tính, đặc biệt là trong các ứng dụng web hoặc các hệ thống xử lý dữ liệu lớn, nơi việc thực hiện các tác vụ đồng thời rất quan trọng để đảm bảo hiệu suất cao. Trong lập trình bất đồng bộ, thay vì chờ đợi kết quả trả về từ một tác vụ, CPU sẽ tiếp tục thực hiện các tác vụ khác, sau đó quay lại tác vụ trước đó để lấy kết quả và xử lý tiếp.

Ngôn ngữ lập trình được sử dụng trong toàn khóa luận là **Python**¹. Python là một ngôn ngữ lập trình bậc cao, hướng đối tượng, được thiết kế cho nhiều mục đích lập trình khác nhau được sử dụng rộng rãi trong các lĩnh vực web, phát triển phần mềm, máy học và khoa học dữ liệu. Ngoài ra, Python có thể chạy trên nhiều nền tảng khác nhau. Python có cấu trúc cú pháp đơn giản và trực quan giúp dễ phát triển các đoạn code phức tạp theo một cách đơn giản, điều này giúp Python là một sự lựa chọn phù hợp cho hệ thống theo dõi và cập nhật thông tin đơn hàng đến đối tác vì hệ thống chuyên về xử lý các **background job**². Một lí do để khóa luận sử dụng Python làm ngôn ngữ lập trình chính vì Python có khả năng xử lý bất đồng bộ rất hiệu quả bằng các built-in module (thư viện được phát triển bởi chính ngôn ngữ) như threading, asyncio, multiprocessing,...cho phép xử lý concurrency

¹<https://www.python.org/>

²một nơi xử lý các luồng, không can thiệp vào luồng chính của trang web

một cách hiệu quả. Concurrency giúp gom một số lượng request rồi xử lý đồng thời chúng cùng một khoảng thời gian, như vậy sẽ giúp giảm đáng kể thời gian chờ I/O. Để đánh giá độ hiệu quả của từng module áp dụng cho bài toán xử lý nhiều request, dưới đây khóa luận sẽ thực nghiệm trên từng module nhằm đưa ra so sánh trực quan nhất để có thể lựa chọn ra module phù hợp cho bài toán.

a) **threading module**

threading là một built-in module trong Python cho phép tạo và quản lý thread trong chương trình Python. Thread là một cách để chạy nhiều tác vụ của chương trình trong một thời điểm, có thể hữu ích để cải thiện hiệu suất của chương trình I/O bound. Bằng cách sử dụng module threading, có thể tận dụng tối ưu CPU khi đợi I/O. Ngoài ra, vì tất cả các thread chia sẻ cùng một bộ nhớ, để thực hiện nhiều tác vụ trong Python bằng cách sử dụng thread, ta sẽ phải cẩn thận và sử dụng lock khi cần thiết. Lock và unlock cung cấp một cách để ngăn nhiều thread truy cập vào cùng một dữ liệu cùng một lúc, điều này có thể gây ra **tình trạng tương tranh (race condition)**.

Race condition là sự cố xảy ra khi nhiều thread cùng truy cập và cùng lúc muốn thay đổi dữ liệu. Trong race condition, thuật toán chuyển đổi việc thực thi giữa các thread có thể xảy ra bất cứ lúc nào, nên không thể biết được thứ tự của các thread truy cập và thay đổi dữ liệu đó sẽ dẫn đến giá trị của dữ liệu sẽ không như mong muốn. Để tránh xảy ra race condition, đảm bảo rằng chỉ một thread có thể truy cập dữ liệu được chia sẻ tại một thời điểm bằng cách sử dụng khóa (lock), cờ hiệu (semaphore).

Tiếp tục tăng số lượng thread bắt đầu từ 1 và đến một giới hạn mà thời gian để tạo và giải phóng các thread có thể làm giảm hiệu suất. Ở trường hợp này, số lượng thread phù hợp để triển khai là 5.

Nhận xét: Triển khai threading yêu cầu không quá phức tạp, làm cho nó trở thành một lựa chọn tốt cho việc xử lý concurrency, giúp thực hiện nhanh hơn cho các chương trình I/O bound. Vấn đề khi sử dụng threading là phải xem xét dữ liệu nào được chia sẻ giữa các thread. Các thread có thể tương tác vào dữ liệu của nhau, vì thế có thể gây ra race condition thường dẫn đến các lỗi khó để debug.

b) **asyncio module**

asyncio là một built-in module trong Python là một module giúp xử lý concurrency bằng cách sử dụng *coroutines*, *event loops* và *Tasks*. Module asyncio được giới thiệu trong Python 3.4 và trở nên phổ biến để xây dựng ứng dụng web, network server và các hệ thống concurrency. Module asyncio là một kiểu thiết kế single-threaded và single-process, nó sử dụng **cooperative multitasking**.

Cooperative multitasking là một kỹ thuật trong đó các tác vụ (tasks) tự động quyết định khi nào chuyển quyền điều khiển (control) cho các tác vụ khác, chứ không phải bị ngắt quyền bởi hệ điều hành. Điều này khác với preemptive multitasking, trong đó hệ điều hành quyết định khi nào chuyển quyền điều khiển giữa các tác vụ bằng cách sử dụng context switching. Với cooperative multitasking, các tác vụ phải tự giải phóng điều khiển của mình để cho các tác vụ khác có thể thực hiện, do đó nó cho phép các tác vụ làm việc một cách đồng thời nhưng không tốn nhiều tài nguyên của hệ thống so với preemptive multitasking. Module này bao gồm các APIs sau để có thể viết code bất đồng bộ một cách dễ dàng và trực quan hơn:

- **Coroutines:** Là các hàm có khả năng tạm dừng và tiếp tục tại context nó được tạm dừng, cho phép thực hiện cooperative multitasking (multitasking liên tục) giữa nhiều tác vụ. Các coroutine tương tự như thread, nhưng coroutine yêu cầu ít bộ nhớ hơn và không cần sử dụng khóa hay cờ hiệu để xử lý race condition.
- **Event loops:** Event loop là một thành phần trung tâm của chương trình bất đồng bộ, quản lý thực thi các tác vụ và callback. Event loop liên tục theo dõi một hàng đợi các tác vụ và thực thi chúng một cách tuần tự theo mô hình cooperative multitasking, trong đó các tác vụ nhường quyền điều khiển cho event loop khi chờ I/O. Điều này cho phép các tác vụ chạy đồng thời trên một thread giúp tiết kiệm tài nguyên hơn.
- cú pháp **async** và **await**: **async** giúp định nghĩa một coroutine, **await** là một cú pháp giúp tạm dừng (block) code để đợi lấy kết quả từ coroutine đó.

Nhận xét: Asyncio cho thấy có thể xử lý một lượng lớn tác vụ liên quan đến network connection một cách hiệu quả, cho phép chạy nhiều coroutine trên một thread giúp tiết kiệm bộ nhớ và việc kiểm soát coroutine trên một thread

dễ dàng hơn với việc kiểm soát hàng chục thread. Điểm hạn chế khi sử dụng asyncio là không tương thích với một vài thư viện và framework và chỉ được hỗ trợ từ phiên bản Python 3.4¹, điều này có thể gây khó khăn cho việc phát triển hệ thống cũ.

c) multiprocessing module

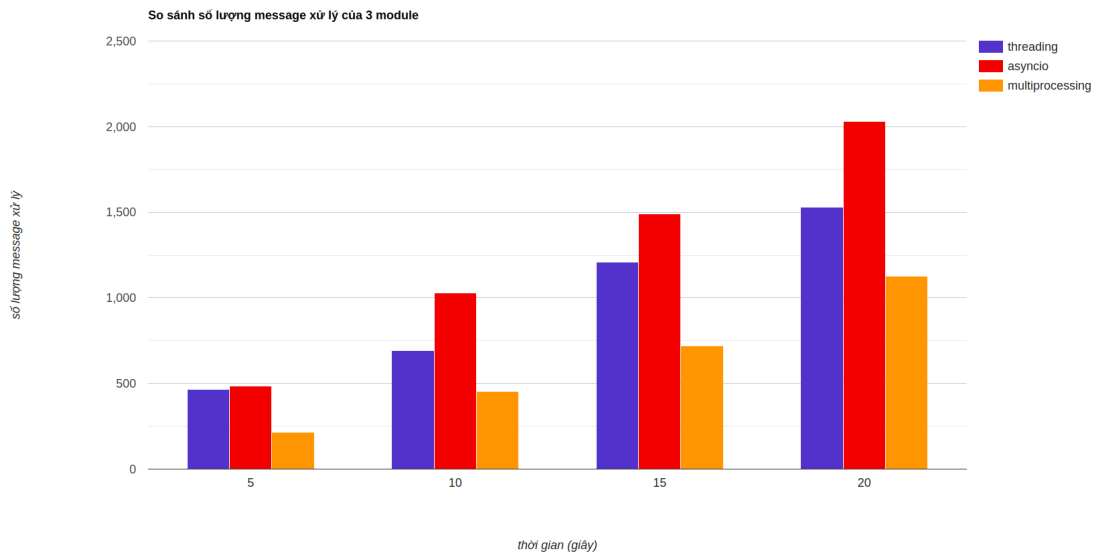
`multiprocessing` module cho phép các chương trình Python tận dụng nhiều CPU để xử lý song song, điều này có thể dẫn đến những cải thiện đáng kể về hiệu suất cho các tác vụ CPU bound. Với multiprocessing, một chương trình Python có thể tạo nhiều process để xử lý các tác vụ khác nhau, mỗi process có không gian bộ nhớ riêng vì vậy chúng không chia sẻ dữ liệu như thread. Tuy nhiên để có thể giao tiếp với nhau gián tiếp qua **IPC (Inter-process communication)** như Shared memory (bộ nhớ dùng chung), Queue (hàng đợi). Multiprocessing rất hữu dụng cho các tác vụ sử dụng nhiều CPU.

Nhận xét: Multiprocessing cho phép thực hiện parallelism trên máy tính có nhiều CPU, giúp các process có thể chạy song song với nhau. Mỗi process trong multiprocessing được cô lập với các process khác, vì vậy nếu một process gặp sự cố, nó sẽ không ảnh hưởng đến các process khác. Bởi vì multiprocessing phù hợp với các tác vụ liên quan đến CPU bound hơn là xử lý I/O bound nên khi sử dụng multiprocessing để xử lý nhiều request đồng thời không đưa ra kết quả tốt. Việc tạo và quản lý nhiều process có thể ảnh hưởng đến hiệu suất vì process yêu cầu nhiều bộ nhớ [8]. Ngoài ra, việc debug khi sử dụng multiprocessing khó hơn nhiều so với việc debug trên chương trình có 1 process và nhiều thread vì phải quản lý nhiều process và thread cùng lúc hơn.

So sánh và kết luận

Triển khai áp dụng 3 module trên vào hệ thống và thử nghiệm trong khoảng thời gian là 20 giây, số lượng message hệ thống xử lý được như sau:

¹<https://docs.python.org/3.4/library/asyncio.html>

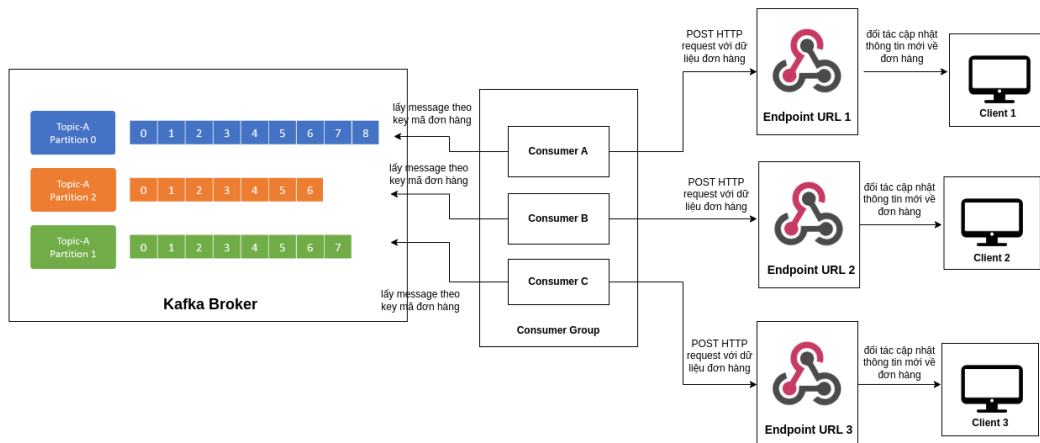


Hình 3.7: So sánh thử nghiệm 3 module concurrency trong bài toán xử lý nhiều request

Từ biểu đồ trên, nhận thấy module asyncio xử lý được nhiều request nhất trong khoảng thời gian 20 giây với khoảng hơn 2000 requests, module multiprocessing dường như không phù hợp với yêu cầu xử lý các tác vụ liên quan đến I/O bound. Module threading cũng cho kết quả khá tốt nhưng so với asyncio thì threading chậm hơn trong việc xử lý I/O bound vì threading sử dụng thread của hệ điều hành vậy nên các thread này được quản lý bởi OS và việc sử dụng nhiều thread sẽ gây tốn tài nguyên khi phải chuyển giữa các luồng với nhau. Hơn nữa, sử dụng thread có thể sẽ gây ra hiện tượng race condition tạo ra những bug không đáng có nếu không được kiểm soát tốt.

Ngoài ra, việc sử dụng asyncio module trong Python cũng phù hợp với bài toán xử lý nhiều request (xử lý I/O bound) hơn là việc sử dụng module threading hay multiprocessing [9].

Sau khi đã tìm được phương pháp xử lý nhiều request đồng thời, hệ thống xử lý webhook được xây dựng theo mô hình sau:



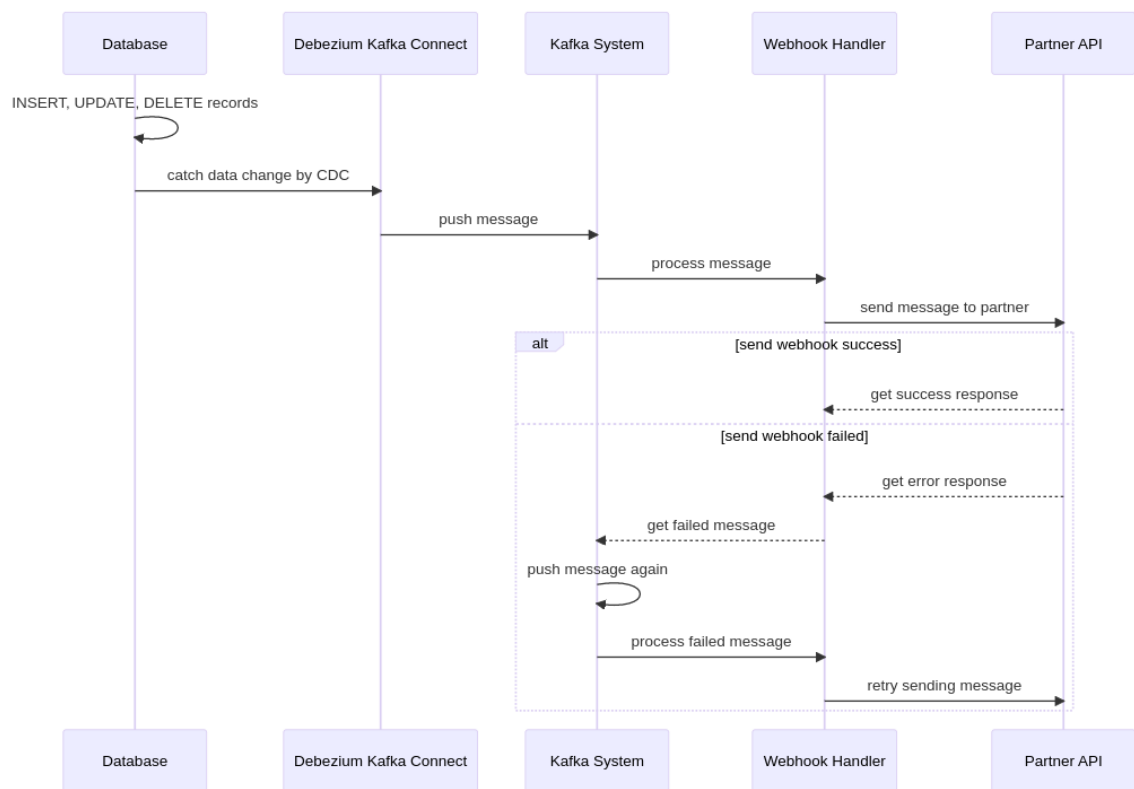
Hình 3.8: Mô hình hệ thống xử lý webhook

Thông qua mô hình ta có thể Kafka Broker là nơi lưu trữ các dữ liệu được thay đổi bởi hệ thống nắm bắt sự thay đổi dữ liệu từ cơ sở dữ liệu. Các consumer trong consumer group sẽ lấy các message này theo message key là mã đơn hàng để đảm bảo thứ tự đơn hàng, các đơn hàng có cùng mã đơn hàng sẽ được đưa vào partition theo thứ tự thay đổi khi lấy ra cũng được lấy theo thứ tự. Sau khi lấy được message, các hàm trong consumer này sẽ định dạng dữ liệu message, các message này có dạng JSON được decode để chuyển về dạng dữ liệu trong Python, lọc ra các trường cần thiết phù hợp với từng đối tác, cuối cùng các dữ liệu này sẽ được gửi đến một API được bên đối tác cung cấp trước đó.

Trong các hàm xử lý dữ liệu tại consumer sử dụng asyncio module để giải quyết bài toán xử lý nhiều request. Sau khi đã gom một lượng request dữ liệu đơn hàng, lúc này được gọi là các coroutine, các coroutine này có thể trao lại quyền sử dụng CPU cho event loop để lấy xử lý các coroutine khác trong khi chờ I/O và xử lý các coroutine này luân phiên nhau [8]. Sau khi đã nhận được thông báo lấy dữ liệu thành công từ phía đối tác (hay chờ xử lý I/O xong) thì event loop sẽ cho coroutine bị tạm dừng tiếp tục chạy từ điểm dừng của nó.

Hệ thống của đối tác có thể sẽ gặp lỗi hoặc phải ngừng hoạt động để nâng cấp. Trong khoảng thời gian đó, dữ liệu được gửi đến hệ thống đối tác có thể không

được xử lý thành công và trả kết quả nhận dữ liệu không thành công. Để xử lý vấn đề này, các dữ liệu này sẽ được đẩy lại vào Kafka để gửi lại. Bằng cách này, cho dù hệ thống đối tác gặp lỗi hay ngừng hoạt động thì sau khi hoạt động lại vẫn có thể nhận được dữ liệu trong khoảng thời gian gặp lỗi (downtime). Luồng tổng quan xử lý message và gửi lại message khi hệ thống đối tác gặp lỗi được mô tả qua biểu đồ tuần tự sau:



Hình 3.9: Biểu đồ tuần tự mô tả luồng hoạt động xử lý message và gửi lại message khi hệ thống đối tác gặp lỗi

- 1. Hệ thống nắm bắt sự thay đổi dữ liệu sẽ bắt sự thay đổi từ cơ sở dữ liệu và đẩy lên Kafka.
- 2. Hệ thống xử lý webhook sẽ consume message từ topic trong Kafka, tiến hành xử lý, định dạng message sau đó gửi đến cho đối tác. Hệ thống sử dụng response HTTP Status code để xác định là có gửi cập nhật cho đối tác thành

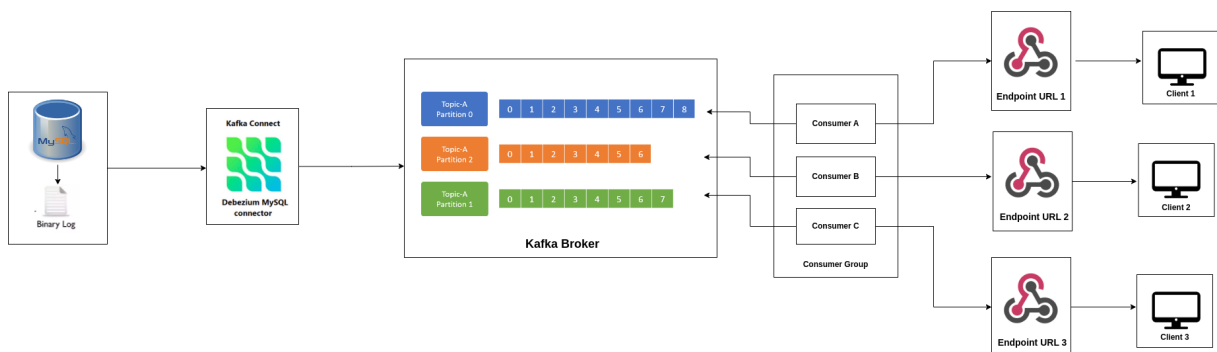
công hay không.

- 2.1. Nếu đối tác nhận được message thành công sẽ trả về một response thành công có trạng thái là 200. Hệ thống lúc này sẽ ghi nhận đã cập nhật thành công cho đối tác
- 2.2. Nếu hệ thống của đối tác gặp lỗi và chưa nhận được message thì sẽ trả về một response lỗi có trạng thái khác 200 hoặc chưa nhận được response, lúc này message sẽ được gửi lại vào một topic khác sau đó hệ thống xử lý webhook sẽ đọc lại message này và gửi đến cho đối tác.

3.4 Giải pháp ưu tiên message

3.4.1 Phân tích bài toán

Sau khi đã thiết kế xong hai hệ thống nắm bắt sự thay đổi dữ liệu và hệ thống xử lý webhook, ta kết nối hai hệ thống này bằng Message Broker Kafka.



Hình 3.10: Mô hình tổng quan hệ thống

Việc sử dụng Message Broker giúp đảm bảo xử lý message một cách nhanh chóng, tách rời hai hệ thống, tránh bị mất message, có khả năng mở rộng cao.

Tuy nhiên, khi một message được gửi đến webhook có thời gian response quá lâu sẽ gây ảnh hưởng đến các message phía sau. Một đối tác có hệ thống xử lý mạnh có thể trả về response rất nhanh sau khi nhận được thông tin trạng thái đơn hàng từ webhook, ngược lại một hệ thống xử lý chậm sẽ trả về response chậm. Từ đó, việc hệ thống phải đợi các response trả về sẽ phụ thuộc vào tốc độ trả về response

nhanh hay chậm của bên đối tác, vì chỉ khi nhận được response thì đơn hàng đó mới được coi là đã được cập nhật trạng thái cho bên đối tác.

Vì thế giải pháp dùng để tối ưu việc gửi message cho bên đối tác là những đối tác trả về response nhanh sẽ được ưu tiên gửi thông tin đơn hàng trước những đối tác trả về response chậm. Tuy nhiên, trong Kafka lại không hỗ trợ xử lý message theo độ ưu tiên, bởi vì Kafka được thiết kế là một hệ thống phân tán truyền tải message theo thời gian thực và đảm bảo tin cậy. Việc triển khai tính năng ưu tiên message sẽ làm phức tạp và làm giảm hiệu suất của hệ thống. Ngoài ra, các message được đưa vào topic theo dạng append-only, nghĩa là các message không thể thay đổi thứ tự hay dữ liệu trong message.

3.4.2 Giải pháp thực hiện

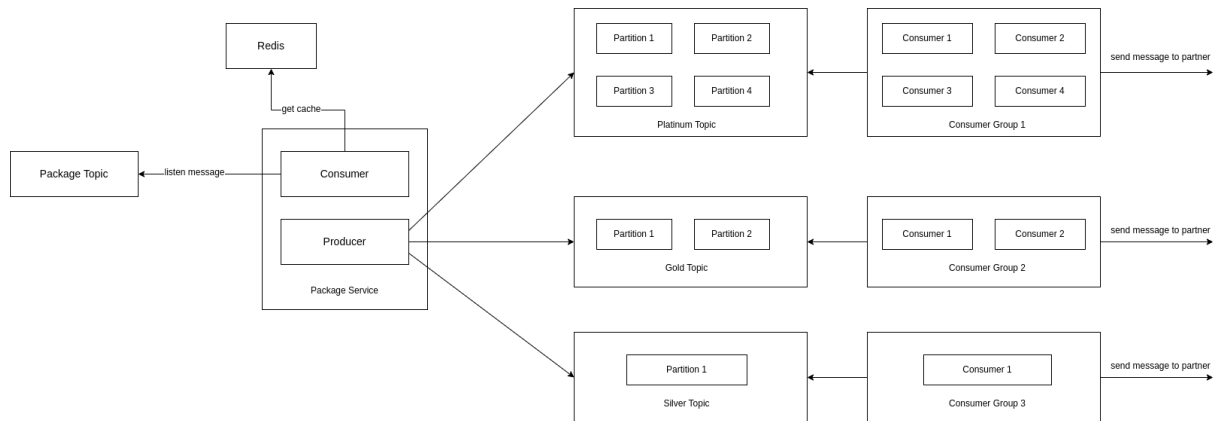
Một cách giải quyết đơn giản cho vấn đề này chính là gom tất cả các message lại trước sau đó sắp xếp chúng theo một thứ tự với message có độ ưu tiên cao hơn thì đứng trước. Nhưng giải pháp này có một vấn đề, các consumer phải lấy message từ những partition khác nhau sau đó mới có thể sắp xếp chúng vì message không chỉ nằm ở một nơi mà nó nằm rất nhiều nơi trong nhiều partition khác nhau. Khi gom nhiều message rồi mới xử lý (batching), điều này làm mất đi tính chất xử lý liên tục luồng dữ liệu. Hơn nữa, việc lấy message từ những partition khác nhau như vậy không thể thực hiện được vì trong các hệ thống lớn sẽ nhiều consumer để xử lý vậy nên các consumer trong cùng consumer group sẽ chỉ có thể xử lý các message trong partition duy nhất, không thể thực hiện đối với các message từ các partition khác.

Thay vì phải gom lại rồi sắp xếp các message, ta có thể nhóm các message vào từng topic riêng, với mỗi topic có một độ ưu tiên khác nhau. Khi đó ta có thể giải quyết được vấn đề phải gom các message lại và sắp xếp vì giờ đây mỗi message nằm trong một topic riêng có độ ưu tiên khác nhau. Ví dụ message A, message B có độ ưu tiên cao như nhau nằm trong topic 1, message C, message D có độ ưu tiên thấp hơn nằm trong topic 2.

Triển khai giải pháp ưu tiên message

Các topic có thể có số lượng partition khác nhau. Topic có độ ưu tiên cao có thể có số lượng partition nhiều hơn do đó có thể chứa nhiều message hơn. Ngoài ra,

các topic có độ ưu tiên cao có thể có nhiều consumer, tương đương với số lượng partition để có thể tăng thông lượng. Thông lượng mong muốn đặt ra là 200-300 req/s. Vì vậy, khi kết hợp hai phương pháp này, ta có thể giải quyết vấn đề ưu tiên message một cách hợp lý bằng cách thay vì lưu trữ message trong một topic, ta chia các topic đó ra thành nhiều các topic có độ ưu tiên khác nhau. Ví dụ như hình 3.11, Platinum topic có độ ưu tiên cao nhất nên sẽ chia ra thành nhiều partition và có nhiều consumer xử lý hơn Gold topic.



Hình 3.11: Phân chia các topic theo độ ưu tiên khác nhau

Một câu hỏi đặt ra là làm thế nào để biết message có độ ưu tiên nào để producer có thể gửi đến topic phù hợp. Có thể biết được điều này bằng cách lưu lại khoảng thời gian sau khi đơn hàng được gửi đến đối tác và nhận được response trả về, ta có thể tính khoảng thời gian này và giá trị này gọi là *metric time*. Ngoài ra, các đơn hàng sẽ chứa thông tin về endpoint URL (`webhook_url`) của đối tác. Ta có thể lưu giá trị endpoint URL cùng với *metric time* vào trong **Redis**¹ theo dạng từ điển với key là từng đối tác và value là *metric time*, `webhook_url`. Việc sử dụng caching như vậy để có thể giúp truy vấn nhanh hơn bởi vì dữ liệu `webhook_url` và *metric time* là dữ liệu ít khi thay đổi.

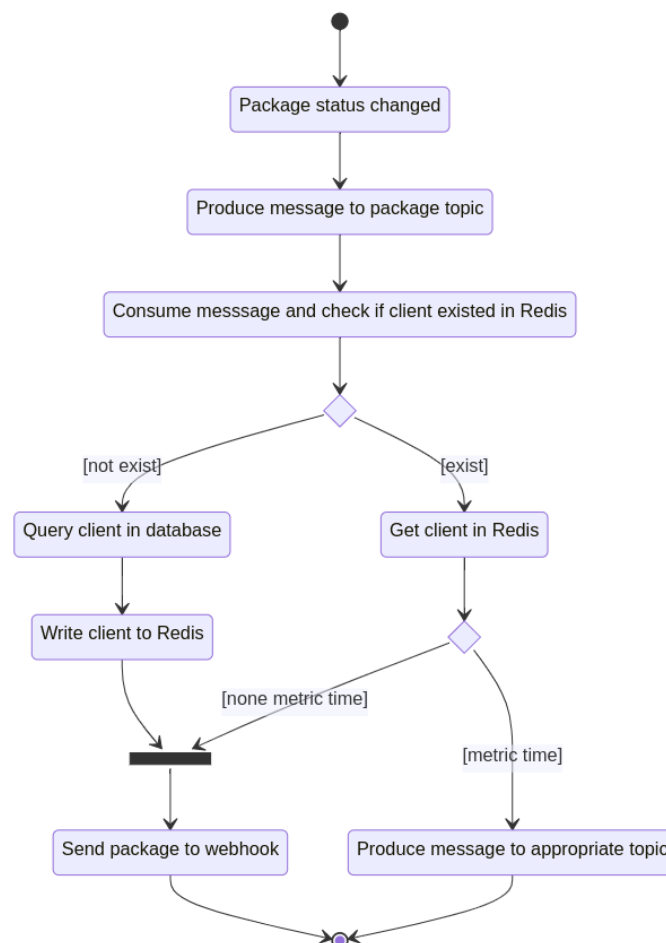
Nói sơ qua về Redis thì Redis là một hệ thống cơ sở dữ liệu key-value mã nguồn mở được sử dụng như một database, cache hoặc message broker. Redis cung cấp khả năng lưu trữ dữ liệu theo dạng key-value, cho phép các dữ liệu phổ biến được lưu trữ và truy xuất nhanh chóng qua RAM. Có rất nhiều mô hình để áp dụng Redis làm cache, khóa luận lựa chọn mô hình **Cache Aside** [10] vì mô hình này

¹<https://redis.io/>

giúp giảm thiểu tải cho cơ sở dữ liệu và cải thiện thời gian phản hồi của ứng dụng cũng như dễ triển khai.

- Ứng dụng sẽ truy xuất dữ liệu bằng cách truy vấn trong cache trước.
- Nếu dữ liệu không tồn tại trong cache, ứng dụng sẽ truy xuất nó từ cơ sở dữ liệu chính.
- Sau đó ghi dữ liệu vào cache để sử dụng cho các lần truy cập tiếp theo.

Luồng hoạt động của hệ thống xử lý webhook khi triển khai giải pháp ưu tiên message được biểu diễn qua biểu đồ hoạt động sau:



Hình 3.12: Biểu đồ hoạt động của giải pháp ưu tiên message

- Đầu tiên, khi trạng thái của đơn hàng được thay đổi, Debezium sẽ gửi message này đến package topic. Package topic có thể coi là topic dùng để chứa message từ hệ thống nắm bắt thay đổi dữ liệu.

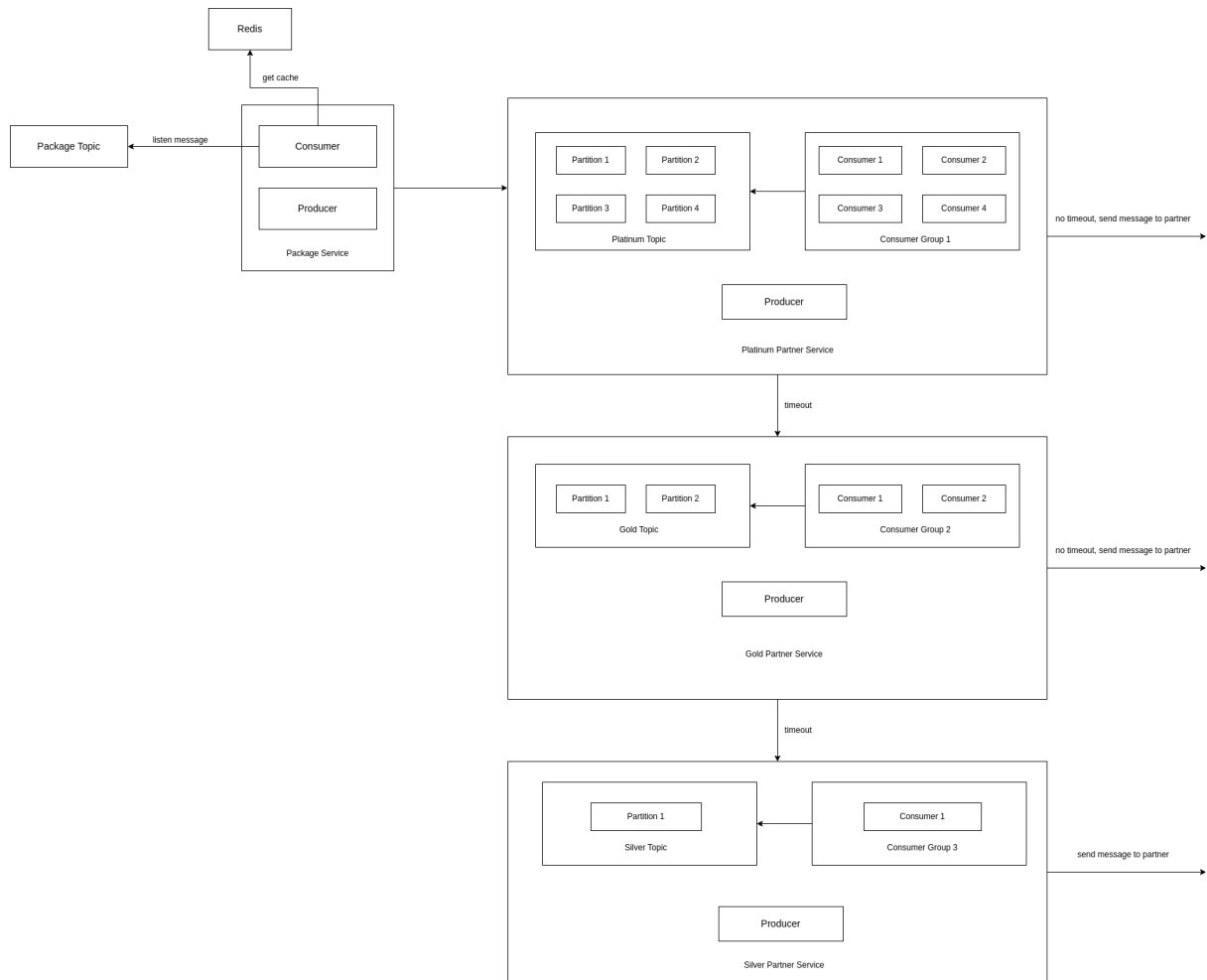
- Các consumer sẽ đọc dữ liệu từ package topic và kiểm tra xem dữ liệu về đối tác của đơn hàng này đã tồn tại trong Redis hay chưa.
- Nếu chưa tồn tại, hệ thống sẽ lấy thông tin webhook_url trong database và ghi vào Redis để sử dụng cho các lần tiếp theo.
- Nếu đã tồn tại, hệ thống sẽ đọc dữ liệu này trong Redis.
- Đồng thời, nếu trong dữ liệu được lưu ở Redis đã tồn tại metric time thì Producer sẽ gửi message này đến topic phù hợp để các consumer đọc message từ topic đó tiếp tục xử lý. Còn chưa có metric time thì đơn hàng này sẽ được gửi đến webhook để tính toán metric time và lưu lại.

Bằng cách này, sau khi được lưu lại metric time, các lần tiếp theo các đơn hàng của đối tác này sẽ nhanh chóng được gửi đến topic phù hợp với độ ưu tiên. Tuy nhiên, giải pháp này vẫn chưa tối ưu vì thời gian metric time không phải lúc nào cũng bất biến. Ví dụ, một hệ thống của đối tác bị quá tải trong thời gian cao điểm nên sẽ trả về response lâu hơn. Nhưng khi hệ thống có lượng request vừa phải thì sẽ trả response về nhanh hơn. Thời gian này phụ thuộc vào từng thời điểm nên khi cố định metric time chỉ sau một lần duy nhất tương tác với hệ thống đối tác thì vẫn chưa hoàn toàn phù hợp.

Tối ưu giải pháp

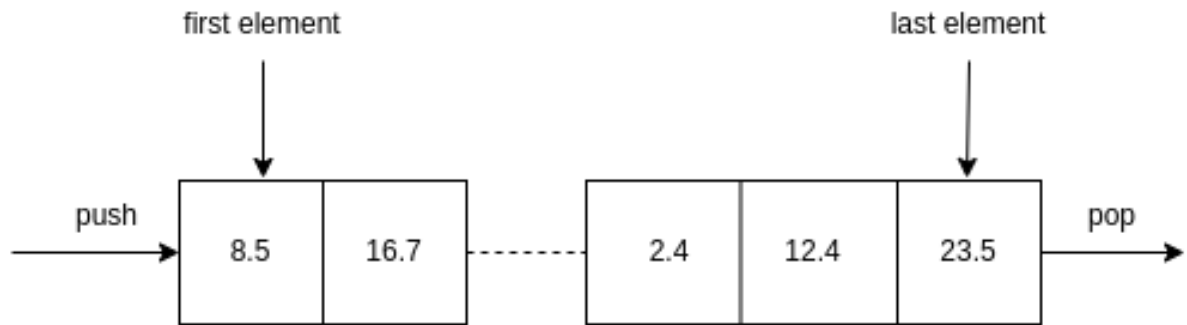
Khóa luận đã đưa ra một giải pháp tối ưu để có thể giúp giải quyết vấn đề trên bằng cách:

- Các đơn hàng của đối tác chưa được lưu vào Redis sẽ được đưa vào hàng đợi ưu tiên cao nhất (topic có độ ưu tiên cao nhất) và request đến webhook trong một khoảng thời gian nhất định. Nếu request bị timeout, thời gian metric time được lưu lại và nó sẽ được đẩy xuống hàng đợi ưu tiên thấp hơn để tiếp tục thực thi, cứ như vậy cho đến khi nó bị đẩy xuống hàng đợi cuối cùng và được gửi đi. Còn nếu không bị timeout thì đơn hàng đó sẽ được tiếp tục xử lý.
- Thay vì lưu trữ metric time một lần duy nhất, ta sẽ lưu trữ metric time của từng lần request để tính ra thời gian trung bình xử lý (average metric time) từ đó các đối tác ở hàng đợi ưu tiên thấp hơn sau một khoảng thời gian hệ thống ổn định trở lại có thể được xử lý ở hàng đợi ưu tiên cao hơn và ngược lại.



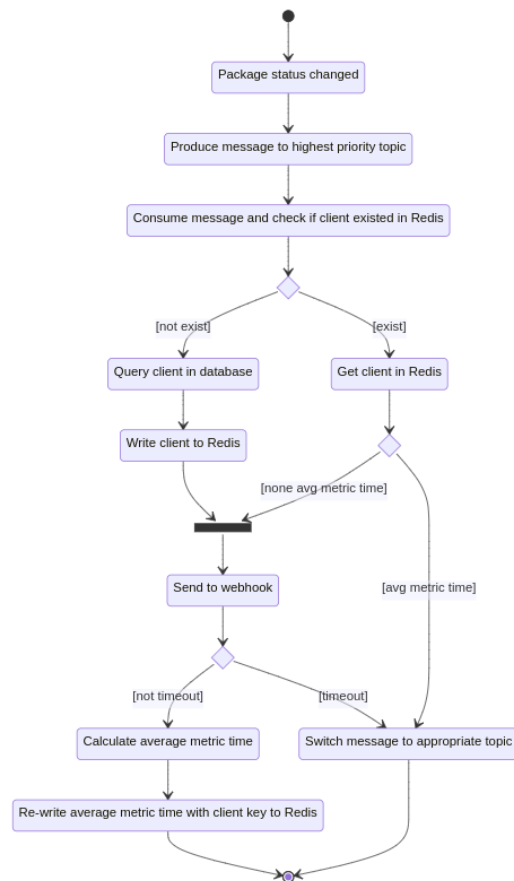
Hình 3.13: Mô hình tối ưu giải pháp ưu tiên message

Vì phải lưu trữ một số lượng metric time vào Redis để tính ra thời gian trung bình xử lý (average metric time) nên khi quá nhiều metric time được lưu sẽ gây quá tải RAM từ đó có thể khiến hệ thống chạy chậm đi. Vậy nên ta chỉ lấy một số lượng metric time nhất định khoảng 100 phần tử để có thể thống kê được thời gian trung bình. Ngoài ra, mỗi khi cập nhật metric time rồi tính toán lại tổng sau đó chia trung bình cũng gây mất thời gian đáng kể. Nên ta có thể đơn giản bằng cách pop một phần tử ở cuối queue trong Redis và push phần tử mới vào rồi chia cho tổng số lượng cố định khoảng 100 phần tử. Minh hoạt của giải pháp tối ưu xử lý tính toán Redis ở hình dưới đây:



Hình 3.14: Tối ưu xử lý tính toán nhiều lần trên Redis

Khi đó luồng hoạt động của giải pháp ưu tiên message được thay đổi như sau:



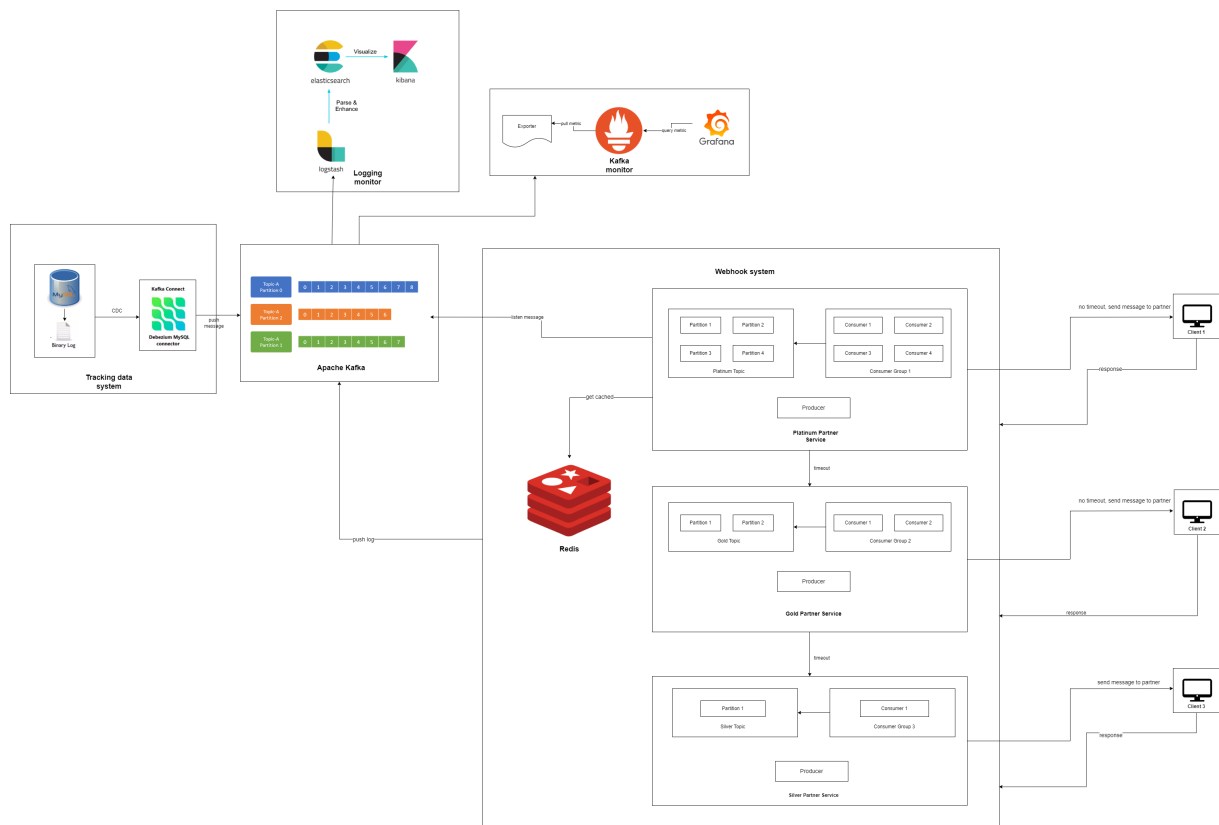
Hình 3.15: Biểu đồ hoạt động khi tối ưu giải pháp ưu tiên message

- Đầu tiên, khi trạng thái của đơn hàng được thay đổi, Debezium sẽ gửi message này đến topic có độ ưu tiên cao nhất.
- Các consumer sẽ đọc dữ liệu từ topic này và kiểm tra xem dữ liệu về đối tác

của đơn hàng này đã tồn tại trong Redis hay chưa.

- Nếu chưa tồn tại, hệ thống sẽ lấy thông tin `webhook_url` trong database và ghi vào Redis để sử dụng cho các lần tiếp theo.
- Nếu đã tồn tại, hệ thống sẽ đọc dữ liệu này trong Redis.
- Đồng thời, nếu trong dữ liệu được lưu ở Redis đã tồn tại `avg_metric_time` thì Producer sẽ gửi message này đến topic phù hợp để các consumer đọc message từ topic đó tiếp tục xử lý. Còn chưa có `avg_metric_time` thì đơn hàng này sẽ được gửi đến webhook để tính toán metric time và push vào queue chứa các metric time.
- Nếu request bị timeout, thời gian metric time được lưu lại và nó sẽ được đẩy xuống hàng đợi ưu tiên thấp hơn để tiếp tục thực thi, cứ như vậy cho đến khi nó bị đẩy xuống hàng đợi cuối cùng và được gửi đi. Còn nếu không bị timeout thì request đó sẽ được tiếp tục xử lý. Bằng cách này các topic được ưu tiên hơn sẽ không bị chặn bởi các request của các đối tác xử lý chậm nên lượng thông lượng rất là ổn định.

Mô hình tổng quan của hệ thống sau khi đã triển khai các thuật toán tối ưu xử lý request để tăng thông lượng:



Hình 3.16: Mô hình tổng quan của hệ thống

Chương 4: Cài đặt và kết quả thực nghiệm

Trong chương này, khóa luận sẽ trình bày về việc cài đặt, cấu hình và đưa ra các quá trình thực nghiệm. Cách cài đặt từng phần của hệ thống sẽ được mô tả chi tiết trong chương này. Phần thực nghiệm sẽ đi sâu vào việc áp dụng các công cụ để giám sát, phân tích và quản lý hệ thống.

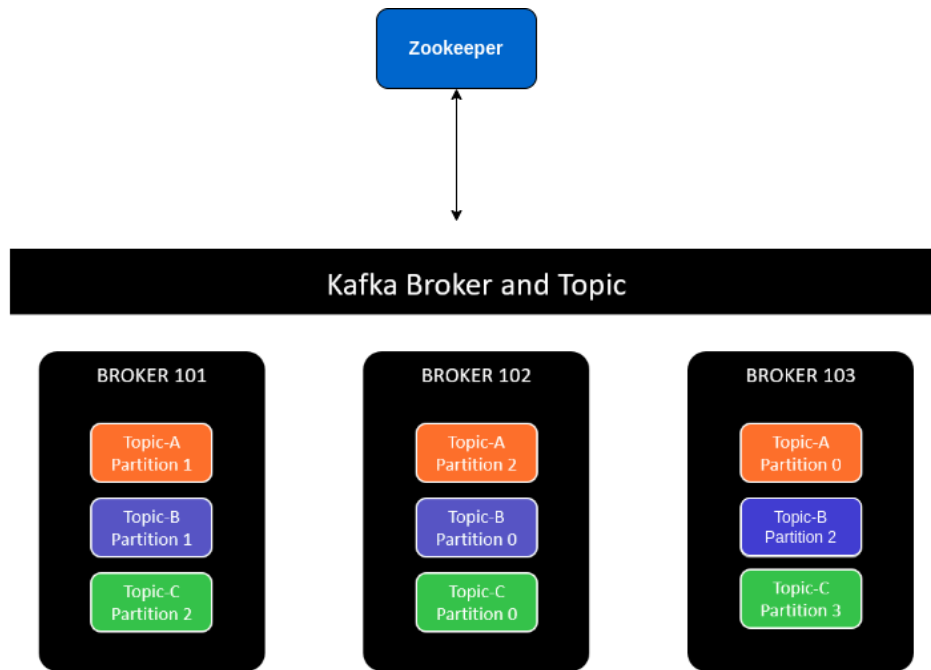
4.1 Cài đặt hệ thống

4.1.1 Triển khai môi trường và cài đặt hệ thống bằng Docker

Sau khi tìm hiểu về Docker, ta sẽ tiến hành cài đặt hệ thống theo các bước sau:

- *Bước 1*, cài đặt Docker và Docker compose theo đường dẫn sau: <https://docs.docker.com/engine/install/> theo từng hệ điều hành khác nhau. Khóa luận lựa chọn cài đặt trên một con máy chủ ảo VPS (Virtual Private Server) có hệ điều hành Ubuntu 20.04.
- *Bước 2*: tiến hành tải mã nguồn tại https://github.com/q-thang/tracking_package_system.
- *Bước 3*: Tạo file `.env` từ file biến môi trường mẫu `.env.example`.
- *Bước 4*: Truy cập vào thư mục chứa source code và chạy câu lệnh `sudo docker-compose up` để tiến hành cài đặt các container service bằng Docker compose. Để dừng các service, ta sử dụng câu lệnh `sudo docker-compose down`.

Sau khi đã cài đặt xong hệ thống, ta kết nối Debezium connector với hệ quản trị cơ sở dữ liệu MySQL. Apache Kafka được cài theo kiến trúc một Zookeeper trên cụm Cluster gồm 3 broker, có `replication.factor = 3` để phân tán 3 partition trong một topic đều trên 3 broker.



Hình 4.1: Kiến trúc Kafka được triển khai trong hệ thống

Cuối cùng, bằng cách sử dụng Supervisor¹ ta có thể giúp cho các tiến trình luôn đảm bảo tính sẵn sàng mà không cần phải tương tác với màn hình dòng lệnh. Supervisor có khả năng khởi động lại tiến trình nếu nó bị dừng hoặc gặp lỗi, giám sát hoạt động của tiến trình, và ghi lại các sự kiện liên quan đến tiến trình. Supervisor sẽ thực hiện giám sát các Consumer được khởi tạo để lắng nghe message từ các topic để tiến hành xử lý. Có nhiều Consumer trong cùng một consumer group xử lý các message trong cùng topic, với mỗi consumer tương đương với một tiến trình.

4.2 Xác định thông lượng của hệ thống

Kịch bản để xác định thông lượng được chạy trên máy tính có thông số phần cứng như sau:

¹Một phần mềm quản lý các tiến trình trên Linux



Size	
Size	Standard D4s v3
vCPUs	4
RAM	16 GiB

Hình 4.2: Thông số phần cứng máy tính (CPU) sử dụng để xác định thông lượng

Kịch bản sẽ được tiến hành như sau:

- Đầu tiên, sử dụng một đoạn script để thay đổi trạng thái đơn hàng của bảng packages, mục đích là tạo ra nhiều dữ liệu đơn hàng thay đổi để test.
- Sau đó, hệ thống nắm bắt sự thay đổi dữ liệu sẽ bắt được những thay đổi này.
- Các dữ liệu được thay đổi sẽ được đẩy vào Kafka topic.
- Hệ thống xử lý webhook sẽ consume dữ liệu từ topic và tiến hành xử lý dữ liệu và đưa đến đối tác phù hợp.
- Nếu response trả về có trạng thái là 200, log của response đó sẽ được lưu lại vào Elasticsearch thông qua Logstash.
- Tiến hành trực quan hóa dữ liệu trên Kibana để xem thống kê response thành công.

4.2.1 Phân tích và thống kê response log với ELK stack

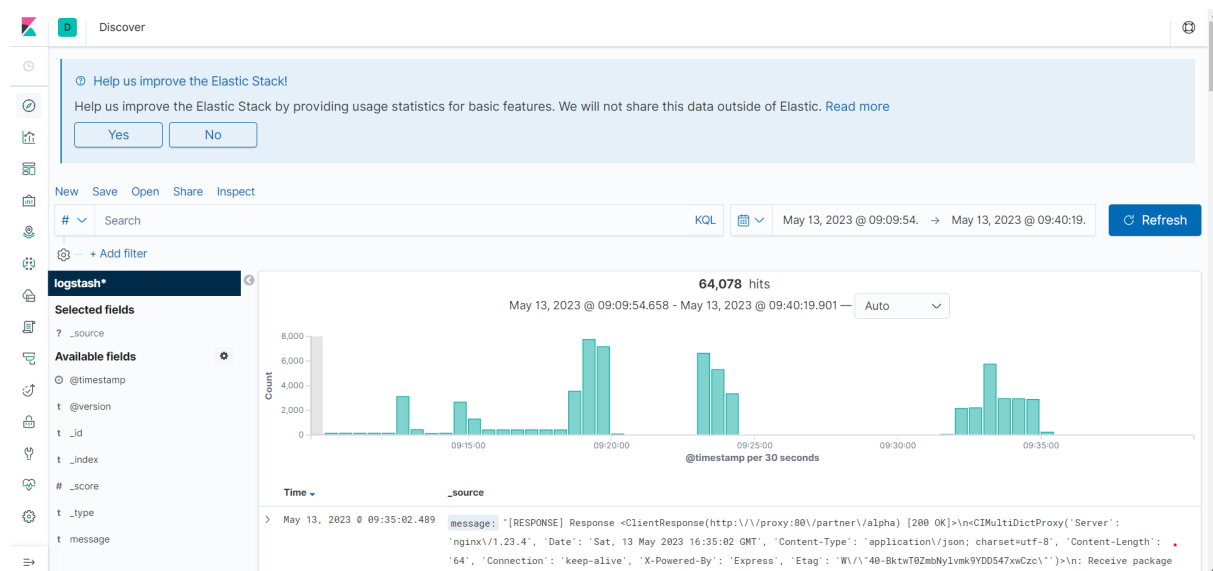
Hệ thống sử dụng ELK stack để phục vụ hai mục đích sau:

- Xem log thời gian xử lý và gửi message đến các đối tác.
- Trong một khoảng thời gian, hệ thống gửi thành công được bao nhiêu message đến đối tác.

ELK stack được cài đặt bằng Docker đã được triển khai ở mục 4.1. Sau khi nhận response từ hệ thống đối tác, các response này sẽ được ghi log dưới dạng JSON và được Producer gửi đến Kafka topic. Logstash sau đó sẽ đọc các dữ liệu từ

Kafka topic, áp dụng các bộ lọc và chuẩn hóa dữ liệu trước khi gửi chúng tới Elasticsearch để lưu trữ. Dưới đây là file config logstash với input là Kafka, output là Elasticsearch được sử dụng trong hệ thống.

Sau khi cài đặt xong, ta có thể truy cập vào Kibana để truy vấn và hiển thị log như hình dưới đây:



Hình 4.3: Sử dụng Kibana để trực quan hóa dữ liệu

Từ bảng điều khiển và biểu đồ để hiển thị trên, ta có thể xem các thông tin như số lượng request thành công và thất bại trong một khoảng thời gian, thời gian phản hồi trung bình,... Nhận thấy, với khoảng thời gian cao tải trong ngày như buổi sáng, trong khoảng thời gian 30 giây số response tối đa trả về dao động từ 2000-8000, ta suy ra được trong 1 giây số response trả về thành công dao động từ 230-300. Thông lượng lớn có thể đạt đến 21,600,000 req/ngày. Con số này cho thấy đã thỏa mãn với lượng thông lượng mà mục tiêu khóa luận mong muốn.

4.2.2 Thống kê message trong Apache Kafka với Grafana, Prometheus và Exporter

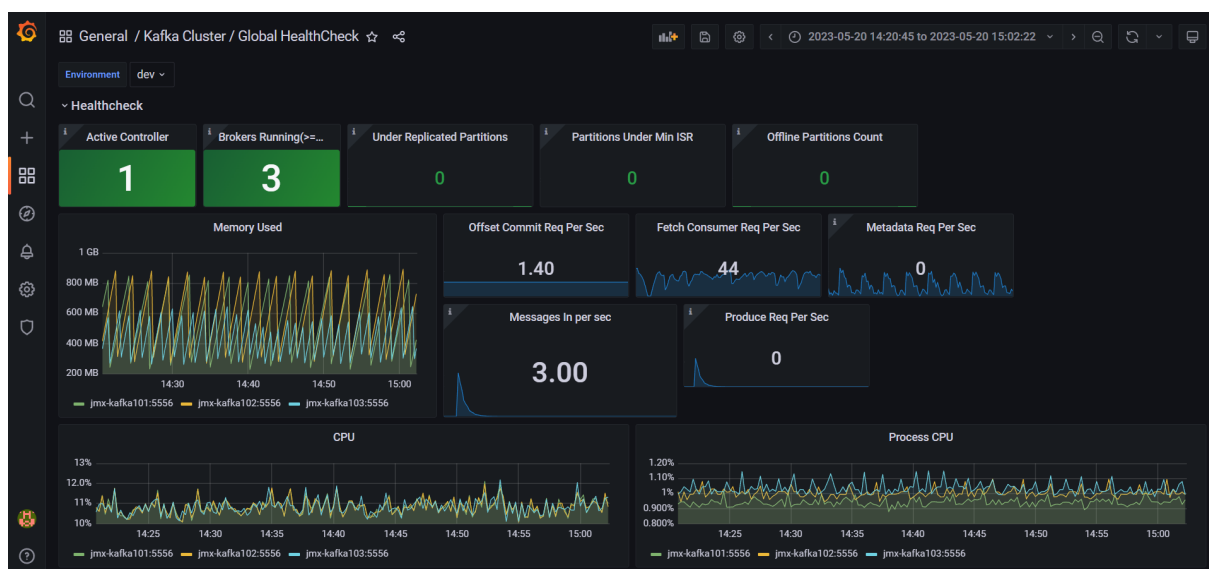
Để có thể theo dõi các thông số như message lag, số lượng message xử lý trong một giây, số lượng message trong một topic, partition, tình trạng của các consumer group,...ta có thể sử dụng Prometheus, Grafana và Exporter để giám sát các metrics của Kafka và hiển thị chúng trên các dashboard trực quan. Các thông tin này sẽ

giúp ta có cái nhìn tổng quan về tình trạng của Kafka và giúp phát hiện các vấn đề nhanh chóng.

Triển khai vào hệ thống

Để giám sát Kafka bằng Prometheus, đầu tiên Kafka Exporter, một công cụ thu thập và xuất các metrics của Kafka dưới dạng dữ liệu tương thích với Prometheus. Sau khi cài đặt Kafka Exporter, ta cấu hình Prometheus để thu thập các metric từ Kafka Exporter và lưu trữ chúng trong cơ sở dữ liệu Prometheus. Prometheus cung cấp một giao diện đồ họa cho phép ta truy vấn các metrics và tạo các biểu đồ, bảng điều khiển để hiển thị các thông tin đó.

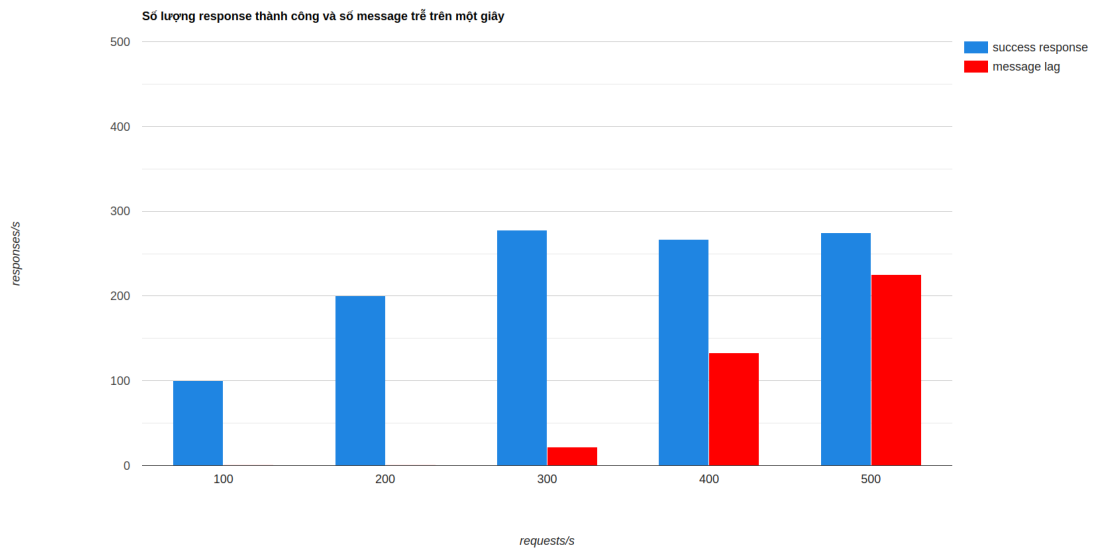
Sau khi đã thu thập các metrics của Kafka bằng Prometheus, ta có thể sử dụng Grafana để hiển thị các metrics đó trên các dashboard trực quan.



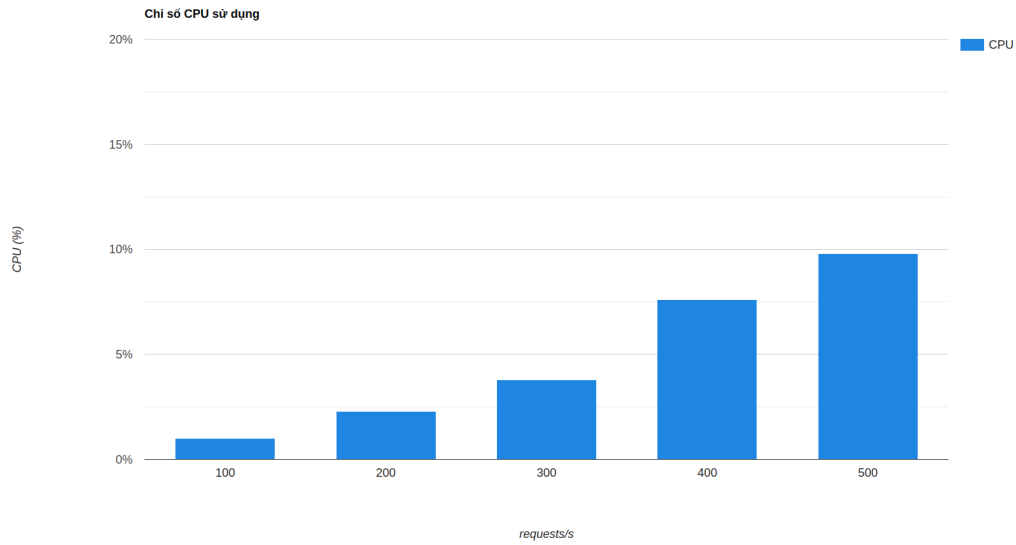
Hình 4.4: Trực quan hóa dữ liệu message bằng Grafana

Từ hình trên, ta có thể theo dõi các thông số như số lượng message được gửi trong từng topic (Messages In Per Topics), số lượng message lag trong từng consumer group (Consumer Group Lag), số lượng message được producer gửi đến (Produce Req Per Sec), số lượng broker đang chạy (Broker Running),... Một trong các thông số quan trọng là số lượng message lag trong consumer, thông số này dùng để mô tả sự khác biệt giữa message mới nhất trong Kafka topic và message đang được consumer xử lý. Độ trễ message (lag) càng nhỏ thì mức xử lý message real-time càng cao.

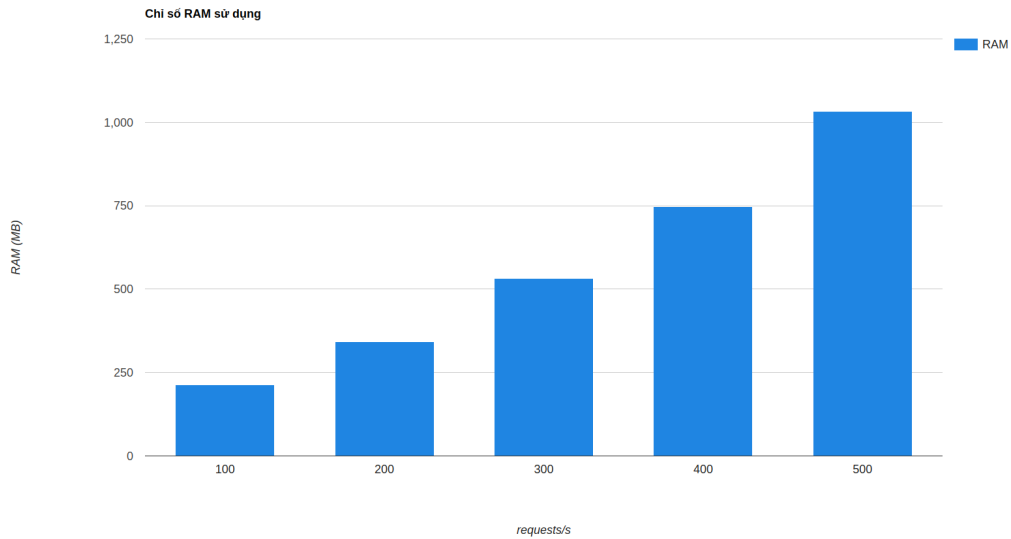
Ngoài ra, các thông số CPU, RAM, số lần đọc/ghi file của các Kafka Exporter cũng được hiển thị trên dashboard. Dựa vào các chỉ số này, một kịch test hiệu năng (performance testing) để đánh giá tải của hệ thống (CPU, RAM) dưới một tải lượng lớn là số lượng request tăng dần. Mục tiêu của việc thực hiện test hiệu năng là đo lường và xác định được khả năng của hệ thống trong một môi trường thực tế, đảm bảo rằng nó có thể hoạt động một cách ổn định và hiệu quả trong điều kiện tải tối đa. Kịch bản được xây dựng bằng cách tăng dần số request/s để đánh giá chỉ số CPU, RAM tiêu thụ, số lượng message thành công và số message trễ trên mỗi giây.



Hình 4.5: Số lượng response thành công và số message trễ trên một giây tương ứng với số request/s



Hình 4.6: Chỉ số CPU sử dụng tương ứng với số request/s



Hình 4.7: Chỉ số RAM sử dụng tương ứng với số request/s

Theo số liệu thống kê, có thể nhận thấy hệ thống hoạt động ổn định và hiệu quả trong điều kiện tải tối đa là 300 request/s. Khi này số lượng message xử lý thành công đạt đến gần 300 response và vài chục message trễ. Khi tăng số request/s lên

thì số message trễ tăng rất nhanh và số lượng response thành công dường như đã chững lại ở con số 300. Chỉ số CPU và RAM khi nhận 300 request/s tương ứng là xấp xỉ 4% và xấp xỉ 500MB, khi tăng số lượng request thì khối lượng công việc cần xử lý bởi CPU và RAM cũng tăng lên. CPU sẽ phải xử lý nhiều yêu cầu hơn và RAM sẽ phải lưu trữ thêm dữ liệu. Nếu không được quản lý và tối ưu tốt, điều này có thể dẫn đến giảm hiệu suất và ảnh hưởng đến hệ thống.

Kết luận

Kết luận

Khóa luận đã xây dựng một hệ thống hoàn toàn tự động giúp theo dõi trạng thái của đơn hàng được cập nhật theo thời gian thực, giúp cho các công ty chuyên về lĩnh vực vận chuyển hàng hóa tối ưu hơn trong chuỗi cung ứng hàng hóa, giảm thiểu thời gian vận chuyển, giảm thiểu việc mất hoặc hư hỏng đơn hàng, cung cấp một dịch vụ tin cậy đến người dùng hơn. Hệ thống được chia làm 2 hệ thống nhỏ hơn đó là: Hệ thống bắt sự thay đổi dữ liệu từ data source và hệ thống xử lý webhook.

Đối với hệ thống nắm bắt sự thay đổi dữ liệu từ data source, khóa luận đã sử dụng kỹ thuật CDC và công cụ Debezium để thực hiện CDC trong môi trường cơ sở dữ liệu, giúp theo dõi các thay đổi trong cơ sở dữ liệu và cung cấp thông tin về chúng thông qua các sự kiện (events) và gửi đến Apache Kafka.

Đối với hệ thống xử lý webhook, những dữ liệu đã được thay đổi ngay lập tức được đưa đến hệ thống này. Tại đây, dữ liệu sẽ được lọc ra các trường cần thiết để gửi sang cho đối tác. Mỗi đối tác sẽ cần những thông tin khác nhau để sử dụng trong hệ thống của họ. Khóa luận cũng đã áp dụng xử lý concurrency để giải quyết nhiều tác vụ cùng lúc khi có nhiều request trong thời điểm cao tải.

Ngoài ra, khóa luận cũng đã sử dụng Kafka làm Message Broker có nhiệm vụ nhận, lưu trữ và truyền tải thông điệp (message) giữa hai hệ thống với nhau. Với vai trò này, Kafka giúp cho hai hệ thống có thể liên lạc với nhau một cách tin cậy, đồng bộ và hiệu quả hơn trong môi trường phân tán. Ngoài ra, Message Broker còn hỗ trợ các tính năng như đảm bảo tính toàn vẹn, độ tin cậy và độ trễ của thông điệp trong quá trình truyền tải.

Kết quả đạt được là hệ thống giúp cho các đối tác có thể quản lý, theo dõi đơn hàng hiệu quả, cung cấp thông tin về đơn hàng theo thời gian thực và chính xác về trạng thái đơn hàng cho đối tác. Lượng throughput lớn có thể đạt đến 1500000 đơn hàng trên ngày. Tuy nhiên, khóa luận yêu cầu một tài nguyên lớn hoặc chi phí phát triển và vận hành cao.

Hướng phát triển

Trong tương lai, mong muốn phát triển hệ thống mở rộng để có thể là một dịch vụ được sử dụng bởi công ty **Giao hàng tiết kiệm**, khóa luận đề xuất bổ sung các tính năng và tối ưu hóa hệ thống như sau:

- **Bổ sung hệ thống cảnh báo khi có nhiều message lag:** Điều này để giúp theo dõi và xử lý kịp thời các vấn đề liên quan đến trễ message và cải thiện độ tin cậy của hệ thống. Hệ thống cảnh báo message lag Kafka là một phần quan trọng của quản lý và giám sát Kafka cluster, giúp đảm bảo hoạt động của hệ thống luôn được ổn định và đáp ứng được nhu cầu sử dụng.
- **Bổ sung luồng webhook thủ công:** Luồng này cho phép các nhân viên chăm sóc khách hàng ghi nhận và cập nhật các thông tin liên quan đến đơn hàng một cách thủ công đến đối tác trong trường hợp một đơn hàng không thể gửi đến đối tác sau khi đã gửi lại (retry) nhiều lần hoặc khi hệ thống gặp sự cố.
- **Bổ sung tracking người giao hàng:** Ngoài tracking đơn hàng, việc tracking người giao hàng để quản lý tình trạng và lịch trình giao hàng một cách hiệu quả hơn. Việc bổ sung tracking người giao hàng trong hệ thống tracking đơn hàng bằng webhook giúp quản lý tình trạng và lịch trình giao hàng một cách hiệu quả hơn. Nhờ đó, nhà bán hàng có thể biết được vị trí chính xác của người giao hàng và theo dõi được tiến độ giao hàng của mỗi đơn hàng. Điều này giúp cải thiện trải nghiệm của khách hàng và giảm thiểu các vấn đề liên quan đến việc vận chuyển hàng hóa. Ngoài ra, việc bổ sung tracking người giao hàng cũng giúp đơn vị vận chuyển quản lý nhân viên của mình một cách chuyên nghiệp và hiệu quả hơn, đảm bảo tính chính xác và độ tin cậy trong việc giao hàng.
- **Sử dụng dịch vụ Schema Registry của Kafka:** Schema Registry là một dịch vụ quản lý và lưu trữ các schema (lược đồ) cho các dữ liệu được gửi và nhận trong hệ thống Kafka sử dụng Avro¹, Protobuf² để serialize (chuyển đổi dữ liệu từ một đối tượng sang bytes) và deserialize (chuyển đổi dữ liệu dạng bytes về đối tượng) dữ liệu. Khi dữ liệu được gửi đi, Avro hay Protobuf sử

¹<https://avro.apache.org/>

²<https://protobuf.dev/>

dụng schema tương ứng để encode dữ liệu. Schema Registry cho phép lưu trữ các schema để các ứng dụng có thể truy cập và sử dụng lại các schema này khi cần thiết, giúp tối ưu hóa việc quản lý schema trong hệ thống sử dụng Avro hoặc Protobuf. Ví dụ khi sử dụng Avro, dữ liệu sẽ được lưu trữ dưới dạng binary và có thể được truyền đi một cách hiệu quả. Do đó, việc sử dụng Avro có thể giúp tối ưu hóa việc truyền tải dữ liệu giữa các ứng dụng giúp giảm độ lớn của các message và giảm thiểu tài nguyên sử dụng của hệ thống.

Tài liệu tham khảo

- [1] B. Jin, S. Sahni, and A. Shevat. *Designing Web APIs: Building APIs That Developers Love*. O'Reilly Media, 2018, pp. 19–21.
- [2] D. Scott, V. Gamov, and D. Klein. *Kafka in Action*. Manning, 2022, pp. 20–24, 58–61, 144–151.
- [3] Neha Narkhede, Gwen Shapira, and Todd Palino. *Kafka: the definitive guide: real-time data and stream processing at scale*. " O'Reilly Media, Inc.", 2017, pp. 45–48, 64–67, 142–152.
- [4] K. Petrie, I. Ankorion, and D. Potter. *Streaming Change Data Capture: A Foundation for Modern Data Architectures*. O'Reilly Media, 2018, pp. 15–18.
- [5] S. Chhajed. *Learning ELK Stack*. Community experience distilled. Packt Publishing, 2015, pp. 24–28.
- [6] Brian Brazil. *Prometheus: Up & Running: Infrastructure and Application Performance Monitoring*. " O'Reilly Media, Inc.", 2018, pp. 102–106, 178–189.
- [7] Liang Hao et al. “Methods for Solving the Change Data Capture Problem”. In: *Advances in Natural Computation, Fuzzy Systems and Knowledge Discovery*. Ed. by Ning Xiong et al. Cham: Springer International Publishing, 2023, pp. 7–8.
- [8] C. Hattingh. *Using Asyncio in Python: Understanding Python's Asynchronous Programming Features*. O'Reilly Media, 2020, pp. 31–42, 50–57, 102–114.
- [9] Loigen Sodian et al. “Concurrency and Parallelism in Speeding Up I/O and CPU-Bound Tasks in Python 3.10”. In: *2022 2nd International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI)*. 2022, pp. 5–6.
- [10] J. Kreibich. *Redis: The Definitive Guide: Data modeling, caching, and messaging*. O'Reilly Media, Incorporated, 2013, pp. 28–33.